
IMPACT User Manual

Release 2.0

Ben Wooding and contributors

Jul 02, 2026

GETTING STARTED

1	Getting Started	3
2	Example Case Studies	5
3	API Modules	7
3.1	Installation	7
3.2	Quick Start	9
3.3	Docker	11
3.4	GPU Setup	12
3.5	Configuration	13
3.6	Verification and Synthesis	15
3.7	Loading and Saving	16
3.8	Makefiles	18
3.9	Feature Manual (v2.0)	19
3.10	Probabilistic Specifications	21
3.11	Expected Rewards	22
3.12	Long-Run Average and Steady-State	24
3.13	ω -Regular and LTL Objectives	25
3.14	Multi-Objective Verification	28
3.15	CTMCs and CSL Time-Bounded Reachability	29
3.16	Parametric Region Verification	31
3.17	Orthogonal and Mixture IMDPs	32
3.18	Exact Rational Solving	34
3.19	Statistical Model Checking	35
3.20	Solver Engines	37
3.21	File Formats and Converters	38
3.22	CLI Reference: <code>imdp_solve</code>	40
3.23	Tool comparison: IMPaCT vs PRISM vs Storm vs IntervalMDP.jl	43
3.24	IMPaCT vs PRISM	46
3.25	IMPaCT vs Storm	47
3.26	IMPaCT vs IntervalMDP.jl	49
3.27	ARCH-COMP Benchmarks	52
3.28	AS — Anaesthesia Delivery (3D)	53
3.29	BA — Building Automation (4D)	55
3.30	VP — Van der Pol Oscillator (2D)	56
3.31	IC — Integrator Chain (2D)	58
3.32	PD — Package Delivery (2D)	60
3.33	PR_minimal — Patrol Robot (2D)	62
3.34	AV_minimal — Autonomous Vehicle (3D)	63
3.35	LM — Lane Merging Vehicle (6D)	65
3.36	MDP (<code>class MDP</code>)	67
3.37	IMDP (<code>class IMDP</code>)	71
3.38	GPU Synthesis	73
3.39	IO Utilities (<code>IMPaCT_IO</code>)	74

3.40	Examples	74
3.41	Citing	80
Index		83

Interval MDP Parallel Construction for Controller Synthesis of Large-Scale Stochastic Systems

IMPACT is an open-source C++ tool for parallelized verification and controller synthesis of large-scale stochastic systems using interval Markov decision processes (IMDPs). Built on [AdaptiveCpp](#) (SYCL) for adaptive parallelism across CPUs and GPUs from all hardware vendors.

GPU Accelerated SYCL-based parallelism via AdaptiveCpp. Runs on CPUs and GPUs from Intel, NVIDIA, and others.

Formal Guarantees Interval iteration algorithms with convergence guarantees for infinite-horizon specifications.

Flexible Specifications Safety, reachability, and reach-while-avoid properties over finite and infinite time horizons.

Scalable Tested from 2D to 14D systems. Parallel construction tackles the state-explosion problem.

GETTING STARTED

Installation Install IMPaCT from source on Linux or macOS, or use Docker.

Installation **Quick Start** Walk through a complete example: define a system, abstract, and synthesize a controller.

Quick Start **Docker** The easiest way to get started — run IMPaCT in a container.

Docker **GPU Setup** Configure GPU acceleration for NVIDIA, Intel, and other hardware.

GPU Setup

EXAMPLE CASE STUDIES

Case Study	Dimensions	Specification	Description
2D Robot	2	Reachability	Mobile robot reaching a target region
2D Robot (Disturbed)	2	Reachability	Robot with additive disturbance
2D Robot Reach-Avoid	2	Reach-Avoid	Reach target while avoiding obstacles
3D Autonomous Vehicle	3	Reach-Avoid	Vehicle navigation with obstacle avoidance
3D Room Temperature	3	Safety	Temperature regulation over finite horizon
4D Building Automation	4	Safety	Multi-room temperature control
7D Building Automation	7	Safety (Verification)	Large-scale IMC verification
14D Stochastic System	14	Safety	High-dimensional scalability demonstration

See all examples ->

API MODULES

MDP Base class for state/input/disturbance spaces, dynamics, and noise distributions.

MDP (class MDP) **IMDP** Interval MDP class for abstraction, synthesis, and controller save/load.

IMDP (class IMPDP) **GPU Synthesis** Sorted synthesis algorithms optimized for GPU execution via SYCL.

GPU Synthesis **IO Utilities** HDF5 save/load functions for matrices, vectors, and controllers.

IO Utilities (IMPACT_IO)

3.1 Installation

IMPACT requires the following dependencies:

- Python 3, CMake, Boost
- [AdaptiveCpp](#) (with clang++ and OpenMP)
- HDF5, [Armadillo](#) (with HDF5 enabled)
- GSL, NLOpt, GLPK

Tip

The easiest way to use IMPACT is via [Docker](#). For Ubuntu 22.04, use the automated install script.

Ubuntu 22 (Recommended)

Run the automated installer:

```
chmod +x install_ubuntu22.sh
sudo ./install_ubuntu22.sh
```

This installs all dependencies and builds IMPACT. See the [installation video](#) for a walkthrough.

Linux (Manual)

Install essential build tools:

```
sudo apt-get install build-essential
```

Python 3 and CMake:

```
sudo apt-get install python3 cmake
```

Boost:

```
sudo apt-get install libboost-all-dev
```

AdaptiveCpp:

See the [official instructions](#) for the most up-to-date guide. A copy is also bundled in the repository:

```
unzip AdaptiveCpp-develop.zip
cd AdaptiveCpp-develop
sudo apt install -y libclang-16-dev clang-tools-16 libomp-16-dev
sudo cmake .
sudo make install
```

If issues occur, add flags such as `-DCMAKE_CXX_COMPILER=/path/to/clang++-16`.

HDF5:

```
sudo apt-get install libhdf5-serial-dev
```

Armadillo:

```
sudo apt install libopenblas-dev liblapack-dev libarpack2-dev libsuperlu-dev
tar -xvf armadillo-12.6.4.tar.xz
cd armadillo-12.6.4
sudo cmake .
sudo make install
```

Important

Enable HDF5 in Armadillo: find the `armadillo_bits` directory (`sudo find / -type d -name 'armadillo_bits'`), edit `config.hpp`, and uncomment the line `ARMA_USE_HDF5`.

GSL:

```
tar -xzf gsl-2.7.1.tar.gz
cd gsl-2.7.1
./configure && make && sudo make install
```

NLOpt:

```
tar -xzf nlopt-2.7.1.tar.gz
cd nlopt-2.7.1 && mkdir build && cd build
cmake .. && make && sudo make install
```

GLPK:

```
sudo apt-get install glpk-utils libglpk-dev
```

macOS**Note**

IMPACT has been tested on macOS with Intel chips. For Apple Silicon (M1/M2+), check [AdaptiveCpp support](#).

Install dependencies via Homebrew:

```
brew install python cmake boost hdf5 gsl nlopt glpk
```

Armadillo:

```
brew install armadillo
```

Enable HDF5 in Armadillo: `find armadillo_bits (sudo find / -type d -name 'armadillo_bits')`, edit `config.hpp`, and uncomment `ARMA_USE_HDF5`.

AdaptiveCpp:

```
unzip AdaptiveCpp-develop.zip
cd AdaptiveCpp-develop
sudo cmake . -DCMAKE_INSTALL_PREFIX=/usr/local \
             -DCMAKE_CXX_COMPILER=/path/to/clang++ \
             -DOpenMP_ROOT=/path/to/libomp/include
sudo make install
```

Docker

See [Docker](#) for container-based installation — no local dependencies needed.

3.1.1 Verification

After installation, verify by building and running an example:

```
cd examples/ex_2Drobot-R-U
make
./robot2D
```

You should see output showing the abstraction and synthesis progress.

3.2 Quick Start

This guide walks through a complete example: synthesizing a controller for a 2D robot to reach a target region. This corresponds to the `ex_2Drobot-R-U` example.

3.2.1 Prerequisites

Ensure IMPACT is installed (see [Installation](#)) or use [Docker](#).

3.2.2 Configuration File

Each IMPACT case study is defined in a C++ configuration file. Create a file (e.g., `robot2D.cpp`) with the following structure.

1. Include headers and set dimensions:

```
#include "../src/IMDP.h"
#include <armadillo>

using namespace std;
using namespace arma;

const int dim_x = 2; // state dimensions
const int dim_u = 2; // input dimensions
const int dim_w = 0; // disturbance dimensions (0 = none)
```

2. Define the spaces:

Each space is a hyper-rectangle defined by lower bounds, upper bounds, and discretization step sizes:

```
// State space: [-10, 10] x [-10, 10] with step size 1
const vec ss_lb = {-10, -10};
const vec ss_ub = {10, 10};
const vec ss_eta = {1, 1};

// Input space
const vec is_lb = {-1, -1};
const vec is_ub = {1, 1};
const vec is_eta = {0.2, 0.2};
```

3. Define noise and target region:

```
// Standard deviation of Gaussian noise per dimension
const vec sigma = {sqrt(1/1.3333), sqrt(1/1.3333)};

// Target region: square from (5,5) to (8,8)
auto target_condition = [](const vec& ss) {
    return (ss[0] >= 5.0 && ss[0] <= 8.0) &&
           (ss[1] >= 5.0 && ss[1] <= 8.0);
};
```

4. Define dynamics:

```
auto dynamics = [](const vec& x, const vec& u) -> vec {
    vec xx(dim_x);
    xx[0] = x[0] + 2*u[0]*cos(u[1]);
    xx[1] = x[1] + 2*u[0]*sin(u[1]);
    return xx;
};
```

5. Main function — create, abstract, and synthesize:

```
int main() {
    // Create IMPDP object
    IMPDP mdp(dim_x, dim_u, dim_w);

    // Set up spaces
    mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
    mdp.setInputSpace(is_lb, is_ub, is_eta);
    mdp.setTargetSpace(target_condition, true);

    // Set dynamics and noise
    mdp.setDynamics(dynamics);
    mdp.setNoise(NoiseType::NORMAL);
    mdp.setStdDev(sigma);

    // Compute abstraction
    mdp.targetTransitionVectorBounds();
    mdp.minAvoidTransitionVector();
    mdp.maxAvoidTransitionVector();
    mdp.transitionMatrixBounds();

    // Synthesize controller (true = pessimistic)
    mdp.infiniteHorizonReachController(true);

    // Save results
    mdp.saveController();
}
```

(continues on next page)

(continued from previous page)

```
    return 0;
}
```

3.2.3 Build and Run

```
cd examples/ex_2Drobot-R-U
make
./robot2D
```

The tool will output progress for the abstraction and synthesis steps. Results are saved as HDF5 files in the same directory.

Tip

Suppress debug output with `export ACPP_DEBUG_LEVEL=0` before running.

3.2.4 Output Files

IMPACT saves results in HDF5 format (.h5 files):

- `ss.h5` — state space
- `is.h5` — input space
- `ts.h5` — target space
- `minTM.h5`, `maxTM.h5` — transition matrix bounds
- `controller.h5` — synthesized controller

These can be loaded in MATLAB, Python, or R for visualization. See the `misc/` folder for plotting utilities.

3.2.5 What's Next?

- *Configuration* — Full guide to defining systems
- *Verification and Synthesis* — Synthesis algorithms and options
- *Examples* — All example case studies
- *IMDP (class IMPDP)* — Complete IMPDP API reference

3.3 Docker

The easiest way to use IMPACT is via Docker — no local dependency installation required.

3.3.1 Building the Image

```
docker build -t impact .
```

The first build may take a while as it compiles all dependencies.

3.3.2 Running the Container

```
docker run --rm -it --name impact impact
```

This opens a shell inside the container. Run examples with:

```
cd /app/examples/ex_2Drobot-R-U
make
./robot2D
```

3.3.3 Mounting a Volume

To use your own examples or persist results, mount a local directory:

```
docker run --rm -it --name impact -v ~/my-examples:/app/examples impact
```

Files are shared between the container and host, so results appear in `~/my-examples`.

Warning

Files created inside the container (not in a mounted volume) are lost when the container stops.

3.3.4 Copying Results

Alternatively, copy files out of a running container:

```
docker cp impact:/app/examples/ex_2Drobot-R-U/controller.h5 .
```

3.4 GPU Setup

IMPACT supports GPU-accelerated synthesis via AdaptiveCpp's SYCL implementation. GPU acceleration applies to the **sorted synthesis algorithms** — abstraction is computed on the CPU and loaded from file.

3.4.1 Prerequisites

Install the base IMPACT dependencies first (see [Installation](#)), then add GPU-specific libraries.

Identify your GPU:

```
lspci | grep -i vga
```

LLVM

```
wget https://apt.llvm.org/llvm.sh
chmod +x llvm.sh
sudo ./llvm.sh 16
sudo apt install -y libclang-16-dev clang-tools-16 libomp-16-dev llvm-16-dev
```

CUDA (NVIDIA)

```
sudo apt install nvidia-cuda-toolkit
```

Verify with `nvcc --version`.

3.4.2 Rebuild AdaptiveCpp with GPU Support

After installing GPU libraries, rebuild AdaptiveCpp with the appropriate flags:

```
cd AdaptiveCpp-develop
sudo cmake . -DCUDAToolkit_LIBRARY_ROOT=/usr/lib/cuda
sudo make install
```

See the [AdaptiveCpp documentation](#) for other GPU backends (Intel, AMD).

3.4.3 Makefile Changes

To use GPU synthesis, update your example's Makefile:

1. Change the target from `--acpp-targets="omp"` to your GPU target (e.g., `cuda:sm_70` for NVIDIA V100).
2. Replace `IMDP.cpp` with `GPU_synthesis.cpp` in the source files.

```
CC = acpp
CFLAGS = --acpp-targets="cuda:sm_70" -O3 ...
SRCS = example.cpp ../../src/GPU_synthesis.cpp ../../src/MDP.cpp
```

Note

GPU synthesis requires pre-computed abstraction matrices loaded from file. Compute the abstraction on CPU first, save to HDF5, then load for GPU synthesis. See [Loading and Saving](#) for details.

3.5 Configuration

Each IMPaCT case study is defined in a C++ configuration file (e.g., `robot2D.cpp`). This page explains every configurable component.

3.5.1 IMDP Object

Create an IMDP object specifying the dimensions of each space:

```
IMDP imdp(dim_x, dim_u, dim_w);
```

- `dim_x` — state space dimensions
- `dim_u` — input space dimensions (set to 0 for IMC verification)
- `dim_w` — disturbance space dimensions (set to 0 if none)

When `dim_u = 0`, IMPaCT automatically treats the problem as IMC verification and returns satisfaction probabilities instead of a controller.

3.5.2 State, Input, and Disturbance Spaces

Each space is a hyper-rectangle defined by three vectors of length equal to the number of dimensions:

```
void setStateSpace(vec lb, vec ub, vec eta);
void setInputSpace(vec lb, vec ub, vec eta);
void setDisturbSpace(vec lb, vec ub, vec eta);
```

- `lb` — lower bound of each dimension
- `ub` — upper bound of each dimension
- `eta` — discretization step size of each dimension

If input or disturbance spaces are not present, simply omit the call.

Example:

```
const vec ss_lb = {-10, -10};
const vec ss_ub = {10, 10};
const vec ss_eta = {1, 1};
imdp.setStateSpace(ss_lb, ss_ub, ss_eta);
```

3.5.3 Target and Avoid Regions

By default, all states are considered safe. For reachability or reach-avoid specifications, define target and/or avoid regions using boolean functions:

```
void setTargetSpace(const function<bool(const vec*)>& condition, bool remove);
void setAvoidSpace(const function<bool(const vec*)>& condition, bool remove);
void setTargetAvoidSpace(
    const function<bool(const vec*)>& target_condition,
    const function<bool(const vec*)>& avoid_condition,
    bool remove);
```

Set `remove = true` to move matching states out of the state space (typical usage). Set `remove = false` to keep them in the state space (useful for counting states).

Example:

```
// Target: square region from (5,5) to (8,8)
auto target_condition = [](const vec& ss) {
    return (ss[0] >= 5.0 && ss[0] <= 8.0) &&
           (ss[1] >= 5.0 && ss[1] <= 8.0);
};
imdp.setTargetSpace(target_condition, true);
```

3.5.4 Dynamics

Define system dynamics as a lambda function. The number of parameters must match the IMPDP object dimensions:

```
// 3 parameters: state, input, disturbance
void setDynamics(function<vec(const vec&, const vec&, const vec&)> d);

// 2 parameters: state, input (no disturbance)
void setDynamics(function<vec(const vec&, const vec&)> d);

// 1 parameter: state only (autonomous system)
void setDynamics(function<vec(const vec&)> d);
```

IMPACT detects the number of parameters automatically.

Example:

```
auto dynamics = [](const vec& x, const vec& u) -> vec {
    vec xx(dim_x);
    xx[0] = x[0] + 2*u[0]*cos(u[1]);
    xx[1] = x[1] + 2*u[0]*sin(u[1]);
    return xx;
};
imdp.setDynamics(dynamics);
```

3.5.5 Noise Distributions

IMPACT supports three noise configurations:

Normal distribution with diagonal covariance (default):

```
imdp.setNoise(NoiseType::NORMAL);
imdp.setStdDev(sigma); // vec of standard deviations
```

Uses the closed-form CDF product, which is the fastest option.

Multivariate normal with full covariance:

```
imdp.setNoise(NoiseType::NORMAL, false, 1000);
imdp.setInvCovDet(inv_cov, det); // inverse covariance + determinant
```

Uses Monte Carlo integration with the specified number of samples.

Custom distribution:

Edit `src/custom.cpp` to define your PDF, then:

```
imdp.setNoise(NoiseType::CUSTOM, false, 1000);
```

The custom PDF receives a `customParams` struct with access to the current state, dynamics, input, disturbance, integration bounds, and discretization parameter.

3.5.6 Optimization Algorithm

The nonlinear optimization algorithm for abstraction can be changed:

```
imdp.setAlgorithm(nlopt::LN_SBPLX); // default
```

See the [NLOpt documentation](#) for other algorithms (e.g., LN_COBYLA).

3.5.7 Stopping Condition

Set the convergence threshold for infinite-horizon synthesis:

```
imdp.setStoppingCondition(0.00001); // default
```

3.6 Verification and Synthesis

After configuring the system and computing the abstraction, IMPACT performs formal verification or controller synthesis using interval iteration algorithms.

3.6.1 Abstraction

The abstraction computes transition probability bounds for every state-to-state pair. These are always required:

```
// Transition matrices (state-to-state)
imdp.minTransitionMatrix();
imdp.maxTransitionMatrix();

// Avoid transition vectors (state-to-outside)
imdp.minAvoidTransitionVector();
imdp.maxAvoidTransitionVector();
```

For reachability or reach-avoid specifications, also compute target vectors:

```
imdp.minTargetTransitionVector();
imdp.maxTargetTransitionVector();
```

Low-Cost Abstraction

For sparse systems, a computational trick can speed up the abstraction: if the maximal transition probability is zero, the minimal must also be zero. Use these functions to exploit this:

```
imdp.transitionMatrixBounds(); // replaces separate min/max calls
imdp.targetTransitionVectorBounds(); // replaces separate min/max calls
```

The sparser the system, the greater the speedup.

3.6.2 Controller Synthesis

IMPACT provides synthesis for two specification types across two time horizons.

All functions take a boolean `IMDP_lower` parameter:

- `true` — **pessimistic** strategy (worst-case noise, lower bound probabilities)
- `false` — **optimistic** strategy (best-case noise, upper bound probabilities)

Infinite-Horizon

Uses the **interval iteration** algorithm with convergence guarantees:

```
imdp.infiniteHorizonReachController(true); // reachability / reach-avoid
imdp.infiniteHorizonSafeController(true); // safety
```

Reach-avoid is detected automatically when an avoid region has been defined.

Finite-Horizon

Uses **value iteration** for a specified number of time steps:

```
imdp.finiteHorizonReachController(true, 10); // 10 time steps
imdp.finiteHorizonSafeController(true, 10);
```

3.6.3 Sorted Synthesis (GPU-Compatible)

Sorted variants remove the dependency on the GLPK library and enable GPU execution:

```
imdp.infiniteHorizonReachControllerSorted(true);
imdp.infiniteHorizonSafeControllerSorted(true);
imdp.finiteHorizonReachControllerSorted(true, 10);
imdp.finiteHorizonSafeControllerSorted(true, 10);
```

These functions are implemented in `src/GPU_synthesis.cpp`. To use them on a GPU, update the Makefile (see *GPU Setup*).

Note

GPU synthesis requires pre-computed abstraction matrices loaded from HDF5 files. Compute the abstraction on CPU first, then load for GPU synthesis. See *Loading and Saving* for save/load details.

3.6.4 IMC Verification

When the input dimension is 0 (no control inputs), IMPACT automatically performs **verification** instead of synthesis. The same functions are used — the output is a lookup table of satisfaction probabilities rather than a controller.

Example:

```
IMDP imdp(7, 0, 0); // 7D system, no inputs, no disturbance
// ... set state space, dynamics, noise ...
imdp.infiniteHorizonSafeController(true); // returns probabilities
```

3.7 Loading and Saving

IMPACT uses **HDF5** for all data persistence. This enables interoperability with MATLAB, Python (h5py), R (rhdf5), and other tools. The HDF5 field name is `dataset` by default.

3.7.1 Saving Data

Save any component after it has been computed:

```
// Spaces
imdp.saveStateSpace();           // -> ss.h5
imdp.saveInputSpace();           // -> is.h5
imdp.saveDisturbSpace();         // -> ws.h5
imdp.saveTargetSpace();          // -> ts.h5
imdp.saveAvoidSpace();           // -> as.h5

// Transition matrices and vectors
imdp.saveMinTransitionMatrix();   // -> minTM.h5
imdp.saveMaxTransitionMatrix();   // -> maxTM.h5
imdp.saveMinTargetTransitionVector(); // -> minTV.h5
imdp.saveMaxTargetTransitionVector(); // -> maxTV.h5
imdp.saveMinAvoidTransitionVector(); // -> minAV.h5
imdp.saveMaxAvoidTransitionVector(); // -> maxAV.h5

// Controller
imdp.saveController();           // -> controller.h5
```

3.7.2 Loading Data

Load pre-computed data from HDF5 files:

```
// Spaces
imdp.loadStateSpace("ss.h5");
imdp.loadInputSpace("is.h5");
imdp.loadDisturbSpace("ws.h5");
imdp.loadTargetSpace("ts.h5");
imdp.loadAvoidSpace("as.h5");

// Transition matrices and vectors
imdp.loadMinTransitionMatrix("minTM.h5");
imdp.loadMaxTransitionMatrix("maxTM.h5");
imdp.loadMinTargetTransitionVector("minTV.h5");
imdp.loadMaxTargetTransitionVector("maxTV.h5");
imdp.loadMinAvoidTransitionVector("minAV.h5");
imdp.loadMaxAvoidTransitionVector("maxAV.h5");

// Controller
imdp.loadController("controller.h5");
```

This is useful for:

- **GPU synthesis:** compute abstraction on CPU, load on GPU for synthesis
- **Data-driven methods:** load transition matrices computed externally
- **Resuming computation:** save intermediate results and continue later

Tip

The `misc/` folder contains MATLAB, Python, and R utilities for reading HDF5 files and plotting results.

3.7.3 Debugging Tips

Warning

If synthesis diverges to values greater than 1, your loaded matrices may need to be **transposed**. Use tools like MATLAB's `h5disp` and `h5read` to verify dimensions before loading.

Warning

If the controller shows both upper and lower bounds as zero, this indicates an **infeasible solution** — typically because a minimal value exceeds a maximal value somewhere. Verify that $\max \geq \min$ for all entries.

3.8 Makefiles

Each IMPACT example uses a Makefile to compile the configuration file with all dependencies. Understanding the Makefile structure helps when creating new examples or troubleshooting build issues.

3.8.1 Structure

A typical Makefile compiles a command equivalent to:

```
acpp robot2D.cpp ../../src/IMDP.cpp ../../src/MDP.cpp \
--acpp-targets="omp" -O3 \
-lnlopt -lm \
-I/usr/include/hdf5/serial \
-L/usr/lib/x86_64-linux-gnu/hdf5/serial \
-lhdf5 -lglpk -lgs1 -lgslcblas \
-DH5_USE_110_API -larmadillo \
-o robot2D
```

3.8.2 Key Components

Compiler: `acpp` (AdaptiveCpp). Also works with `syclcc` in some cases.

Targets: `--acpp-targets="omp"` selects OpenMP as the parallelism backend. For GPU, replace with your target (e.g., `cuda:sm_70`).

Source files:

- Your configuration file (e.g., `robot2D.cpp`)
- `../../src/IMDP.cpp` and `../../src/MDP.cpp` — core library files
- For GPU: replace `IMDP.cpp` with `GPU_synthesis.cpp`

Linker flags (`-l`):

- `-lnlopt` — NLOpt optimization
- `-lm` — math library
- `-lhdf5` — HDF5 data storage
- `-lglpk` — GLPK linear programming (not needed for sorted synthesis)

- `-lgsl -lgslcblas` — GNU Scientific Library
- `-larmadillo` — Armadillo linear algebra

Include/Library paths (`-I`, `-L`):

Explicit paths to HDF5 headers and libraries. Adjust these if HDF5 is installed in a different location on your system.

Preprocessor: `-DH5_USE_110_API` selects the HDF5 1.10 API.

Optimization: `-O3` enables maximum compiler optimization.

3.8.3 Creating a New Example

1. Create a new directory under `examples/`.
2. Copy a Makefile from an existing example.
3. Update the source file name in the Makefile.
4. Ensure the relative paths to `../src/` are correct.
5. Run `make` and then `./your_executable`.

3.8.4 Troubleshooting

- **Library not found:** add `-I` and `-L` flags pointing to the correct paths.
- **Compiler issues:** try `sycc` instead of `acpp`, or vice versa.
- **Debug output:** set `export ACPP_DEBUG_LEVEL=0` to suppress AdaptiveCpp warnings.
- See the [Makefile guide](#) for general Makefile help.

3.8.5 Videos

- [Configuration file walkthrough](#)
- [Makefile setup walkthrough](#)

3.9 Feature Manual (v2.0)

This manual documents every verification and synthesis feature of IMPACT v2.0 beyond the abstraction front end: what each feature computes (with precise robust semantics), the literature algorithm it implements, a validated CLI example, and how it compares to the peer tools **PRISM 4.8.1**, **Storm 1.13.0**, and **IntervalMDP.jl 0.6**.

Everything here runs through `tools/indp_solve`, the dependency-free CLI over the neutral `.indp` exchange format (see [File Formats and Converters](#) and [CLI Reference: indp_solve](#)). Every numeric value quoted in this manual was produced by a live run against the models in `benchmarks/crosstool/models/` and cross-checked against the reference values in the matching `*.ref.json` files; cross-tool methodology is in `TOOL_COMPARISONS.md` and `FEATURE_PARITY.md` at the repository root.

3.9.1 Conventions

- **pess** (pessimistic / robust): the controller maximises the objective while nature resolves every transition-probability interval *adversarially* — the value a controller can *guarantee* for all distributions within the intervals.
- **opt** (optimistic): nature resolves the intervals *cooperatively* — the best case.
- Solvers return a **sound bracket** `[lower, upper]` with `gap <= 2*eps` (default `eps = 1e-6`); finite-horizon dynamic programming is exact.
- The shared inner solve for all robust Bellman backups is **O-maximization** (Givan, Leach, Dean, *Bounded-parameter MDPs*, AIJ 2000; Lahijanian, Andersson, Belta, IEEE TAC 2015), `src/omaximization.h`.

3.9.2 Feature overview

Feature	Page	Algorithm	Key literature	Peer support
Reachability, safety, reach-avoid; bounded $P[F \leq k]$; PCTL next/until	<i>Probabilistic Specifications</i>	OVI / interval iteration over O-max	Hartmanns-Kaminski CAV 2020; Baier et al. CAV 2017; Haddad-Monmege TCS 2018; Hansson-Jonsson FAC 1994; Puggelli et al. CAV 2013	all four tools (validated equal)
Expected reward (reachability, discounted)	<i>Expected Rewards</i>	robust dynamic programming	Iyengar Math. OR 2005; Nilim & El Ghaoui Oper. Res. 2005	PRISM (reach), IntervalMDP.jl (discounted); Storm partial
Long-run average, steady-state	<i>Long-Run Average and Steady-State</i>	two-phase VI (MEC gain + cash-out)	Ashok et al. CAV 2017; Chatterjee et al. IJCAI 2024; Puterman 1994 Ch. 8	Storm/PRISM point-only; interval LRA is IMPaCT-only
ω -regular (Büchi, persistence, patrol) and arbitrary LTL	<i>ω-Regular and LTL Objectives</i>	robust accepting-EC + Spot product	de Alfaro 1997; Sickert et al. CAV 2016; Hahn et al. TACAS 2020; Duret-Lutz et al. (Spot 2.0)	IMPaCT-only on intervals
Multi-objective / Pareto	<i>Multi-Objective Verification</i>	weighted-sum scalarised VI	Etessami et al. TACAS 2007; Forejt et al. TACAS 2011	PRISM/Storm point-only
CTMC + CSL time-bounded; interval CTMC	<i>CTMCs and CSL Time-Bounded Reachability</i>	uniformisation + Fox-Glynn	Baier et al. IEEE TSE 2003; Fox-Glynn CACM 1988; Katoen et al. CAV 2007	PRISM/Storm (point rates); interval CTMC is IMPaCT-only
Parametric region verification	<i>Parametric Region Verification</i>	parameter lifting $\rightarrow$ interval MDP	Quatmann et al. ATVA 2016	Storm (storm-pars)
Orthogonal and mixture IMDPs	<i>Orthogonal and Mixture IMDPs</i>	recursive per-dimension O-max	Mathiesen, Haesaert, Laurenti arXiv:2411.11803	IntervalMDP.jl (validated to 10 digits)
Exact rational solving with certificate	<i>Exact Rational Solving</i>	robust policy iteration over rationals	Iyengar 2005 §3.3; Puterman 1994; Haddad-Monmege TCS 2018	PRISM/Storm exact are point-only ; interval exact is IMPaCT-only
Statistical model checking	<i>Statistical Model Checking</i>	Monte-Carlo + Wilson CI + SPRT	Younes-Simmons CAV 2002; Hérault et al. VMCAI 2004; Wald 1945	PRISM (-sim), Storm
Solver engines and method selection	<i>Solver Engines</i>	II / VI / OVI; MEC collapse	Baier et al. CAV 2017; Haddad-Monmege TCS 2018; Hartmanns-Kaminski CAV 2020	—
File formats and converters	<i>File Formats and Converters</i>	.imdp, .odimdp, .pimdp, DRN/PRISM/tr converters	—	interchange with all peers
Complete CLI reference	<i>CLI Reference: imdp_solve</i>	imdp_solve — every property and flag	—	—

3.9.3 Scope notes

- The **abstraction** of continuous stochastic dynamics into an IMPDP (the core of the IMPACT tool paper, QEST+FORMATS 2024) is documented in the User Guide; this manual covers the *solving* layer that the abstraction feeds (IMPDP: `exportIMPDP` bridges the two, see [File Formats and Converters](#)).
- Known limitations are tracked as numbered issues in `issues/INDEX.md` and are cited inline on each page (e.g. ISSUE-0003, ISSUE-0010, ISSUE-0016, ISSUE-0022, ISSUE-0023).

3.10 Probabilistic Specifications

Reachability, safety, reach-avoid, step-bounded reachability, and the PCTL path operators on interval MDPs (IMDPs).

3.10.1 What is computed

For an IMPDP, the controller maximises the probability of the path property while *nature* picks, at every step, a transition distribution within the per-successor probability intervals:

- **pessimistic** (`--bound pess`, default): nature is adversarial. The result is the *robust* value — a guarantee that holds for every distribution in the intervals.
- **optimistic** (`--bound opt`): nature is cooperative — the best-case value.

All unbounded properties return a sound sandwich $\text{lower} \leq V^*(s) \leq \text{upper}$ with $\text{gap} \leq 2 \cdot \text{eps}$; finite-horizon (`--horizon k`) properties are computed by exact dynamic programming (no gap). Properties:

Property	Semantics
<code>reach L</code>	$\max P(F L)$ — eventually reach a state labelled L
<code>safety L</code>	$\max P(G !L) = 1 - \min P(F L)$ — never reach the avoid set
<code>until a,b</code>	$\max P(a U b)$ — reach b while remaining in a (reach-avoid : let a be the complement of the obstacle set)
<code>reach L</code> <code>--horizon k</code>	$P[F \leq k L] = \text{true } U \leq k L$ (step-bounded)
<code>until a,b</code> <code>--horizon k</code>	$a U \leq k b$ (step-bounded until)
<code>next L</code>	$P(X L)$ — one robust Bellman step

3.10.2 Algorithm and literature

The inner robust Bellman solve $\text{opt}_{\{p \text{ in } [lo, hi]\}} \sum p(s') V(s')$ is **O-maximization** (closed-form sort-and-assign), from Givan, Leach, Dean, *Bounded-parameter Markov decision processes*, Artificial Intelligence, 2000, and Lahijanian, Andersson, Belta, IEEE Trans. Automatic Control, 2015 (`src/omaximization.h`, verified against brute-force vertex enumeration).

The outer fixpoint (`src/solve.h`) is solved by one of two methods (see [Solver Engines](#)):

- **Optimistic value iteration** (default) — Hartmanns & Kaminski, CAV 2020: VI from below plus a verified inductive (pre-fixpoint) upper bound.
- **Interval iteration with end-component collapse** — Baier et al., CAV 2017; Haddad & Monmege, TCS 2018.

PCTL operators (`next`, `until`, bounded until; `src/pctl.h`) follow Hansson & Jonsson (Formal Aspects of Computing, 1994); the robust interval-MDP reading follows Puggelli, Li, Sangiovanni-Vincentelli, Seshia, CAV 2013, and Baier & Katoen, *Principles of Model Checking*, Ch. 10.

3.10.3 CLI examples (validated)

```
# Point chain:  $P(F \text{ target}) = 0.25$  exactly (all four tools agree)
tools/imdp_solve benchmarks/crosstool/models/chain_point.imdp reach target --bound_
↪ both
# result prop=reach bound=pess state=0 lower=0.25 upper=0.250001 iters=3

# Safety:  $P(\text{never reach avoid}) = 0.72$ 
tools/imdp_solve benchmarks/crosstool/models/safety_point.imdp safety avoid --bound_
↪ pess
# result prop=safety bound=pess state=0 lower=0.719999 upper=0.72 iters=3

# Interval MDP with a controller choice: robust 0.4 (pick action B), optimistic 0.5
tools/imdp_solve benchmarks/crosstool/models/choice_interval.imdp reach target --
↪ bound both
# result prop=reach bound=pess state=0 lower=0.4 upper=0.400001 iters=2
# result prop=reach bound=opt state=0 lower=0.5 upper=0.500001 iters=2

# Step-bounded reachability  $P[F \leq 3 \text{ goal}] = 1 - 0.5^3 = 0.875$  (exact DP, == Storm)
tools/imdp_solve benchmarks/crosstool/models/bounded_demo.imdp reach goal --horizon 3
# result prop=reach bound=pess state=0 lower=0.875 upper=0.875 iters=3

# PCTL next:  $P(X \text{ target})$  from state 1 of the chain = 0.5
tools/imdp_solve benchmarks/crosstool/models/chain_point.imdp next target --state 1
# result prop=next bound=pess state=1 lower=0.5 upper=0.5 iters=1
```

3.10.4 Peer tools and validation

Robust reachability/safety is the one query all four tools support. On the shared models (identical .imdp exported to each tool's native format) **IMPACT** == **IntervalMDP.jl** == **PRISM** == **Storm**: chain_point 0.25, safety_point 0.72, choice_interval 0.4/0.5, fork_interval 0.4/0.6, and the 207-state multiObj_robotIMDP 0.894663 (agreement $\sim 1e-6$); see TOOL_COMPARISONS.md §1 and the ARCH-COMP 2025 sweep (§2), where all four agree on every robust value. Step-bounded $P[F \leq k]$ was validated against Storm $P_{\max}=?[F \leq k \text{ "goal"}]$ for $k = 1, 2, 3$ (bounded_demo.ref.json). PRISM peers require the $P_{\max\min}/P_{\max\max}$ query form and reject interval edges with a 0 lower bound (see [File Formats and Converters](#)).

3.10.5 Limitations

- Pessimistic interval iteration alone does not converge on *nature-confinable* end components (ISSUE-0003, resolved by the OVI default; see [Solver Engines](#)).
- OVI can be slow on large strongly-recurrent IMDPs, mainly in the optimistic sense (ISSUE-0010, now largely mitigated by topological Gauss–Seidel — BA at $1e-6$ went from >180 s to 7.8 s); --method mec remains an alternative.
- The optimistic sense is convention-dependent across tools for $[0, hi]$ “leak” edges (ISSUE-0018, TOOL_COMPARISONS.md §4); the robust value is unambiguous and is the headline number.
- imdp_solve exposes controller-*maximising* queries ($P_{\max}$); $P_{\min}$ flavours are future work (src/pctl.h).

3.11 Expected Rewards

Robust expected total reward on interval MDPs: **reachability reward** (accumulate until a target is reached) and **discounted reward**.

3.11.1 What is computed

State rewards are given by `reward <s> <v>` lines in the `.imdp` file (missing states default to 0). The controller maximises the expected accumulated reward; nature resolves the transition intervals adversarially (pess) or cooperatively (opt) on the *continuation value* via O-maximization.

- **Reachability reward** (`reward LABEL, discount  $\gamma = 1$`): expected total state reward accumulated until the target set is first reached. The target is absorbing and reward stops there. Bellman recursion (`src/solve.h`):

$$V(s) = \text{reward}[s] + \max_a \text{opt}_{\{p \text{ in } [lo, hi]\}} \sum_{s'} p(s') V(s'), \quad V(\text{target}) = 0$$

$+\infty$ semantics: if the target is not reached almost surely and rewards are positive, the value diverges; IMPACT reports `HUGE_VAL` (prints `inf`), matching PRISM/Storm's Infinity convention for stochastic-shortest-path divergence.

- **Discounted reward** (`reward LABEL --discount g,  $\gamma \in (0,1)$`): $V(s) = \text{reward}[s] + \gamma \cdot \max_a \text{opt}_p \sum p V$. This is a contraction, so it always converges and needs no target (the label argument is still required syntactically). This is the objective `IntervalMDP.jl` natively supports.

Expected time / expected steps to reach is the special case of reachability reward with unit state rewards (`reward s 1` for every non-target state), as noted in `FEATURE_PARITY.md`.

3.11.2 Algorithm and literature

Robust dynamic programming over the interval ambiguity set, with the inner minimisation/maximisation solved in closed form by O-maximization:

- Iyengar, *Robust Dynamic Programming*, Mathematics of Operations Research, 2005.
- Nilim & El Ghaoui, *Robust control of MDPs with uncertain transition matrices*, Operations Research, 2005.
- Givan, Leach, Dean, *Bounded-parameter MDPs*, AIJ 2000 (the interval inner solve).
- Puterman, *Markov Decision Processes*, 1994 (SSP finiteness for the $\gamma = 1$ case).
- Convergence certification: interval iteration / OVI (Baier et al. CAV 2017; Hartmanns & Kaminski CAV 2020), as for reachability.

Implementation: `solve::expReachReward`, `solve::expDiscountedReward` and the `maxReachReward{Pessimistic,Optimistic}` wrappers in `src/solve.{h,cpp}`.

3.11.3 CLI examples (validated)

The `reward_chain` model has $r(s_0)=1$, $r(s_1)=2$; from `s1` nature chooses to move to the target with probability in $[0.5, 1.0]$ or stay with $[0.0, 0.5]$:

```
# Reachability reward: pess (nature leaves s1 asap) V0 = 3; opt (nature stays) V0 = 5
tools/imdp_solve benchmarks/crosstool/models/reward_chain.imdp reward target --bound_
↪ both
# result prop=reward bound=pess state=0 lower=3 upper=3
↪ iters=3
# result prop=reward bound=opt state=0 lower=4.999999046 upper=4.999999046
↪ iters=23

# Discounted reward, gamma = 0.9: pess 2.8, opt 2/(1-0.45)*0.9+1 = 4.2727...
tools/imdp_solve benchmarks/crosstool/models/reward_chain.imdp reward target --
↪ discount 0.9 --bound both
# result prop=reward bound=pess state=0 lower=2.8 upper=2.8
↪ iters=3
# result prop=reward bound=opt state=0 lower=4.272726893 upper=4.272726893
↪ iters=21
```

Both match the analytic hand values in `reward_chain.ref.json` (3 / 5 and 2.8 / 4.272727).

3.11.4 Peer tools and validation

- **PRISM 4.8.1**: interval reachability reward via `Rmaxmin / Rmaxmax` — matches IMPaCT (3.000000002 / 4.999996 on `reward_chain`).
- **IntervalMDP.jl 0.6**: discounted reward only (`InfiniteTimeReward` requires `discount < 1`) — matches IMPaCT to 9+ digits (2.8 / 4.272726893).
- **Storm 1.13.0**: its interval reward **ignores** `--uncertainty-resolution` and returns only the nature-max value (4.999996 = the optimistic value); `Rmin` on intervals is unsupported. Documented in `reward_chain.ref.json` and `TOOL_COMPARISONS.md §1b`.

IMPACT is the only tool providing *both* robust senses for *both* reward objectives on interval MDPs.

3.11.5 Limitations

- Rewards are **state-based** only (no action or transition rewards in the `.imdp` format).
- Reward objectives are not yet available in the multi-objective engine (*Multi-Objective Verification*) — a documented refinement in `ROADMAP_UPGRADES.md §3`.
- Divergent ($+\infty$) reachability-reward cases are reported but not distinguished from very large finite values until convergence is abandoned; use the discounted variant when the target is not reached almost surely.

3.12 Long-Run Average and Steady-State

Robust long-run average (mean-payoff) reward and steady-state probabilities on interval MDPs — the interval/robust versions are unique to IMPACT.

3.12.1 What is computed

- **lra** — the robust long-run average reward $\liminf (1/T) \sum_{t < T} r(s_t)$, controller maximising, nature resolving the intervals adversarially (pess) or cooperatively (opt) at every step.
- **ss LABEL** — the steady-state (long-run fraction of time) probability of the states labelled LABEL. IMPACT computes it as the **long-run average of the indicator reward** `1_LABEL`, the classical identity (Baier & Katoen, *Principles of Model Checking*, Ch. 10; Puterman 1994) — this is PRISM’s S operator, generalised to robust interval MDPs.

3.12.2 Algorithm and literature

Two-phase value iteration (`solve::longRunAverage, src/solve.{h,cpp}`):

1. **MEC gain**: the robust average reward (gain) of each maximal end component is computed by *relative value iteration* with Puterman’s aperiodicity transform $L_{-\tau} h = (1-\tau)h + \tau(r + \text{opt}_a \text{opt}_p \sum p \cdot h)$, restricted to each MEC’s staying actions ($\text{support} \subseteq \text{MEC}$ — exactly why nature cannot escape it). The interval ambiguity set is resolved by O-maximization.
2. **Cash-out**: a max-reachability VI funnels every state to the best robustly-reachable MEC gain.

Literature (verified in `ROADMAP_UPGRADES.md §2`):

- Ashok, Chatterjee, Daca, Křetínský, Meggendorfer, *Value Iteration for Long-Run Average Reward in MDPs*, CAV 2017 (arXiv:1705.02326) — the two-phase VI.
- Chatterjee, Goharshady, Karrabi, Novotný, Žikelić, *Solving Long-run Average Reward Robust MDPs via Stochastic Games*, IJCAI 2024 (arXiv:2312.13912) — the robust/interval setting (that work uses policy iteration/games; IMPACT takes the VI route).
- Puterman, *Markov Decision Processes*, 1994, Ch. 8 (average-reward optimality, relative VI, aperiodicity transform); Iyengar 2005 (robust inner solve).

3.12.3 CLI examples (validated)

```
# Point 2-cycle,  $r(0)=1$ ,  $r(1)=3$ : mean payoff  $(1+3)/2 = 2$  ( $=$  Storm  $R\{r\}max=?[LRA]$ )
tools/imdp_solve benchmarks/crosstool/models/lra_cycle.imdp lra - --bound both
# result prop=lra bound=pess state=0 lower=2 upper=2 iters=0

# Interval MEC,  $r(0)=4$ ,  $r(1)=0$ , stay  $[0.5,1]/switch [0,0.5]$ : nature traps the play in
# state 1 (robust gain 0) or keeps it in state 0 (optimistic gain 4). IMPaCT-only.
tools/imdp_solve benchmarks/crosstool/models/lra_interval.imdp lra - --bound both
# result prop=lra bound=pess state=0 lower=2.265286626e-06 upper=2.265286626e-06
↪ iters=0
# result prop=lra bound=opt state=0 lower=4 upper=4
↪ iters=0

# Steady-state: DTMC with stationary distribution  $(4/7, 3/7)$ ;  $S(g=\{1\}) = 3/7$ 
tools/imdp_solve benchmarks/crosstool/models/ss_dtmc.imdp ss g
# result prop=ss bound=pess state=0 lower=0.4285698197 upper=0.4285698197
↪ iters=0
```

The `lra` property takes no target label — pass `-` as the LABEL placeholder. Values match the analytic references 2, 0/4, and $3/7 = 0.4285714$ within the VI tolerance (`lra_cycle.ref.json`, `lra_interval.ref.json`, `ss_dtmc.ref.json`).

3.12.4 Peer tools and validation

- **Storm** computes point-model LRA ($R\{r\}max=? [LRA] = 2$ on `lra_cycle`, matching IMPaCT) and steady-state ($S=? [g] = 0.4285714286$ on `ss_dtmc`, matching IMPaCT), but reports “*LRA with intervals not implemented*” for interval models.
- **PRISM**’s `S` operator is DTMC/CTMC-only (no MDP steady-state reward, no interval LRA).
- **IntervalMDP.jl** has no LRA / steady-state objective.

Robust interval LRA and robust steady-state were therefore validated against the analytic hand values (`lra_interval 0 / 4`) — **an IMPaCT-only capability** (`FEATURE_PARITY.md`, `TOOL_COMPARISONS.md §1b`).

3.12.5 Limitations

- Convergence of the gain phase is controlled by the same `--eps`; the robust pessimistic value on `lra_interval` converges to $\sim 2e-6$ rather than exactly 0 (residual of the relative VI) — tighten `--eps` for sharper values.
- The steady-state of a *multichain* interval MDP is the reachability-weighted best MEC gain (phase 2), i.e. the controller-optimal long-run frequency; per-initial distribution steady-state vectors are not exposed.
- An exact policy-iteration LRA (as in Chatterjee et al., IJCAI 2024) is a documented refinement (`ROADMAP_UPGRADES.md`).

3.13 ω -Regular and LTL Objectives

Robust Büchi, persistence, generalized-Büchi (“patrol”), a built-in LTL fragment, and arbitrary LTL via Spot — on interval MDPs, where no peer tool offers any ω -regular capability.

3.13.1 What is computed

Property	Objective
buchi L	max P(G F L) — visit L infinitely often
persist L	max P(F G L) — eventually remain in L forever
patrol a,b,...	max P(G F a ∧ G F b ∧ ...) — generalized Büchi
ltl "FORMULA"	built-in fragment: atoms, ! & ->, X, F, G, U, G F, F G, conjunctions of G F
ltlx "FORMULA"	arbitrary LTL: Spot (deterministic class) with automatic Owl LDBA fallback

Pessimistic = the controller wins against *every* nature resolution; optimistic = nature cooperates.

3.13.2 Algorithm and literature

Büchi (src/omega.h) reduces to reachability of accepting end components — the classical reduction (de Alfaro, PhD thesis, 1997; Baier & Katoen, *Principles of Model Checking*, Ch. 10). The two senses differ structurally:

- *Optimistic*: support-graph accepting MECs (exact for that sense and point MDPs).
- *Pessimistic (robust)*: support MECs are **unsound** — inside a support EC nature can route around the accepting state using $\text{lo}=\emptyset$ edges (ISSUE-0009). IMPACT computes the robust almost-sure-Büchi winning region by a nested fixpoint of robust EC closure and robust almost-sure reachability — the qualitative 2.5-player Büchi computation (Chatterjee-Henzinger graph games; the IMDP reading follows Dutreix-Coogan permanent components and Asadi et al. force attractors) — then robust reachability of it.

Persistence F G p is robust reachability of the largest controllably-invariant sub-region of p; **patrol** is degeneralized to Büchi by a round-robin counter product.

Arbitrary LTL (ltlx, src/ltl_spot.h): a **two-stage** pipeline covering *all of LTL* (ISSUE-0016, resolved 2026-07-02):

1. translate the formula with Spot's `ltl2tgba -D` (Duret-Lutz et al., *Spot 2.0*);
2. if the result is a **deterministic (generalized) Büchi** automaton: synchronous product with the IMDP (a deterministic automaton is always good-for-MDPs — Hahn, Perez, Schewe, Somenzi, Trivedi, Wojtczak, *Good-for-MDPs Automata*, TACAS 2020 — so the product value is the LTL value), solved robustly with `omega::maxGenBuchi{Pessimistic,Optimistic}`;
3. otherwise (co-Büchi F G-type and beyond) **ltlx falls back automatically** to a **limit-deterministic Büchi automaton** from Owl's `ltl2ldb` (Křetínský, Meggendorfer, Sickert, *Owl: A Library for ω -Words, Automata, and LTL*, ATVA 2018, DOI 10.1007/978-3-030-01090-4_34; LDBA theory: Sickert, Esparza, Jaax, Křetínský, CAV 2016). The LDBA's ϵ -jumps to its accepting component appear as same-letter nondeterminism, which the product resolves by the **controller** — each matching automaton edge multiplies the product action space (sound because such LDBAs are good-for-MDPs, Hahn et al. TACAS 2020); transition-based Büchi marks become state-based accepting *entry copies*, and the same robust Büchi engine applies unchanged.

The Owl fallback is enabled by pointing `IMPACT_LTL2LDBA` at Owl's native binary (no JRE needed for the Linux amd64 release):

```
export IMPACT_LTL2LDBA="/opt/owl/owl-linux-musl-amd64-21.0/bin/owl ltl2ldb"
```

Without it, nondeterministic-automaton formulas are rejected soundly with an error suggesting the `ltl` fragment (which covers persistence) as the dependency-free alternative.

Set the environment variable `IMPACT_LTL2TGBA` to the `ltl2tgba` command if it is not on `PATH` (it may embed `LD_LIBRARY_PATH=... ../ltl2tgba`).

3.13.3 CLI examples (validated)

```
# Robust vs optimistic Büchi where nature can route around the accepting state
tools/imdp_solve benchmarks/crosstool/models/buchi_routearound.imdp buchi acc --bound_
↪ both
# result prop=buchi bound=pess state=0 lower=0 upper=1e-06 iters=1
# result prop=buchi bound=opt state=0 lower=1 upper=1 iters=1

# Persistence with a [0,hi] leak edge: robust 0.5, optimistic 1.0
tools/imdp_solve benchmarks/crosstool/models/persist_leak.imdp persist safe --bound_
↪ both

# Patrol two regions on a forced cycle: probability 1
tools/imdp_solve benchmarks/crosstool/models/patrol_cycle.imdp patrol r0,r2

# Arbitrary LTL via Spot on an interval MDP (IMPACT-only): robust 0.3 / optimistic 0.7
export IMPACT_LTL2TGBA=ltl2tgba
tools/imdp_solve benchmarks/crosstool/models/ltl_interval.imdp ltlx "G F a" --bound_
↪ both
# result prop=ltlx bound=pess state=0 lower=0.3 upper=0.3000001 iters=2
# result prop=ltlx bound=opt state=0 lower=0.7 upper=0.7000001 iters=2

# Nondeterministic-automaton formula via the Owl LDBA fallback (ISSUE-0016, resolved):
export IMPACT_LTL2LDBA="/opt/owl/owl-linux-musl-amd64-21.0/bin/owl ltl2ldb"
tools/imdp_solve benchmarks/crosstool/models/ltl_demo.imdp ltlx "F G a" --bound opt
# result prop=ltlx bound=opt state=0 lower=0.5 upper=0.5000001 iters=18
# (== the built-in persistence solver == Storm Pmax=?[F G "a"] = 0.5)

# The same on the INTERVAL model - robust full LTL, unique to IMPACT:
tools/imdp_solve benchmarks/crosstool/models/ltl_interval.imdp ltlx "F G a" --bound_
↪ both
# result prop=ltlx bound=pess state=0 lower=0.3 upper=0.3000001 iters=13
# result prop=ltlx bound=opt state=0 lower=0.7 upper=0.7000001 iters=14

# Beyond-fragment formula, LDBA route, validated against Storm (0.5):
tools/imdp_solve benchmarks/crosstool/models/ltl_demo.imdp ltlx "F (a & X a)" --bound_
↪ opt
# result prop=ltlx bound=opt state=0 lower=0.5 upper=0.5000001 iters=13
```

Reference values: buchi_reach 0.5, buchi_routearound 0 / 1, persist_leak 0.5 / 1.0, patrol_cycle 1.0; ltl_demo point MDP $F a / G F a / X X a = 0.5$ and $G (a \rightarrow X a) = 1.0$ (all *.ref.json).

3.13.4 Peer tools and validation

No peer does robust ω -regular on intervals: Storm reports “*LTL with intervals not implemented*”, PRISM “*Computation not implemented yet*”, IntervalMDP.jl supports only DFA/co-safe reachability. Validation is therefore two-legged (TOOL_COMPARISONS.md §1, §1b): on **point** MDPs, ltlx values equal Storm $P_{\max}=?[\varphi]$ for $F a$, $G F a$, $X X a$, $G(a \rightarrow X a)$; on **interval** models the values match the exact analytic references above. The robust-Büchi core (ISSUE-0009) is oracle-validated separately.

3.13.5 Limitations

- ltlx covers **all of LTL**: the deterministic-(generalized-)Büchi class via Spot, and the nondeterministic class (co-Büchi $F G$ etc.) via Owl’s limit-deterministic automata with the controller-resolved jump product (ISSUE-0016, resolved; validated $F G a = \text{persistence solver} = \text{Storm on the point model}$, and $0.3/0.7$ analytically on the interval model). LDBAs with *generalized* (multi-set) acceptance are not yet handled by the fallback (Owl’s ltl2ldb emits single-set Büchi).
- ltlx requires external translators (Spot’s ltl2tgba; Owl’s ltl2ldb for the fallback); the ltl fragment

has no external dependency.

- On unbounded-noise abstractions the infinite-horizon ω -regular value is often 0 because no non-trivial robust ECs survive abstraction (ISSUE-0011, a modelling fact, not a solver bug).

3.14 Multi-Objective Verification

Weighted-sum multi-objective robust reachability and Pareto frontier tracing on interval MDPs.

3.14.1 What is computed

Given k target label sets $T_1, \dots, T_k$ (all made absorbing, so each objective is “the first target hit is T_i ”) and a weight vector w , `multi` maximises

$$w \cdot (P(\text{reach } T_1), \dots, P(\text{reach } T_k))$$

over controller policies, with nature adversarial (`--bound pess`, default) or cooperative (`--bound opt`). The output is the weighted optimum at the initial state **and** the per-objective vector $P(\text{reach } T_i)$ achieved by the weight-optimal policy — one achievable point on the (robust) Pareto frontier. With no `--weights` argument and exactly two objectives, the weight simplex is swept ($\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$) to trace the frontier.

3.14.2 Algorithm and literature

The achievable set of a multi-objective MDP is convex, so its Pareto frontier is traced by the weighted-sum optima over the weight simplex:

- Etessami, Kwiatkowska, Vardi, Yannakakis, *Multi-Objective Model Checking of Markov Decision Processes*, TACAS 2007 (convexity; memoryless-randomised strategies suffice; ε -approximable Pareto curve).
- Forejt, Kwiatkowska, Norman, Parker, Qu, *Quantitative Multi-Objective Verification for Probabilistic Systems*, TACAS 2011 (the PRISM implementation this parallels).

Implementation (`solve::multiReach`, `src/solve.{h,cpp}`): the robust VI is the ordinary O-maximization backup with the value promoted to a vector and scalarised ($vv[i] = \sum_j w_j V_j[to]$); the policy recorded at the weighted optimum is then evaluated per objective.

3.14.3 CLI examples (validated)

The `multi_demo` point MDP offers action A with $(P(F \ t1), P(F \ t2)) = (0.9, 0.1)$ and action B with $(0.2, 0.8)$:

```
# Sweep the weight simplex (2 objectives) - traces the Pareto frontier
tools/imdp_solve benchmarks/crosstool/models/multi_demo.imdp multi t1,t2
# pareto bound=pess state=0 weights=0,1 objectives=0.2,0.8 weighted=0.8
# pareto bound=pess state=0 weights=0.25,0.75 objectives=0.2,0.8 weighted=0.65
# pareto bound=pess state=0 weights=0.5,0.5 objectives=0.9,0.1 weighted=0.5
# pareto bound=pess state=0 weights=0.75,0.25 objectives=0.9,0.1 weighted=0.7
# pareto bound=pess state=0 weights=1,0 objectives=0.9,0.1 weighted=0.9

# One point for an explicit weight vector (required for k > 2 objectives)
tools/imdp_solve benchmarks/crosstool/models/multi_demo.imdp multi t1,t2 --weights 0.
↪ 5,0.5
# pareto bound=pess state=0 weights=0.5,0.5 objectives=0.9,0.1 weighted=0.5
```

The sweep recovers exactly the two Pareto vertices $(0.9, 0.1)$ and $(0.2, 0.8)$ (`multi_demo.ref.json`).

3.14.4 Peer tools and validation

- **Storm**: `multi(Pmax=?[F t1], Pmax=?[F t2])` reports “2 Pareto optimal points” on the same model, matching IMPaCT’s vertices; the corners match Storm’s single-objective `Pmax=?[F t1] = 0.9` and `Pmax=?[F t2] = 0.8`.
- **PRISM**: supports `multi(...)` Pareto queries — non-interval models only.
- **IntervalMDP.jl**: no multi-objective support.

Multi-objective on **interval** MDPs is IMPaCT-only (`TOOL_COMPARISONS.md §1b`, `FEATURE_PARITY.md`); peers were compared on the degenerate (point) model.

3.14.5 Robust-Pareto semantics

For interval models the reported point is what the weight-optimal policy **guarantees robustly per objective**: nature is re-resolved adversarially for each objective evaluation. This is a sound achievable point; the fully convex robust-Pareto set under a *single* adversarial nature across all objectives is a documented refinement (`ROADMAP_UPGRADES.md §3`, `multi_demo.ref.json`).

3.14.6 Limitations

- Objectives are **reachability probabilities only** — reward objectives are a planned refinement (`ROADMAP_UPGRADES.md §3`).
- The automatic sweep works for exactly 2 objectives; for $k > 2$ supply `--weights` vectors yourself.
- Weighted-sum scalarisation enumerates the *vertices* of the Pareto frontier; points on non-vertex faces are convex combinations of the reported vertices (randomised policies), not printed explicitly.

3.15 CTMCs and CSL Time-Bounded Reachability

Continuous-time Markov chains (CTMCs), CSL time-bounded reachability $P(F \leq t \text{ goal})$, and the interval (uncertain-rate) CTMC extension unique to IMPaCT.

3.15.1 Model class

A CTMC is written in the plain `.imdp` format where each state has a **single action** whose entries `to:r:r` are transition **rates** (not probabilities; no self rate). An **interval CTMC** uses rate intervals `to:rlo:rhi` — uncertain rates. The entries are interpreted as rates only by the `cs1` property; see [File Formats and Converters](#).

```
# ctmc_demo.imdp: s0 --rate 1--> s1 (absorbing goal); P(F<=t goal) = 1 - e^{-t}
states 2
init 0
label goal 1
tran 0 0 1:1:1
```

3.15.2 What is computed

`cs1 LABEL --time t` returns the per-state probability of reaching `LABEL` within continuous time `t` (goal states made absorbing). For an interval CTMC, `--bound pess` resolves the uncertain rates adversarially per uniformisation step (sound lower CSL bound) and `--bound opt` cooperatively (sound upper bound) — the three-valued CSL reading of Katoen et al.

3.15.3 Algorithm and literature

Uniformisation + Fox-Glynn (`src/ctmc.{h,cpp}`): Jensen’s uniformisation builds the embedded DTMC $P = I + Q/\Lambda$ with $\Lambda = \max \text{exit rate}$; the time-bounded value is the Poisson-weighted transient of P , with the Poisson weights computed by the numerically stable Fox-Glynn truncation. For interval rates, the uniformised chain is an interval DTMC whose step distributions are resolved by O-maximization.

- C. Baier, B. Haverkort, H. Hermanns, J.-P. Katoen, *Model-Checking Algorithms for Continuous-Time Markov Chains*, IEEE Trans. Software Eng. 29(6):524-541, 2003 (DOI 10.1109/TSE.2003.1205180).
- B. L. Fox, P. W. Glynn, *Computing Poisson probabilities*, CACM 31(4):440-445, 1988 (DOI 10.1145/42404.42409).
- Interval CTMC: Katoen, Klink, Leucker, Wolf, *Three-Valued Abstraction for CTMCs*, CAV 2007 (DOI 10.1007/978-3-540-73368-3_37); Badings, Jansen, Junges, Stoelinga, Volk, *Sampling-Based Verification of CTMCs with Uncertain Rates*, CAV 2022 (arXiv:2205.08300).

Steady-state analysis of a CTMC reuses the LRA machinery: the uniformised DTMC has the same stationary distribution as the CTMC (see [Long-Run Average and Steady-State](#)).

3.15.4 CLI examples (validated)

```
# Point CTMC:  $P(F \leq 1 \text{ goal}) = 1 - e^{-1} = 0.6321206$ 
tools/imdp_solve benchmarks/crosstool/models/ctmc_demo.imdp csl goal --time 1.0
# result prop=csl bound=pess state=0 lower=0.6321205588 upper=0.6321205588
↪iters=0

tools/imdp_solve benchmarks/crosstool/models/ctmc_demo.imdp csl goal --time 2.0
# result prop=csl bound=pess state=0 lower=0.8646647168 upper=0.8646647168
↪iters=0

# Interval CTMC (rate in [0.5, 1.5]) - IMPACT-only:
# robust  $1 - e^{-0.5} = 0.3934693$ , best-case  $1 - e^{-1.5} = 0.7768698$ 
tools/imdp_solve benchmarks/crosstool/models/ctmc_interval.imdp csl goal --time 1.0 --
↪bound both
# result prop=csl bound=pess state=0 lower=0.3934693403 upper=0.3934693403
↪iters=0
# result prop=csl bound=opt state=0 lower=0.7768698399 upper=0.7768698399
↪iters=0
```

3.15.5 Peer tools and validation

- **Point CTMC:** IMPACT == Storm CSL (storm --prism ctmc_demo.sm -pc --prop 'P=?[F<=t "goal"]') == the analytic value $1 - e^{-t}$ **to 10 digits** for $t = 0.5, 1, 2$ (ctmc_demo.ref.json; the PRISM-language twin ctmc_demo.sm ships alongside the .imdp). PRISM supports point-rate CSL equivalently.
- **Interval CTMC:** Storm and PRISM support point rates only, so the robust values were validated against the exact analytic bounds above (ctmc_interval.ref.json) — **an IMPACT-only capability** (FEATURE_PARITY.md).

3.15.6 Limitations

- The csl property covers **time-bounded reachability** $P(F \leq t \text{ goal})$; nested CSL (time-bounded until with a lower time bound, nested P/S operators) is not exposed. Unbounded CSL reachability equals the embedded-DTMC reachability (reach), and steady-state uses ss.
- The model must be a chain at the CTMC level: one action per state (CTMDPs are not supported; nondeterminism plus continuous time is the Markov-automata gap tracked in FEATURE_PARITY.md §B).
- The interval-rate resolution is per uniformisation step (step-wise adversary), the standard sound bound of the three-valued CSL construction; a time-invariant adversary could be tighter.

3.16 Parametric Region Verification

Verify a *parametric* Markov model over a whole parameter region in one solve, by reducing region verification to robust interval-MDP reachability (Parameter Lifting).

3.16.1 What is computed

Given a Markov model whose transition probabilities are expressions in a parameter p and a region $p \in [lo, hi]$, the lifted interval MDP's robust `[pess, opt]` reachability values **bound the parametric value over the entire region**:

```
pess <= min_{p in [lo,hi]} P_p(F target)   and   max_{p in [lo,hi]} P_p(F target)
↪ <= opt
```

The bound is **exact** when each parameter occurs once per transition row, and a **sound over-approximation** otherwise (each parameter *occurrence* is relaxed independently — precisely the Parameter Lifting relaxation).

3.16.2 Algorithm and literature

Parameter Lifting: relax parameter dependencies so each occurrence ranges over the region independently; the result is an interval MDP, and robust reachability on it bounds the parametric solution function over the region.

- Quatmann, Dehnert, Jansen, Junges, Katoen, *Parameter Synthesis for Markov Models: Faster Than Ever*, ATVA 2016 (DOI 10.1007/978-3-319-46520-3_4).

This is a natural fit: parametric region checking $\equiv$ interval/robust solving, which is IMPACT's native engine (FEATURE_PARITY.md §B). The lifting itself is the standalone script `benchmarks/crosstool/peers/param_lift.py`; the solve is the ordinary *Probabilistic Specifications* machinery.

3.16.3 The .pimdp format

Like `.imdp` (see *File Formats and Converters*) with two changes:

- a region directive `param <name> <lo> <hi>`;
- `tran <s> <a> <to>:<expr> ...` where `<expr>` is **affine in the single parameter**: $c, p, 1-p$, or $a*p+b$.

```
# param_demo.pimdp - P(F target) = p; region p in [0.3, 0.7]
param p 0.3 0.7
states 3
init 0
label target 1
tran 0 0 1:p 2:1-p
tran 1 0 1:1
tran 2 0 2:1
```

3.16.4 CLI example (validated)

```
# 1. Lift the parametric model over its region to an interval MDP
python3 benchmarks/crosstool/peers/param_lift.py \
    benchmarks/crosstool/models/param_demo.pimdp > /tmp/param_demo.imdp

# 2. Robust reachability on the lifted IMDP = region bounds on P_p(F target)
tools/imdp_solve /tmp/param_demo.imdp reach target --bound both
# result prop=reach bound=pess state=0 lower=0.3 upper=0.300001 iters=2
# result prop=reach bound=opt state=0 lower=0.7 upper=0.700001 iters=2
```

Here $P_p(F \text{ target}) = p$, so the region $[0.3, 0.7]$ maps exactly to the value range $[0.3, 0.7]$ — the lifting is exact because p occurs once per row (`param_demo.ref.json`).

3.16.5 Peer tools and validation

- **Storm** (`storm-pars`) is the reference implementation of Parameter Lifting region checking. In the recorded validation the lifted IMPaCT bounds $[0.3, 0.7]$ were confirmed against **Storm corner instantiations** ($p=0.3 \rightarrow 0.3, p=0.7 \rightarrow 0.7$) and the analytic value (`param_demo.ref.json`; `storm-pars` itself was not built in the comparison container — the corners confirm the endpoints, while the lifting covers the whole region in one interval-MDP solve).
- **PRISM** supports parametric models via its param engine (solution functions); region verification per se is Storm's feature.
- **IntervalMDP.jl**: no parametric support.

3.16.6 Limitations

- Expressions must be affine in **one** parameter per transition expression (`param_lift.py`); multi-parameter models need one `param` region but each edge expression references a single parameter. Rational-function transition probabilities (parametric *solution functions*, Daws ICTAC 2004 state elimination) are a planned item, not implemented (`FEATURE_PARITY.md §B`).
- When a parameter occurs multiple times in one distribution row, the lifted bound is sound but not tight (the relaxation decouples the occurrences) — the standard Parameter Lifting trade-off; refine by splitting the region.
- Region *refinement loops* (automatic splitting until a threshold is decided) are not provided; run the lift-and-solve per subregion yourself.

3.17 Orthogonal and Mixture IMDPs

Factored interval MDPs whose transition ambiguity decomposes per state dimension — `IntervalMDP.jl`'s scalability representation, now solved natively by IMPaCT.

3.17.1 Model class

An **orthogonally-decoupled IMDP** (**odIMDP**) has a product state space $S = S_1 \times \dots \times S_n$; the ambiguity set of each (state, action) factors as a product of **per-dimension marginal interval distributions**. Storing n marginals of size $|S_d|$ instead of one joint distribution of size $\prod |S_d|$ is the memory/compute scalability feature — and matches the per-dimension transition bounds IMPaCT's own sparse abstraction produces for decoupled dynamics (`src/abstraction.cpp`).

A **mixture IMDP** generalises this: one action is a *mixture* of orthogonal components, where nature picks both a weight vector within interval weights and each component's marginals within theirs.

3.17.2 What is computed

reach (the only property supported on `.odimdp` inputs): robust max-reachability with the controller maximising and nature choosing factored distributions adversarially (`pass`) or cooperatively (`opt`).

Reduction-order convention (ISSUE-0022). The factored ambiguity set is *reduction-order sensitive*: nature's inner marginal choices may condition on the destination coordinates already realised by the outer dimensions (per-prefix rectangular semantics), so different reduction orders define different nature classes and can give different pessimistic values. IMPaCT reduces **dimension 1 innermost** (then 2, ..., n outermost) — exactly `IntervalMDP.jl`'s convention (its `state_action_bellman` reduces dimension 1 first per combination of the remaining coordinates). When citing or comparing factored values, always state the convention.

3.17.3 Algorithm and literature

Robust Bellman backup by **recursive per-dimension O-maximization** (`src/odimdp.{h,cpp}`): reduce the value hyper-rectangle one dimension at a time, optimising the dim- d marginal per prefix of the outer destination coordinates. For mixtures: per-component factored O-max, then O-max over the interval mixture weights.

- Mathiesen, Haesaert, Laurenti, *Scalable control synthesis for stochastic systems via structural IMPDP abstractions*, arXiv:2411.11803 (odIMPDP and mixture ambiguity sets).
- Givan, Leach, Dean, *Bounded-parameter MDPs*, AIJ 2000 (the per-marginal inner solve).

3.17.4 The .odimdp format

```
odimdp
dims n1 n2 ...           # per-dimension sizes; states linearised, dim 0
↳ fastest
init s
label NAME s ...
otran <s> <a> <d> to:lo:hi ... # dim-d marginal of action a (single component)
mtran <s> <a> <k> <d> to:lo:hi ... # dim-d marginal of mixture component k
mweight <s> <a> k:lo:hi ... # interval mixture weights over components
```

See benchmarks/crosstool/models/od_demo.odimdp and mix_demo.odimdp for complete commented examples.

3.17.5 CLI examples (validated)

```
# Orthogonal IMPDP, 2x2 product space, interval marginals per dimension
tools/imdp_solve benchmarks/crosstool/models/od_demo.odimdp reach target --bound both
# result prop=reach bound=pess state=0 lower=0.3714280537 upper=0.3714280537
↳ iters=17
# result prop=reach bound=opt state=0 lower=0.8536579107 upper=0.8536579107
↳ iters=20

# Mixture-of-orthogonal IMPDP with interval mixture weights
tools/imdp_solve benchmarks/crosstool/models/mix_demo.odimdp reach target --bound both
# result prop=reach bound=pess state=0 lower=0.47567519 upper=0.47567519
↳ iters=17
# result prop=reach bound=opt state=0 lower=0.9619447054 upper=0.9619447054
↳ iters=19
```

(Default --eps 1e-6 VI; the converged references are 0.3714285709 / 0.8536585360 and 0.4756756752 / 0.9619450314.)

3.17.6 Peer tools and validation

The oracle is **IntervalMDP.jl** on the *identical* models: **OrthogonalIntervalMarkovDecisionProcess** (benchmarks/crosstool/peers/odimdp_oracle.jl) and **MixtureIntervalMarkovDecisionProcess** (peers/mixture_oracle.jl), with **InfiniteTimeReachability** and **Pessimistic/Optimistic × Maximize**. At tight thresholds **IMPACT matches IntervalMDP.jl to 10 digits in both senses** on both models (od_demo.ref.json; FEATURE_PARITY.md). PRISM and Storm have no factored IMPDP representation.

The first implementation reduced the *highest* dimension innermost and disagreed with IntervalMDP.jl in the pessimistic sense (0.3478 vs 0.3714 on od_demo) while agreeing optimistically — the discriminating experiment that pinned the convention (ISSUE-0022).

3.17.7 Limitations

- Only reach is dispatched for .odimdp inputs (no safety/ ω -regular/reward on the factored representation yet); --bound, --eps, --state apply as usual.
- The .odimdp solver runs plain VI with residual stopping (single value reported as both lower and upper), not the certified OVI bracket of the joint solver.
- Cross-tool factored values are only comparable under a pinned reduction order (ISSUE-0022).

3.18 Exact Rational Solving

`reach --exact`: robust reachability computed **exactly over the rationals**, with a machine-checked fixpoint certificate — including on *interval* models, where the peer tools' exact modes do not work at all.

3.18.1 What is computed

The exact robust value $P(F \text{ target})$ at one queried state, as a fraction (e.g. $2/5$), for a point or interval `.imdp` model (decimal inputs are parsed exactly as fractions). `--bound pess` is adversarial nature, `opt` cooperative. The result line reports the fraction, a decimal approximation, the policy-iteration round count, and whether the **certificate** passed:

```
result prop=reach-exact bound=pess state=0 exact=2/5 approx=0.4 certified=yes
↳iters=2
```

Rationals are long long numerator/denominator with `__int128` intermediates and gcd normalisation; **overflow throws** rather than silently losing exactness.

3.18.2 Algorithm and literature

Policy iteration in the Hoffman-Karp style over the controller policy *and* nature's O-maximization vertex — both spaces are finite because the O-max vertex depends only on the *ordering* of the value vector (`src/exact.{h, cpp}`):

1. **Qualitative preprocessing**: compute the exact robust `prob=0` set `P0` (the largest target-free set nature can confine the play to) and pin it to 0.
2. **Exact policy evaluation**: rational Gaussian elimination on the induced point chain (chain-level unreachability pinned to 0).
3. **Improvement**: better controller action / nature vertex by exact O-maximization (which is division-free — sort plus additive mass assignment — so nature's vertex is computed exactly); repeat until stable.
4. **Certificate**: verify the result is an exact fixpoint of the robust Bellman operator. Fixpoints vanishing on `P0` are unique for reachability, so the certified fixpoint *is* the robust value.

Literature (from `src/exact.h` and `FEATURE_PARITY.md`):

- Iyengar, *Robust Dynamic Programming*, Math. OR 2005, §3.3 (robust policy iteration).
- Puterman, *Markov Decision Processes*, 1994, Ch. 6-7 (classical policy iteration).
- Givan, Leach, Dean, AIJ 2000 (division-free O-maximization).
- Uniqueness of the pinned fixpoint: Baier & Katoen, *Principles of Model Checking*, Thm 10.100ff; Haddad & Monmege, TCS 2018 (interval/MDP setting).

3.18.3 CLI examples (validated)

```
# Point chain: exact 1/4 (== PRISM -exact == Storm --exact)
tools/imdp_solve benchmarks/crosstool/models/chain_point.imdp reach target --exact --
↳bound both
# result prop=reach-exact bound=pess state=0 exact=1/4 approx=0.25
↳certified=yes iters=2

# INTERVAL models - exact robust values, certified (peers cannot do this at all)
tools/imdp_solve benchmarks/crosstool/models/choice_interval.imdp reach target --
↳exact --bound both
# result prop=reach-exact bound=pess state=0 exact=2/5 approx=0.4 certified=yes
↳iters=2
# result prop=reach-exact bound=opt state=0 exact=1/2 approx=0.5 certified=yes
↳iters=2
```

(continues on next page)

(continued from previous page)

```
tools/imdp_solve benchmarks/crosstool/models/fork_interval.imdp reach target --exact -
↪ -bound both
# result prop=reach-exact bound=pass state=0 exact=2/5 approx=0.4 certified=yes_
↪ iters=3
# result prop=reach-exact bound=opt state=0 exact=3/5 approx=0.6 certified=yes_
↪ iters=2
```

3.18.4 Point vs interval, and the peers

PRISM's `--exact` and Storm's `--exact` support **point models only** — verified empirically: both error on interval inputs (FEATURE_PARITY.md §B). On the point chain all three agree on exactly 1/4. The certified interval values 2/5, 1/2, 3/5 above make **exact robust interval solving unique to IMPACT**.

3.18.5 Infeasible-row strictness (ISSUE-0023)

Exact arithmetic makes model-validity checking unavoidable. Rows whose upper bounds sum to slightly below 1 (e.g. truncated decimals like 0.999997 in the IntervalMDP.jl-shipped robot model) admit **no** probability distribution, and exact mode aborts:

```
tools/imdp_solve benchmarks/crosstool/models/multiObj_robotIMDP.imdp reach reach --
↪ exact
# exact error: exact: infeasible interval row (sum hi < 1)
```

Floating-point solvers (IMPACT's own O-max, and evidently PRISM / Storm / IntervalMDP.jl) silently tolerate such rows, effectively giving the missing $\sim 3e-6$ mass to the highest-value successor — so cross-tool comparisons on such models compare *silent-repair conventions*. The strict abort is deliberate (the same strictness that certifies fixpoints certifies input sanity); a possible future `--exact-repair` option would scale the upper bounds to feasibility and report the perturbation (ISSUE-0023).

3.18.6 Limitations

- `--exact` supports the `reach` property only (safety = 1 — reach to avoid can be obtained by complementing by hand; other properties are floating-point only).
- Fixed-width rationals: models whose exact solution needs very large numerators/denominators throw on overflow rather than approximating (switch to the floating-point solver in that case).
- Values are reported for a single queried state (`--state`, default `init`), not the whole vector.

3.19 Statistical Model Checking

Monte-Carlo simulation of point Markov chains: probability estimation with confidence intervals, and sequential hypothesis testing (SPRT) for threshold queries.

3.19.1 What is computed

`smc LABEL` samples paths of a **point** chain (one action per state, `lo == hi`) from the initial state (or `--state S`) and estimates the step-bounded reachability $P(F \leq \text{horizon } \text{LABEL})$; paths stop early in absorbing states. Default horizon is 1000 (override with `--horizon k`), default sample count 100000 (`--samples N`). The estimate line reports:

- `estimate` — fraction of sampled paths that hit the target;
- `wilson95=[lo,hi]` — the Wilson 95% confidence interval;
- `chernoff_eps` — the APMC / Chernoff-Hoeffding half-width ε such that $P(|\text{estimate} - p| \geq \varepsilon) \leq \delta$ with $\delta = 0.05$.

With `--threshold p`, a second line gives the **Wald SPRT** verdict for $H_0: P \geq \theta + \delta$ vs $H_1: P \leq \theta - \delta$ with error bounds $\alpha = \beta = 0.01$ and indifference region 2δ ($\delta = 0.01$): `accept(P>=theta)`, `reject(P<theta)`, or `undecided` (sample budget exhausted, capped at $10 \times$ `--samples`), plus the number of samples the sequential test actually used.

Seeds are fixed (estimation seed 20260702, SPRT seed 20260703), so runs are reproducible.

3.19.2 Algorithm and literature

Plain Monte-Carlo path sampling with rigorous statistics (`src/smc.{h,cpp}`):

- Younes, Simmons, *Probabilistic Verification of Discrete Event Systems Using Acceptance Sampling*, CAV 2002 (SPRT-based statistical model checking).
- Hérault, Lassaigne, Magniette, Peyronnet, *Approximate Probabilistic Model Checking*, VMCAI 2004 (APMC: Chernoff-Hoeffding sample bounds).
- Wald, *Sequential Tests of Statistical Hypotheses*, Annals of Mathematical Statistics 16(2), 1945 (the SPRT itself).

3.19.3 CLI examples (validated)

`bounded_demo` reaches the goal with probability $1 - 0.5^3 = 0.875$ within 3 steps:

```
# Estimation with Wilson 95% CI and Chernoff half-width
tools/imdp_solve benchmarks/crosstool/models/bounded_demo.imdp smc goal --horizon 3
# result prop=smc state=0 estimate=0.87471 wilson95=[0.8726437795,0.876747433]
#      chernoff_eps=0.004294694083 samples=100000

# SPRT: is P >= 0.8? (true: 0.875 >= 0.8) - accepted after 570 samples
tools/imdp_solve benchmarks/crosstool/models/bounded_demo.imdp smc goal --horizon 3 --
->threshold 0.8
# result prop=smc-sprt threshold=0.8 verdict=accept(P>=theta) samples=570

# SPRT: is P >= 0.95? (false) - rejected after 199 samples
tools/imdp_solve benchmarks/crosstool/models/bounded_demo.imdp smc goal --horizon 3 --
->threshold 0.95
# result prop=smc-sprt threshold=0.95 verdict=reject(P<theta) samples=199
```

The Wilson interval $[0.87264, 0.87675]$ contains the analytic 0.875, and the SPRT decides both directions correctly with two to three orders of magnitude fewer samples than the fixed-size estimate.

3.19.4 Peer tools and validation

- **PRISM**: `-sim` (its discrete-event simulator, also DTMC-only for probability estimation). On the same query PRISM `-sim` returned **0.87479** vs IMPACT's **0.87471** — both within their confidence intervals of the analytic 0.875 (FEATURE_PARITY.md §B).
- **Storm**: has a simulation-based engine for point models.
- **IntervalMDP.jl**: no statistical model checking.

3.19.5 Limitations

- **Point chains only**: the model must have one action per state with degenerate intervals; `smc` throws otherwise. MDP scheduler sampling is future work — the same scope as PRISM's simulator (`src/smc.h`). There is no meaningful SMC semantics for *interval* uncertainty without fixing a nature policy; use the robust numeric solvers for intervals.
- The property is step-bounded reachability only (choose `--horizon` large enough to approximate unbounded reachability on absorbing chains; paths that reach an absorbing non-target state stop early, so a large horizon is cheap there).

- Statistical guarantees are confidence statements, not the sound `[lower, upper]` brackets of the numeric engines — use `smc` for quick estimation or very large chains, and the numeric solvers for certified values.

3.20 Solver Engines

IMPACT ships a toolbox of infinite-horizon convergence methods. This page explains what each computes, when each is sound, and which to choose.

3.20.1 The three iteration methods (`IterationMethod`)

The abstraction-side synthesis API (`IMDP::setIterationMethod, src/IMDP.h`) selects among three methods for infinite-horizon robust value iteration; the same solvers back the CLI (the dense SYCL path dispatches infinite-horizon OVI through the shared CPU solver `src/solve.cpp`):

- **IntervalIteration** (dense-path default) — sound interval iteration: iterate the robust Bellman operator from 0 (lower) and from 1 (upper), stop when $|\text{upper} - \text{lower}| < \text{eps}$. A rigorous two-sided bracket (Baier et al., CAV 2017; Haddad & Monmege, TCS 2018), but on weakly-contracting / end-component models the gap can close slowly or never (ISSUE-0003).
- **ValueIteration** — what the peers (PRISM / Storm / IntervalMDP.jl) run by default: plain VI with *residual* stopping. Far fewer sweeps and converges even with end components (the from-0 iterate tends to the least fixed point = the robust value), but the residual stop is **not** a sound two-sided certificate.
- **OptimisticVI** (CLI default) — optimistic value iteration (Hartmanns & Kaminski, CAV 2020): VI from below for the lower bound plus a *verified inductive* upper bound — a candidate U with $F(U) \leq U$ implies $V^* \leq U$ by Knaster-Tarski. A sound two-sided certificate that needs **no end-component handling**. The from-below phase is **topological Gauss–Seidel** (ISSUE-0010, resolved 2026-07-02): the support graph is decomposed into strongly connected components, iterated to convergence in reverse topological order with successor components already fixed (Dai, Mausam, Weld, Goldsmith, *Topological Value Iteration Algorithms*, JAIR 42:181–209, 2011, DOI 10.1613/jair.3390; acyclic states take a single pass), with in-place Gauss–Seidel sweeps (Puterman, 1994, §6.3.3). Both preserve monotone convergence from below to the same least fixpoint, and the inductive certificate is verified globally afterwards, so soundness is independent of the sweep schedule. Measured effect: the ARCH BA robust safety solve at `--eps 1e-6` went from **>180 s (timeout)** to **7.8 s** ($>23\times$), with all reference values bit-identical.

In `imdp_solve`, `--method ovi` (default) selects `solve::Method::OptimisticVI` and `--method mec` selects `solve::Method::MECCollapse` (interval iteration after support-graph maximal-end-component collapse).

3.20.2 The nature trap (ISSUE-0003)

Pessimistic interval iteration with *support-graph* MEC collapse does not converge on **nature-confinable end components**: nature can confine the play in a set by sending a leaving edge to its lower bound 0, creating a non-unique fixpoint that the upper iterate never leaves. Minimal example (`benchmarks/crosstool/models/naturetrap.imdp`): state 0 stays with $[0.5, 1.0]$ or leaves with $[0.0, 0.5]$ — nature zeroes the leaving edge, so the true robust value at 0 is 0, but the support graph sees no MEC and the classic upper bound sticks at 1.

```
# OVI (default): converges to the true robust value 0 with a certified bracket
tools/imdp_solve benchmarks/crosstool/models/naturetrap.imdp reach target --method ovi
# result prop=reach bound=pess state=0 lower=0 upper=1e-06 iters=2

# MEC collapse in the OPTIMISTIC sense: valid and fast (support EC = optimistic EC)
tools/imdp_solve benchmarks/crosstool/models/naturetrap.imdp reach target --bound opt_
↪ --method mec
# result prop=reach bound=opt state=0 lower=0.9999980927 upper=1 iters=20
```

OVI resolving the nature trap is why it is the default; `MECCollapse` remains valid for point MDPs and for the optimistic sense (`src/solve.h`).

3.20.3 When to use which

Situation	Method
Default; interval models, pessimistic sense	OptimisticVI (sound, converges on nature traps)
Large <i>recurrent</i> models (topological Gauss–Seidel handles most; ISSUE-0010 resolved)	OptimisticVI, or MECCollapse for the optimistic sense
Point MDPs, either sense	either; MECCollapse is fast on controller ECs
Matching a peer tool’s convergence behaviour	ValueIteration (dense API only; residual stop, no certificate)
Exact certified values on small models	<i>Exact Rational Solving</i> (rational policy iteration)

3.20.4 Performance notes

- **ISSUE-0010** (resolved 2026-07-02): OVI’s from-below iterate mixed slowly on large, strongly-recurrent IMDPs (the ARCH BA case needed >180 s at `eps 1e-6`). Resolved by the topological Gauss–Seidel from-below phase described above: BA robust safety at `eps 1e-6` now solves in **7.8 s** with identical values. For stubborn cases the older remedies remain available: `--method mec` or a looser `--eps`.
- **ISSUE-0017** (resolved): the SYCL dense VI was slow versus Storm/IntervalMDP.jl on recurrent IMDPs; the infinite-horizon dense path now dispatches through the shared CPU OVI solver (`IMDP::infiniteHorizonOVIDispatch, src/IMDP.h`).
- All methods share the same O-maximization inner solve, so per-sweep cost is identical; they differ only in sweep counts and certification.

3.20.5 Soundness summary

OptimisticVI and IntervalIteration return certified `[lower, upper]` with `gap <= 2*eps`; ValueIteration returns an uncertified estimate. The pessimistic MECCollapse upper bound is *valid* but may not converge on interval nature traps — the CLI defaults protect you from this; only override `--method` in the situations listed above.

3.21 File Formats and Converters

The neutral exchange formats read by `imdp_solve`, and the converters that move models between IMPaCT and PRISM / Storm / IntervalMDP.jl.

3.21.1 .imdp — explicit (interval) MDP

Whitespace-separated text, # comments, blank lines ignored (`src/imdp_io.h`):

```
states <N>                                # required, first
init <s>                                   # optional, default 0
label <NAME> <s> <s> ...                   # repeatable; accumulates state sets
reward <s> <value>                        # optional; per-state reward (expected-reward / LRA
↳ props)
tran <s> <a> <to>:<lo>:<hi> ...             # one line = one action of state <s>: an interval
↳ distribution
```

- **Point MDPs / DTMCs** use `lo == hi` on every edge.
- **Rewards**: states without a reward line have reward 0. Rewards are state-based.
- **Rates (CTMCs)**: a CTMC is a plain `.imdp` whose single action per state carries transition **rates** in the `to:r:r` entries (interval rates `to:rlo:rhi` give an interval CTMC). The entries are interpreted as rates only by the `cs1` property — see *CTMCs and CSL Time-Bounded Reachability*. Example: `benchmarks/crosstool/models/ctmc_demo.imdp`.

- The parser validates state ranges and rejects unknown directives; `io::write` round-trips `io::parse`.

3.21.2 .odimdp — orthogonal / mixture IMPD

Factored per-dimension marginals over a product state space (see *Orthogonal and Mixture IMPDs*):

```
odimdp
dims <n1> <n2> ...           # nStates = n1*n2*...; linearised, dim 0
↳ fastest
init <s>
label <NAME> <s> ...
otran <s> <a> <d> to:lo:hi ... # dim-d marginal interval distribution (1
↳ component)
mtran <s> <a> <k> <d> to:lo:hi ... # dim-d marginal of mixture component k
mweight <s> <a> <k>:lo:hi ... # interval mixture weights over components
```

to in `otran/mtran` is a **dimension-d destination coordinate** `0..dims[d]-1`, not a global state. Examples: `od_demo.odimdp`, `mix_demo.odimdp`.

3.21.3 .pimdp — parametric IMPD

Like `.imdp` plus a region directive; solved by lifting to an interval IMPD (see *Parametric Region Verification*):

```
param <name> <lo> <hi>           # the parameter region
tran <s> <a> <to>:<expr> ...       # <expr> affine in ONE parameter: c, p, 1-p, a*p+b
```

3.21.4 PRISM-language subset

`imdp_solve` also reads `.prism` / `.pm` / `.nm` files through a PRISM-language front-end (`src/prism.h`) targeting the same Problem struct, so both front ends feed one solver path. Validated: `choice.prism` gives the identical robust values (0.4 / 0.5) as its `.imdp` twin (`choice.prism.ref.json`).

3.21.5 Exporting an abstraction: `IMDP::exportIMDP`

The abstraction front end writes its in-memory interval matrices to the neutral format with `exportIMDP("model.imdp")` (`src/GPU_synthesis.cpp`) — this is how the cross-tool comparisons run peers on IMPACT's abstractions (`TOOL_COMPARISONS.md`). Note: the export ignores the disturbance dimension (`dim_w > 0` prints a warning).

3.21.6 Converters (benchmarks/crosstool/peers/)

Script	Direction	Notes
imdp_to_drn.py MODEL.imdp OUT.drn	.imdp → Storm DRN	@type: MDP, @value_type: double-interval; preserves state order and labels, carries reward lines
imdp_to_prism_inter- py MODEL.imdp OUTBASE [--floor EPS]	.imdp → PRISM ex- plicit(.tra/. sta/.lab)	PRISM ≥ 4.8 auto-detects the IMDP; PRISM rejects $[0,hi]$ edges (“lower bound of 0”), so pass <code>--floor 1e-9</code> for abstracted models — the PRISM value is then an ε -perturbation (marked ** in comparisons)
imdp_to_prism.py	.imdp → PRISM explicit, point- only	for degenerate models
tra_to_imdp.py BASE > out.imdp	PRISM- explicit interval . tra+.lab → .imdp	e.g. IntervalMDP.jl’s shipped <code>multiObj_robotIMDP</code>
param_lift.py MODEL.pimdp > MODEL.imdp	.pimdp → . imdp	Parameter Lifting (<i>Parametric Region Verification</i>)
intervalmdp_runner. jl	runs Inter- valMDP.jl on .imdp- derived models	peer oracle

```
# Round-trip example: solve IMPACT's model in Storm
python3 benchmarks/crosstool/peers/imdp_to_drn.py \
  benchmarks/crosstool/models/choice_interval.imdp /tmp/choice.drn
storm --explicit-drn /tmp/choice.drn --uncertainty-resolution robust \
  --prop 'Pmax=? [F "target"]'
```

3.21.7 Interoperability caveats

- **PRISM zero lower bounds:** fixed positive support graph required; floor $[0,hi]$ edges first (above).
- **Infeasible rows:** some third-party models contain rows with $\text{sum}(hi) < 1$ (truncated decimals); floating-point solvers silently repair these, IMPACT’s exact mode rejects them (ISSUE-0023, see [Exact Rational Solving](#)).
- **Thresholds:** IntervalMDP.jl’s default $1e-6$ residual can stop $\sim 1e-6$ short of the fixpoint on slowly-mixing chains — tighten it when comparing digits (TOOL_COMPARISONS.md §4).

3.22 CLI Reference: imdp_solve

The dependency-free command-line solver over the neutral model formats. Source: `tools/imdp_solve.cpp`.

3.22.1 Build

```
c++ -std=c++17 -O2 tools/imdp_solve.cpp \
  src/imdp_io.cpp src/prism.cpp src/solve.cpp src/omaximization.cpp \
  src/graph_utils.cpp src/omega.cpp src/pctl.cpp src/ltlspec.cpp \
  src/ltl_spot.cpp src/ctmc.cpp src/odimdp.cpp src/smc.cpp src/exact.cpp \
  -o tools/imdp_solve
```

ltx shells out to Spot’s `ltl2tgba`; set `IMPACT_LTL2TGBA` to its command if not on PATH.

3.22.2 Usage

```
imdp_solve MODEL(.imdp|.prism|.pm|.nm|.odimdp) PROP LABEL [flags]
```

Output: one tab-separated result line per requested bound, result prop=... bound=pess|opt state=S lower=L upper=U iters=N (multi prints pareto lines; smc prints smc/smc-sprt lines; --exact prints exact=/approx=/certified=). Exit codes: 0 success, 1 parse/solve error, 2 usage error. All examples below use models in benchmarks/crosstool/models/.

3.22.3 Properties

PROP	Meaning	Example
reach L	max P(F L); --horizon k gives P[F≤k L]; --exact gives rational value	imdp_solve chain_point.imdp reach target --bound both → pess/opt 0.25
safety L	max P(G !L), L = avoid set	imdp_solve safety_point.imdp safety avoid → 0.72
until a,b	max P(a U b) (reach-avoid); --horizon k gives a U≤k b	imdp_solve patrol_cycle.imdp until r0,r2 → 0
next L	P(X L)	imdp_solve chain_point.imdp next target --state 1 → 0.5
reward L	expected reward to L ($\gamma=1$) or discounted (--discount g<1; L still required)	imdp_solve reward_chain.imdp reward target --bound both → 3 / 5
lra -	long-run average reward (no label; pass -)	imdp_solve lra_cycle.imdp lra - → 2
ss L	steady-state probability of L (= LRA of indicator)	imdp_solve ss_dtmc.imdp ss g → 0.42857
csl L	CTMC P(F≤--time L); model entries are rates	imdp_solve ctmc_demo.imdp csl goal --time 1.0 → 0.6321205588
buchi L	max P(G F L)	imdp_solve buchi_routearound.imdp buchi acc --bound both → 0 / 1
persist L	max P(F G L)	imdp_solve persist_leak.imdp persist safe --bound both → 0.5 / 1
patrol a,b,..	generalized Büchi: every set infinitely often	imdp_solve patrol_cycle.imdp patrol r0,r2 → 1
ltl "φ"	built-in fragment (X F G U, GF, FG, patrol conjunctions)	imdp_solve ltl_demo.imdp ltl "G F a" → 0.5
ltlx "φ"	arbitrary LTL via Spot + Owl LDBA fallback (all of LTL)	imdp_solve ltl_interval.imdp ltlx "G F a" --bound both → 0.3 / 0.7
multi t1,t2,..	multi-objective reachability; sweep or --weights	imdp_solve multi_demo.imdp multi t1, t2 → vertices (0.9,0.1), (0.2,0.8)
smc L	Monte-Carlo estimate of P(F≤--horizon L), point chains	imdp_solve bounded_demo.imdp smc goal --horizon 3 → estimate 0.87471

LABEL is a label name declared in the model; for patrol/until/multi it is comma-separated; for ltl/ltlx it is the LTL formula (quote it); for lra it is unused (pass -).

.odimdp inputs support reach only: imdp_solve od_demo.odimdp reach target --bound both → 0.37143 / 0.85366.

3.22.4 Flags

Flag	Default	Meaning	Example
<code>--bound pess opt bot</code>	pess	nature adversarial / cooperative / print both	<code>... reach target --bound both</code>
<code>--eps E</code>	1e-6	convergence tolerance (bracket gap $\leq 2E$)	<code>... reach target --eps 1e-9</code>
<code>--method ovi mec</code>	ovi	OVI vs MEC-collapse interval iteration (<i>Solver Engines</i>)	<code>... reach target --bound opt --method mec</code>
<code>--state S</code>	model init	report the value of state S	<code>... next target --state 1</code>
<code>--horizon k</code>	0 (unbounded); smc: 1000	step bound for reach/until (exact DP) and smc	<code>... reach goal --horizon 3</code> $\rightarrow 0.875$
<code>--discount g</code>	1.0	reward: $\gamma < 1$ switches to discounted reward	<code>... reward target --discount 0.9</code> $\rightarrow 2.8 / 4.2727$
<code>--time t</code>	1.0	cs1: the CSL time bound	<code>... cs1 goal --time 2.0</code> $\rightarrow 0.8646647168$
<code>--weights w1,...,wk</code>	sweep (k=2)	multi: one weight vector instead of the sweep	<code>... multi t1,t2 --weights 0.5,0.5</code>
<code>--samples N</code>	100000	smc: number of simulated paths	<code>... smc goal --samples 1000000</code>
<code>--threshold p</code>	off	smc: add a Wald SPRT verdict for $P \geq p$	<code>... smc goal --threshold 0.8</code> $\rightarrow \text{accept}(P \geq \theta)$
<code>--exact</code>	off	reach only: rational robust policy iteration + certificate	<code>... reach target --exact</code> $\rightarrow \text{exact}=1/4$ <code>certified=yes</code>
<code>--json</code>	off	emit model structure + per-state [lower, upper] values as JSON (web app)	<code>... reach target --json</code>
<code>--trace</code>	off	JSON plus the per-iteration VI trace (reach/safety; implies <code>--json</code>)	<code>... reach target --trace</code>

3.22.5 Worked example

```
cd benchmarks/crosstool/models
../.../tools/imdp_solve choice_interval.imdp reach target --bound both --eps 1e-6
# result prop=reach bound=pess state=0 lower=0.4 upper=0.400001 iters=2
# result prop=reach bound=opt state=0 lower=0.5 upper=0.500001 iters=2
```

Every example value above is a live output validated against the model's `.ref.json` and, where a peer supports the query, against PRISM / Storm / IntervalMDP.jl (`TOOL_COMPARISONS.md`).

3.22.6 Notes

- The usage: banner lists the most common properties; `ss`, `cs1`, `multi`, `smc` and the `--exact/.odimdp` modes are dispatched as documented here (source of truth: the dispatch in `tools/imdp_solve.cpp`).
- `reward` requires a valid label argument even in discounted mode.
- `--method mec` does not converge for the pessimistic sense on interval nature-traps (ISSUE-0003); keep the OVI default there.
- `--json/--trace` are intended for small models (the web app draws every edge).

3.23 Tool comparison: IMPaCT vs PRISM vs Storm vs IntervalMDP.jl

This section compares IMPaCT against the three peer probabilistic model checkers with interval-MDP (IMDP) support, on identical models with aligned robust semantics. It summarises the repository’s cross-tool study (TOOL_COMPARISONS.md, FEATURE_PARITY.md, benchmarks/crosstool/); pairwise pages give more detail per tool:

- *IMPaCT vs PRISM*
- *IMPaCT vs Storm*
- *IMPaCT vs IntervalMDP.jl*

3.23.1 Versions compared

Tool	Version	Obtained
IMPaCT	v2.0 (IMPaCT-v2.0)	built from source (AdaptiveCpp/SYCL, Armadillo, GLPK, HDF5, GSL, NLOpt)
PRISM	4.8.1	official Linux binary (interval-MDP support since v4.8, 2024)
Storm	1.13.0	built from source in the <code>impact_work</code> container (<code>peers/build_storm.sh</code>)
IntervalMDP.jl	0.6.0	Julia 1.10, in the <code>impact_work</code> container

3.23.2 Methodology

Only IMPaCT *abstracts* a continuous stochastic system into an IMDP; the peers consume a finite IMDP and only *solve* it. For a fair comparison, IMPaCT exports its abstracted IMDP to a neutral `.imdp` file, and every tool solves the **identical model**:

- IMPaCT reads `.imdp` directly (`tools/imdp_solve`);
- Storm reads a DRN produced by `peers/imdp_to_drn.py`;
- PRISM reads explicit interval `.tra/.sta/.lab` from `peers/imdp_to_prism_interval.py`;
- IntervalMDP.jl reads the model via `peers/intervalmdp_runner.jl`.

One driver runs the whole sweep (`peers/sweep_compare.py`); raw output is in `benchmarks/crosstool/sweep_all.tsv` and `sweep_arch.tsv`. Robust semantics are aligned across tools before any number is compared:

Sense	IMPaCT	PRISM	Storm	IntervalMDP.jl
pess (robust: controller maximises, nature adversarial)	<code>--bound pess</code>	<code>Pmaxmin=? [...]</code>	<code>--uncertainty-resolutic robust</code>	Pessimistic, Maximize
opt (nature cooperative)	<code>--bound opt</code>	<code>Pmaxmax=? [...]</code>	<code>--uncertainty-resolutic cooperative</code>	Optimistic, Maximize

3.23.3 Feature matrix

Audited from source and documentation, 2026-06-30, and updated with the v2.0 additions tracked in FEATURE_PARITY.md. ✓ supported, × not supported, N/A out of scope.

Capability	IMPACT	IntervalMDP.jl 0.6	PRISM 4.8.1	Storm 1.13.0
Abstract continuous stochastic dynamics → IMDP (<i>IMPACT-only</i>)	✓ nonlinear (NLopt + sparse)	×	×	×
Noise models (for abstraction)	Gaussian, multivariate, uniform, triangular, Laplace, custom PDF	N/A	N/A	N/A
Interval / robust (I)MDP solving	✓	✓	✓ (since v4.8)	✓
Robust reachability / safety	✓	✓	✓	✓
Robust bounded / finite-horizon	✓	✓	✓ F≤k	✓ step-bounded
Robust on ω - regular IMDPs (Büchi/persistence/LTL) (<i>IMPACT-only</i>)	✓	×	×	×
Expected reward / cost	✓ reachability + discounted (robust)	✓ discounted, exit-time	✓ (+interval reach-reward)	✓
Long-run average / mean-payoff	✓ robust interval (<i>interval LRA IMPACT-only</i>)	×	✓ (non-interval)	✓ (non-interval)
Steady-state (S operator)	✓ robust interval	×	✓ (DTMC/CTMC only)	✓
Multi-objective / Pareto	✓ robust reachability	×	✓ (non-interval)	✓ (non-interval)
CTMC + CSL time-bounded reachability	✓	×	✓	✓
Interval CTMC (uncertain rates), robust CSL (<i>IMPACT-only</i>)	✓	×	×	×
Orthogonal / mixture IMDPs (factored)	✓	✓	×	×
Parametric region verification	✓ (parameter lifting → interval MDP)	×	×	✓ (storm-pars)
Statistical MC / simulation	✓ (point chains)	×	✓ -sim	✓
Controller / strategy synthesis	✓	✓	✓	✓
GPU acceleration	✓ SYCL (any vendor)	✓ CUDA	×	×
Exact / rational arithmetic	✓ --exact, incl. certified interval-exact values	✓ Rational	✓ -exact (errors on intervals)	✓ (errors on intervals)
Symbolic / BDD engine	×	×	✓ MTBDD	✓ dd (CUDD/Sylvan)
Full (non-robust) LTL / PCTL* / CSL	fragment + arbitrary det.-Büchi LTL via Spot	×	✓	✓ (Spot)
Other model classes	POMDP, PTA, TA	orthogonal-IMDP, mixture-IMDP, IMC	DTMC/CTMC/M	DTMC/CTMC/MDP/MC/POMDP, parametric, DFT, GSPN
Solver methods	II, VI, OVI, MEC, O-max	robust VI (O-max)	VI/GS/PI/II/topo	VI/II/OVI/SVI/PI/LP/topological/symbolic
Input formats	C++ dynamics, .imdp, PRISM-subset, HDF5	netCDF, PRISM-explicit, bmdp-tool	PRISM lang, explicit, PEPA	PRISM, JANI, DRN, explicit, GSPN, DFT

3.23.4 Agreement: shared IMPDP models, all four tools

Robust value at the initial state, `press / opt`; ✓ = matches the analytic reference.

model	spec	IMPACT	IntervalMDP.jl	PRISM	Storm	ref
chain_point	reach	0.25 / 0.25	0.25 / 0.25	0.25 / 0.25	0.25 / 0.25	0.25 ✓
safety_point	safety	0.72 / 0.72	0.72 / 0.72	0.72 / 0.72	0.72 / 0.72	0.72 ✓
choice_interval	reach	0.40 / 0.50	0.40 / 0.50	0.40 / 0.50	0.40 / 0.50	0.40 / 0.50 ✓
fork_interval	reach	0.40 / 0.60	0.40 / 0.60	0.40 / 0.60	0.40 / 0.60	0.40 / 0.60 ✓
multiObj_robotIMDP (207-state)	reach	0.894663 / 1.0	0.894663† / 1.0	0.894662 / 1.0	0.894663 / 1.0	agree ~1e-6 ✓
buchi_reach	GF acc	0.5 / 0.5	N/A	N/A	N/A	0.5 ✓
buchi_routearound	GF acc	0 / 1	N/A	N/A	N/A	0 / 1 ✓
persist_leak	FG safe	0.5 / 1.0	N/A	N/A	N/A	0.5 ✓
patrol_cycle	GF r0 ∧ GF r2	1.0 / 1.0	N/A	N/A	N/A	1.0 ✓

All four tools agree on every reachability/safety model. The four ω -regular models are IMPACT-only: Storm reports “*LTl with intervals not implemented*”, PRISM “*Computation not implemented yet*”, and IntervalMDP.jl offers DFA/co-safe reachability only. † IntervalMDP.jl’s default 1e-6 threshold stops at 0.894662 (1.2e-6 low); at a tight threshold it reaches 0.894663, equal to the others.

On the ARCH-COMP 2025 abstracted IMPDPs (AS, BA, IC, PD, VP — IMPACT abstracts, the peers solve the exported model), **all four tools agree on every robust value**; optimistic reach also agrees to high precision (e.g. IC_reach opt $\approx 1.20\text{e-}26$ in all four). See the pairwise pages for the timing breakdowns.

3.23.5 Cross-tool findings

Every observed difference is a documented semantic distinction or a performance characteristic — no tool returned a different correct value on the same model.

- **PRISM rejects zero lower bounds.** PRISM 4.8.1 needs a strictly positive support graph and aborts with “*Transition probability has lower bound of 0*” on abstracted IMPDPs; the sweep floors $[\emptyset, \text{hi}]$ edges to $[1\text{e-}9, \text{hi}]$, so PRISM’s ARCH values are an ε -perturbation. PRISM also reports only the initial-state value.
- **Convergence thresholds must match.** IntervalMDP.jl’s default reachability threshold (1e-6) stops $\sim 1.2\text{e-}6$ short of the fixpoint on slowly-mixing chains. IMPACT’s OVI returns the converged value *and* a certified upper bound at the same tolerance.
- **Storm’s interval rewards ignore the resolution mode.** On interval reward models Storm returns only the nature-max value regardless of `--uncertainty-resolution` (Rmin unsupported), so robust interval rewards were validated against PRISM’s Rmaxmin/Rmaxmax and IntervalMDP.jl instead.
- **Factored (odIMDP) semantics are reduction-order sensitive** (ISSUE-0022). The robust value of an orthogonally-decoupled IMPDP depends on which dimension nature resolves innermost; IMPACT pins IntervalMDP.jl’s dimension-1-innermost convention and then matches it to 10 digits in both senses.
- **Exact mode surfaces infeasible interval rows** (ISSUE-0023). `imdp_solve reach --exact` rejects rows of the IntervalMDP.jl-shipped robot model whose upper bounds sum to slightly below 1 (e.g. 0.999997); all floating-point solvers, IMPACT’s included, silently tolerate this and assign the missing $\sim 3\text{e-}6$ mass to the best successor.
- **Optimistic safety is convention-dependent** across tools (how the cooperative sense resolves $[\emptyset, \text{hi}]$ leak edges); the robust value is unambiguous and agrees, so robust is the headline number everywhere.

3.23.6 Reproducing the comparison

```
# in the impact_work container (repo mounted at /work):
python3 benchmarks/crosstool/peers/sweep_compare.py all      # 4-tool sweep -> TSV
tools/impdp_solve MODEL.impdp reach|safety|buchi|persist|patrol LABEL --bound both
```

Converters and runners live in `benchmarks/crosstool/peers/` (`tra_to_impdp.py`, `impdp_to_drn.py`, `impdp_to_prism_interval.py`, `intervalimpdp_runner.jl`, `build_storm.sh`). Per-model reference values with measured per-tool numbers are the `benchmarks/crosstool/models/*.ref.json` files; the IMPACT-side benchmark log is `benchmarks/RESULTS.md`; design decisions and discrepancies are filed under `issues/`.

3.24 IMPACT vs PRISM

PRISM (compared at version 4.8.1, official Linux binary) is the classic probabilistic model checker: PRISM-language models, full PCTL*/LTL/CSL on DTMCs/CTMCs/MDPs, and — since v4.8 (2024) — interval MDPs. It auto-detects Type: IMPDP on explicit models and solves $P_{\max\min}/P_{\max\max}=?[F \dots]$, which is exactly IMPACT’s robust/optimistic reachability.

3.24.1 Feature overlap

PRISM has (IMPACT does not)	Both	IMPACT has (PRISM does not)
Full PCTL*/LTL/CSL (non-interval)	Interval-MDP robust reachability / safety ($P_{\max\min}/P_{\max\max} = \text{--bound pess/opt}$)	Abstraction of continuous nonlinear stochastic dynamics $\rightarrow$ IMPDP (6 noise models)
Symbolic MTBDD engine	Bounded reachability $F \leq k$	Robust ω -regular and arbitrary det.-Büchi LTL on IMPDPs
IDTMC model class	Reachability rewards on intervals (PRISM $R_{\max\min}/R_{\max\max}$)	Robust interval long-run average, steady-state, multi-objective
Multi-objective / Pareto (non-interval)	Exact / rational arithmetic (PRISM --exact , IMPACT --exact)	Exact arithmetic on interval models (PRISM is point-only)
Steady-state S operator (DTMC/CTMC only)	Statistical model checking (PRISM --sim , IMPACT smc)	Interval CTMC (uncertain rates) with robust CSL
PEPA input, GUI, simulator tooling	CTMC + CSL time-bounded reachability; POMDP; PTA; strategy synthesis	GPU (SYCL) solving; orthogonal/mixture IMPDPs; parametric region verification

3.24.2 Same results on shared models

All values below are from live runs recorded in `TOOL_COMPARISONS.md` and the `benchmarks/crosstool/models/*.ref.json` measured blocks.

Interval reference models. PRISM reproduces every interval reference value exactly: `chain_point` 0.25, `safety_point` 0.72, `choice_interval` 0.40/0.50, `fork_interval` 0.40/0.60. On the 207-state `multiobj_robotIMPDP`, PRISM returns 0.8946624 (pess) — agreeing with IMPACT/Storm’s 0.8946630 to $\sim 1e-6$ at default tolerances — and 1.0 (opt).

PRISM-language front end. IMPACT parses a PRISM-language subset directly: `choice.prism` and the equivalent `choice_interval.impdp` solve to identical values (pess 0.4 / opt 0.5) through one solver path.

Robust rewards. On `reward_chain` (interval MDP with rewards), IMPACT’s reachability reward is pess 3.0 / opt 4.999999 against PRISM’s $R_{\max\min}$ 3.000000002 / $R_{\max\max}$ 4.999996 — the analytic values are 3 and 5. PRISM is the oracle here because Storm’s interval rewards ignore the resolution mode.

Exact arithmetic, 1/4. On the point chain, IMPACT’s `impdp_solve reach --exact` returns exactly 1/4, matching PRISM --exact (and Storm’s exact mode). On *interval* models IMPACT certifies exact robust values (2/5, 1/2, 3/5 on the reference models) where PRISM errors — its exact engine is point-only (verified).

Statistical model checking. IMPaCT's `imdp_solve smc` estimate 0.87471 (Wilson CI containing the analytic 0.875) matches PRISM -sim 0.87479 on the same point chain.

ARCH abstracted IMDPs. After flooring (below), PRISM agrees with the other three tools on every ARCH robust value (AS/IC/VP reach 0, PD 1.0, BA safety 0, IC_reach opt $\approx 1.20\text{e-}26$).

3.24.3 Differences and caveats

- **Zero lower bounds are rejected.** PRISM needs a fixed, strictly positive support graph and aborts with “*Transition probability has lower bound of 0*”. Abstracted IMDPs are full of $[\emptyset, \text{hi}]$ edges, so the sweep floors them to $[1\text{e-}9, \text{hi}]$ for PRISM only; its ARCH values are therefore an ε -perturbation of the true robust value.
- **No robust ω -regular on intervals.** LTL on interval models fails with “*Computation not implemented yet*”; the Büchi/persistence/patrol reference models are PRISM-N/A.
- **Initial-state reporting.** prism reports only the initial-state value; per-state comparisons (e.g. `chain_point state 1 = 0.5`) need the other tools.
- **No abstraction front end.** PRISM consumes a finite model; the entire continuous-dynamics-to-IMDP phase is IMPaCT-only.

3.24.4 Performance

From the four-tool sweep on the ARCH abstracted IMDPs (solve phase only; times in seconds, IMPaCT / IntervalMDP.jl / PRISM / Storm):

benchmark	spec	time (s)
AS (anaesthesia 3D)	reach	0.4 / 0.5 / 0.5 / 0.05
IC_reach (integrator 2D)	reach	0.1 / 0.4 / 0.2 / 0.05
IC_safe (integrator 2D)	safety	0.15 / 0.5 / 0.06 / 0.02
PD_p1 (pkg delivery 2D)	reach	3.4 / 1.4 / 1.7 / 2.8
PD_p3 (pkg delivery 2D)	reach	3.6 / 1.6 / 2.3 / 2.2
VP (Van der Pol 2D)	reach	1.8 / 0.55 / 1.4 / 0.04

PRISM's solve times are in the same band as IMPaCT's on these models — faster on some (IC_safe 0.06 s vs 0.15 s, PD_p1 1.7 s vs 3.4 s), slower on others (small reference models: ~ 0.01 s vs IMPaCT's ~ 0.002 – 0.006 s). Note that the peers are handed a pre-built IMDP: the abstraction cost (e.g. AS 27 s dense / 0.11 s sparse; PD ~ 37 s dense) is an IMPaCT-side cost with no PRISM equivalent, because PRISM cannot perform this phase at all.

3.24.5 Reproduce

```
# convert IMPaCT's exported IMDP to PRISM explicit interval format
python3 benchmarks/crosstool/peers/imdp_to_prism_interval.py MODEL.imdp

# run the whole four-tool sweep (handles the [1e-9,hi] floor for PRISM)
python3 benchmarks/crosstool/peers/sweep_compare.py all
```

Raw results: `benchmarks/crosstool/sweep_all.tsv`, `sweep_arch.tsv`; per-model measured values: `benchmarks/crosstool/models/*.ref.json`.

3.25 IMPaCT vs Storm

Storm (compared at version 1.13.0, built from source in the `impact_work` container via `peers/build_storm.sh`) is a high-performance probabilistic model checker with the broadest model-class coverage of the peers: DTMC/CTMC/MDP/MA/POMDP, parametric models (`storm-pars`), DFTs and GSPNs. It solves interval MDPs with `--uncertainty-resolution robust|cooperative`, which maps directly to IMPaCT's `--bound pess|opt`.

3.25.1 Feature overlap

Storm has (IMPACT does not)	Both	IMPACT has (Storm does not)
Markov automata (MA)	Interval-MDP robust reachability / safety and step-bounded $F \leq k$	Abstraction of continuous nonlinear stochastic dynamics $\rightarrow$ IMPDP (6 noise models)
Parametric solution functions; DFT / GSPN front ends	CTMC + CSL time-bounded reachability	Robust ω -regular and arbitrary det.-Büchi LTL on IMPDPs (Storm: “ <i>LTL with intervals not implemented</i> ”)
JANI input; symbolic dd engine (CUDD/Sylvan)	Long-run average, steady-state, multi-objective (Storm: non-interval only)	Robust interval LRA, steady-state, and multi-objective
Full LTL via Spot (non-interval)	Expected rewards (see the interval-reward caveat below)	Robust interval rewards in both senses (reachability + discounted)
Sound-VI family (SVI etc.), LP solvers	Exact arithmetic; statistical model checking; strategy synthesis	Exact arithmetic on interval models (Storm is point-only); interval CTMC robust CSL
—	Parametric region verification (Storm storm-pars; IMPACT via parameter lifting)	GPU (SYCL) solving; orthogonal/mixture IMPDPs

3.25.2 Same results on Storm-validated queries

Storm is the oracle for most of IMPACT’s v2.0 capability additions; each value below is from a live run recorded in the model’s .ref.json measured block.

model / query	IMPACT	Storm	analytic
bounded_demo: $P[F \leq k \text{ goal}], k=1,2,3$	0.5 / 0.75 / 0.875	0.5 / 0.75 / 0.875 ($P_{\max}=?[F \leq k \text{ "goal"}]$)	$1 - 0.5^k$ ✓
ctmc_demo: CSL $P(F \leq t \text{ goal}), t=1, 2, 0.5$	0.6321205588 / 0.8646647168 / 0.3934693403	identical to 10 digits	$1 - e^{-t}$ ✓
lra_cycle: long-run average reward	2.0 / 2.0	2.0 ($R\{“r”\}_{\max}=?[LRA]$)	2 ✓
ltl_demo: $F a, G F a, X X a$	0.5 / 0.5 / 0.5 (ltlx)	0.5 / 0.5 / 0.5 ($P_{\max}=?[\varphi]$)	✓
multi_demo: Pareto for $(F t_1, F t_2)$	vertices (0.9, 0.1), (0.2, 0.8)	2 Pareto points; corners $P_{\max}[F t_1]=0.9, P_{\max}[F t_2]=0.8$	✓
ss_dtmc: steady-state of g	0.4285698	0.4285714286 ($S=?[g]$)	3/7 ✓
param_demo: region $p \in [0.3, 0.7]$	[0.3, 0.7] via lifting	corner instantiations 0.3 and 0.7	✓

On the shared interval reference models and the ARCH abstracted IMPDPs, Storm agrees with IMPACT on every robust value (multiObj_robotIMPDP pess 0.8946629814 vs IMPACT 0.8946629826; optimistic reach agrees to high precision, e.g. IC_reach opt $\approx 1.20e-26$).

3.25.3 Differences and caveats

- **Interval rewards ignore the resolution mode.** On reward_chain Storm returns only the nature-max reward 4.999996 (= the optimistic value) regardless of --uncertainty-resolution, and Rmin is unsupported — so robust interval rewards were cross-validated against PRISM and IntervalMDP.jl instead.
- **No robust ω -regular / LRA on intervals.** Storm refuses LTL on interval models (“*We have not yet implemented LTL with intervals*”, SparseMdpPrctlModelChecker.cpp:239) and likewise interval LRA; those capabilities are IMPACT-only.

- **Point-only exact and CSL.** Storm’s exact engine and CSL operate on point models; IMPaCT additionally certifies exact robust values on interval models and solves interval CTMCs (uncertain rates), which Storm does not support.
- **No abstraction front end.** Storm consumes DRN/PRISM/JANI models; the continuous-dynamics-to-IMDP phase is IMPaCT-only.

3.25.4 Performance

Storm’s solver is very fast once the model is in memory, but it pays a model-construction cost to ingest the dense DRN. From the 2026-06-27 cross-tool run (seconds; IMPaCT II = interval iteration, VI = value iteration):

benchmark	IMPaCT robust solve (II / VI)	Storm VI	Storm build
AS	4.3 (finite)	0.21	0.50
BA	0.27 (finite)	5.86	6.05
VP	29.1 / 0.59	1.35	0.37
IC_reach	0.06 (finite)	0.03	0.05
IC_safe	0.47 (finite)	0.09	0.18
PD_p1	1.29 / 1.34	8.31	7.26
PD_p3	∞ (II) / 1.36	8.30	7.27

IMPaCT’s finite-horizon robust VI is competitive or faster (BA 0.27 s vs Storm’s 5.86 s + 6.05 s build), and its pure-VI mode beats Storm on VP (0.59 s vs 1.35 s) and PD (1.34–1.36 s vs ~8.3 s). Honest flip side: in the later sweep re-run (`sweep_arch.tsv`), Storm’s end-to-end times on several ARCH models are the best of all four tools (VP 0.04 s, AS 0.05 s, vs IMPaCT 1.8 s / 0.4 s) — and the abstraction that produced these IMDPs (e.g. AS 27 s dense / 0.11 s sparse) is an IMPaCT-side cost Storm never pays because it cannot abstract.

3.25.5 Reproduce

```
# build Storm 1.13.0 from source inside the impact_work container
bash benchmarks/crosstool/peers/build_storm.sh

# convert the exported IMDP to Storm DRN and run the sweep
python3 benchmarks/crosstool/peers/imdp_to_drn.py MODEL.imdp
python3 benchmarks/crosstool/peers/sweep_compare.py all
```

Raw results: `benchmarks/crosstool/sweep_all.tsv`, `sweep_arch.tsv`; per-model measured values: `benchmarks/crosstool/models/*.ref.json`.

3.26 IMPaCT vs IntervalMDP.jl

`IntervalMDP.jl` (compared at version 0.6.0 on Julia 1.10) is a Julia library dedicated to interval MDPs: robust value iteration built on O-maximization — the same inner solve IMPaCT uses — with CUDA acceleration and factored (orthogonal and mixture) IMDP representations for scalability. Of the three peers it is semantically closest to IMPaCT: both solve `Pessimistic/Optimistic × Maximize` queries, which map one-to-one to IMPaCT’s `--bound pess/opt`.

3.26.1 Feature overlap

IntervalMDP.jl has (IMPACT does not)	Both	IMPACT has (IntervalMDP.jl does not)
CUDA-native value iteration	Robust reachability / safety, bounded and unbounded	Abstraction of continuous nonlinear stochastic dynamics $\rightarrow$ IMPDP (6 noise models)
Expected exit-time queries	Discounted expected reward	Robust ω -regular (Büchi/persistence/patrol) and arbitrary det.-Büchi LTL (IntervalMDP.jl: DFA/co-safe reachability only)
netCDF / bmdp-tool input formats	Orthogonal and mixture IMPDPs (IMPACT .odimdp, added in v2.0)	Reachability rewards, long-run average, steady-state, multi-objective (all robust)
DFA-product (co-safe) specifications	GPU acceleration (CUDA vs any-vendor SYCL)	CTMC + CSL, interval CTMC, POMDP/PTA/TA, PRISM-subset input, statistical MC
—	Exact / rational arithmetic; strategy synthesis	Certified OVI bracket; interval iteration (sound two-sided bound)

3.26.2 Same results on IntervalMDP.jl's own example

multiObj_robotIMDP is the one interval-MDP example model shipped by a peer tool: a 207-state robot IMPDP (828 state-action rows, 2784 interval transitions, init state 0, target state 206), converted to .imdp with peers/tra_to_imdp.py. Query: robust reachability, Pessimistic, Maximize = IMPACT reach --bound pess.

solver	init value	iters	note
IntervalMDP.jl (default threshold 1e-6)	0.8946617847	83	stops 1.2e-6 below the fixpoint
IntervalMDP.jl (threshold 1e-9)	0.8946629814	99	
IntervalMDP.jl (threshold 1e-12)	0.8946629826	128	converged
IMPACT imdp_solve OVI (eps 1e-6)	0.8946629826	129	certified [L, L+eps]
IMPACT interval iteration (eps 1e-9)	0.8946629826	42177	sound bracket [0.8946629826, 0.8946629838]

The values agree to $\sim 1e-10$ once thresholds match. The apparent 1.2e-6 gap at defaults is purely IntervalMDP.jl's 1e-6 residual threshold terminating short of the fixpoint on this slowly-mixing chain — a methodological note, not a disagreement. IMPACT's OVI returns the converged value *and* a certified upper bound already at eps 1e-6. Both tools give the optimistic value 1.0 (IMPACT/Storm 0.9999979999, IntervalMDP.jl/PRISM 0.9999969998).

3.26.3 Factored IMPDPs: orthogonal and mixture

IMPACT v2.0 adopts IntervalMDP.jl's flagship scalability representations, validated against it as the oracle (peers/odimdp_oracle.jl, peers/mixture_oracle.jl):

- **Orthogonal IMPDP** (od_demo.odimdp): IMPACT matches OrthogonalIntervalMarkovDecisionProcess to 10 digits in both senses (state 0 pess 0.3714285709 / opt 0.8536585360; state 1 pess 0.7285714278 / opt 0.9999999995).
- **Mixture IMPDP** (mix_demo.odimdp): matches MixtureIntervalMarkovDecisionProcess to 10 digits both senses (pess 0.4756756752 / opt 0.9619450314).
- **Reduction-order semantics (ISSUE-0022)**. The robust value of a factored IMPDP depends on the order in which nature resolves the per-dimension marginals. IMPACT's first implementation reduced the highest dimension innermost and got pess 0.3478 vs IntervalMDP.jl's 0.3714 (optimistic values already matched). Reading IntervalMDP.jl's state_action_bellman identified its convention — dimension 1 innermost — which IMPACT now follows; agreement is only meaningful once this convention is pinned.

Discounted reward ($\gamma=0.9$, reward_chain): IMPACT pess 2.8 / opt 4.272726893 vs IntervalMDP.jl 2.8 / 4.272726893272438 — IntervalMDP.jl is the oracle here since its InfiniteTimeReward requires a discount < 1 and Storm’s interval rewards ignore the resolution mode.

3.26.4 Differences and caveats

- **Specification scope.** IntervalMDP.jl covers reachability, safety, DFA/co-safe reachability, discounted reward and exit time; no Büchi/persistence, LRA, multi-objective, CTMC or statistical MC. IMPACT’s robust ω -regular, LTL, LRA and multi-objective queries have no IntervalMDP.jl counterpart.
- **Optimistic-safety convention.** On BA’s $[0, hi]$ leak edges IntervalMDP.jl’s cooperative resolution reports opt-safety ≈ 0.997 where the PRISM/Storm 1–Pmin encodings give 0 (its BA robust safety, $1.9e-4$, still effectively agrees with the others’ 0). The robust value is unambiguous; compare that.
- **Thresholds.** Match convergence thresholds before comparing digits (see the robot example above).

3.26.5 Performance

The two tools’ solve times are close — expected, since both run robust VI over O-maximization. From the 2026-06-27 cross-tool run (seconds):

benchmark	IMPACT robust solve (II / VI)	IntervalMDP.jl VI
AS	4.3 (finite)	0.40
BA	0.27 (finite)	1.14
VP	29.1 / 0.59	0.61
IC_reach	0.06 (finite)	0.38
IC_safe	0.47 (finite)	0.42
PD_p1	1.29 / 1.34	1.33
PD_p3	∞ (II) / 1.36	1.40

VP 0.59 s vs 0.61 s and PD_p1 1.34 s vs 1.33 s are effectively ties; IntervalMDP.jl is clearly faster on AS (0.40 s vs the dense finite path’s 4.3 s — IMPACT’s sparse path later cut this to 0.07 s, see benchmarks/RESULTS.md). In the sweep TSVs, IntervalMDP.jl’s first query per model carries ~ 0.4 s of Julia runner start-up (its second query on the same 4-state model returns in ~ 0.001 s), so small-model timings favour the C++ tools. On the 207-state robot, IntervalMDP.jl solved in 1.2 ms vs IMPACT’s ~ 10 ms. The sparse-memory story is shared: IMPACT’s sparse path is “the IntervalMDP.jl-style memory win” — at 125k states the sparse model is ~ 255 MB where a dense matrix would be ~ 250 GB, and the 6-D LM case (dense-infeasible at ~ 96 GB/matrix) builds and solves at 196 MB.

3.26.6 Reproduce

```
# run IntervalMDP.jl on any exported .imdp (inside the impact_work container)
julia benchmarks/crosstool/peers/intervalmdp_runner.jl MODEL.imdp

# native robot model runs (default + tight threshold), factored oracles
julia benchmarks/crosstool/peers/imdp_native_robot.jl
julia benchmarks/crosstool/peers/imdp_native_robot_tight.jl
julia benchmarks/crosstool/peers/odimdp_oracle.jl

# full four-tool sweep
python3 benchmarks/crosstool/peers/sweep_compare.py all
```

Raw results: benchmarks/crosstool/sweep_all.tsv, sweep_arch.tsv; per-model measured values: benchmarks/crosstool/models/*.ref.json (see od_demo.ref.json for the factored-semantics note).

3.27 ARCH-COMP Benchmarks

IMPACT ships the ARCH-COMP 2025 stochastic-models benchmark suite under `examples/ARCH-COMP/2025/`. Each case abstracts a continuous stochastic system into an interval MDP (IMDP) and synthesizes a robust controller; the abstracted IMDP is exported so that peer tools (IntervalMDP.jl, PRISM, Storm) can solve the identical model for cross-tool validation. This section documents each case study, the code used to solve it, the measured results, and the design decisions behind them.

3.27.1 Benchmark overview

Case	Dim	Specification	Cells	Key result	Page
AS (anaesthesia)	3	finite-horizon reach, H=10	2460	matches IntervalMDP.jl to $\Delta \leq 4.8e-7$	AS
BA (building automation)	4	finite-horizon safety, H=6	1225	init safety 0.4533 in IMPACT / IntervalMDP.jl / Storm	BA
VP (Van der Pol)	2	infinite-horizon reach	2591	max reach 0.2297; VI solve 0.59 s (was 214 s)	VP
IC (integrator chain)	2	finite reach / safety, H=5	592 / 1681	matches IntervalMDP.jl to $\Delta \leq 6.3e-7$	IC
PD (package delivery)	2	infinite reach-avoid (p1, p3)	571	robust value 1.0 in all four tools	PD
PR_minimal (patrol robot)	2	infinite reach-avoid	1583	former dense-OOM case completes via sparse path (3.86 GB)	PR_minimal
AV_minimal (autonomous vehicle)	3	infinite reach-avoid	7938 (sparse)	builds where dense path OOMs; interior max reach 0.3398	AV_minimal
LM (lane merging)	6	finite-horizon reach, H=200	12150 (sparse)	dense-infeasible (~96 GB/matrix); sparse 196 MB, ~40 s	LM

All robust (pessimistic) values agree across IMPACT, IntervalMDP.jl, PRISM 4.8.1 and Storm 1.13.0 wherever a comparison is applicable; details are on each page and in the top-level `TOOL_COMPARISONS.md`.

3.27.2 Building and running the dense examples

Each benchmark directory contains a Makefile that compiles the example against the IMPACT sources with AdaptiveCpp (SYCL):

```
cd examples/ARCH-COMP/2025/AS # or BA, VP, IC, PD, PR_minimal, AV_minimal, LM
make
./AS # executable named after the .cpp file
```

The Makefile uses `acpp` with an OpenMP target by default and links NLOpt, HDF5, GLPK, GSL and Armadillo:

```
CC = acpp # or syclcc (then remove --acpp-targets="omp")
CFLAGS = --acpp-targets="omp" -O3 -lnlopt -lm -I/usr/include/hdf5/serial \
-L/usr/lib/$(ARCH)-linux-gnu/hdf5/serial -lhdf5 -lglpk -lgsl -lgslcblas \
-DH5_USE_110_API -larmadillo
```

Each example exports its abstracted IMDP (`mdp.exportIMDP("<name>.imdp")`) before synthesis, which is the input used for the cross-tool comparison.

3.27.3 Building and running the sparse driver

`benchmarks/sparse_arch.cpp` re-runs all cases through IMPACT's sparse abstraction (`src/abstraction.cpp`, O(nnz) memory) and the sparse OVI solver (`src/solve.cpp`). It is pure standard C++ — no SYCL or Armadillo required:

```
c++ -std=c++17 -O2 benchmarks/sparse_arch.cpp \
    src/abstraction.cpp src/solve.cpp src/omaximization.cpp src/graph_utils.cpp \
    -o /tmp/sparse_arch
```

Run modes:

```
/tmp/sparse_arch          # FAST enclosure (corner-min + per-dimension max)
/tmp/sparse_arch exact    # EXACT/TIGHT enclosure (K=5-per-dim sub-grid)
/tmp/sparse_arch exactK=7 # custom sub-grid density
/tmp/sparse_arch only=VP  # run a single benchmark by name
```

The driver aligns the sparse grid to the dense point-grid convention (cells centred on $lb + k \cdot \eta$, $(ub-lb)/\eta + 1$ cells per dimension), so cell counts match the dense abstraction exactly and values are directly comparable. For diagonal (uncoupled) dynamics matrices the sparse mean enclosure is exact; for coupled dynamics it is a sound over-approximation (see the BA page). For nonlinear dynamics the FAST/EXACT modes control the tightness of the sampled mean enclosure (see the VP page).

3.27.4 Cross-tool validation

Only IMPACT can abstract continuous dynamics into an IMPDP; the peers solve IMPACT's exported model. The pipeline is: `exportIMPDP` $\rightarrow$ neutral `.imdp` file $\rightarrow$ `imdp_to_drn.py` (Storm), `imdp_to_prism_interval.py` (PRISM), `intervalmdp_runner.jl` (IntervalMDP.jl), driven end-to-end by `benchmarks/crosstool/peers/sweep_compare.py`. All four tools agree on every ARCH robust value; under the native finite horizons IMPACT's per-cell robust values match IntervalMDP.jl to $\leq 2.3e-5$ ($\leq 7e-7$ for the finite-horizon cases).

3.28 AS — Anaesthesia Delivery (3D)

The AS benchmark models automated anaesthesia delivery as a three-compartment pharmacokinetic system with affine (linear) dynamics, two control inputs and additive Gaussian noise. The specification is finite-horizon reachability: steer the drug concentrations into a target box within 10 steps. Source: `examples/ARCH-COMP/2025/AS/AS.cpp`.

3.28.1 System

- **State** (3D): compartment concentrations, gridded over $[1, 6] \times [0, 10] \times [0, 10]$ with steps $\{0.25, 1, 1\}$.
- **Input** (2D): infusion rates over $[0, 7] \times [0, 30]$ with steps $\{1, 30\}$.
- **Noise**: additive Gaussian, standard deviation $\sqrt{0.001}$ in each dimension.
- **Specification**: reach the target box $[4, 6] \times [8, 10] \times [8, 10]$ within horizon $H = 10$ (robust/pessimistic).

The dynamics are affine in the state and inputs:

```
auto dynamics = [](const vec& x, const vec& u) -> vec {
    vec xx(dim_x);
    xx[0] = 0.8192*x[0] + 0.03412*x[1] + 0.01265*x[2] + 0.01883*(u[0] + u[1]);
    xx[1] = 0.01646*x[0] + 0.9822*x[1] + 0.0001*x[2] + 0.0002*(u[0] + u[1]);
    xx[2] = 0.0009*x[0] + 0.00002*x[1] + 0.9989*x[2] + 0.00001*(u[0] + u[1]);
    return xx;
};
```

3.28.2 Code walkthrough

Create the IMPDP object and discretize the state and input spaces:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setInputSpace(is_lb, is_ub, is_eta);
mdp.setTargetSpace(target_condition, true); // label target cells, make absorbing
```

Attach the dynamics and the noise model:

```
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);
```

Build the abstraction — transition-probability intervals to the target region, to the avoid region (here: leaving the grid), and between all cell pairs:

```
mdp.targetTransitionVectorBounds();
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
```

Export the abstracted IMPD for cross-tool comparison, then synthesize the finite-horizon controller (`true` = pessimistic/robust) and save it:

```
mdp.exportIMDP("AS.imdp");
mdp.finiteHorizonReachControllerSorted(true, 10);
mdp.saveController();
```

Build and run with `make && ./AS` in `examples/ARCH-COMP/2025/AS`.

3.28.3 Sparse-abstraction route

AS is affine in the state, so the per-cell mean range is exact and the sparse abstraction (`benchmarks/sparse_arch.cpp`, run with `only=AS`) has no accuracy loss per dimension. The driver aligns the sparse grid to the dense point-grid convention so the cell counts match the dense abstraction exactly. Note that AS has a *coupled* (non-diagonal) dynamics matrix: the sparse per-dimension product bound is sound but slightly conservative compared with the dense joint NLopt optimisation (see the BA page and ISSUE-0020 for the analysis).

3.28.4 Results

Dense SYCL path (AdaptiveCpp/OMP, 56 GB host):

cells	abstraction (s)	synthesis (s)
2460	27.0	4.3

Sparse path (single-threaded, pure C++): 0.83M nonzeros, **19.9 MB** versus a 1.02 GB dense matrix, building in 0.11 s and solving in **0.07 s** (versus 4.3 s dense SYCL). After grid alignment the sparse interior mean is 0.0019 against the dense (NLopt) 0.0045 — sound but conservative, as expected for coupled dynamics.

3.28.5 Cross-tool validation

The exported `AS.imdp` was solved by IntervalMDP.jl, PRISM and Storm on the identical model. Under the native finite horizon ($H = 10$) IMPACT's per-cell robust values match IntervalMDP.jl to $\Delta \leq 4.8\text{e-}7$. On the infinite-horizon reachability reduction all four tools return robust value 0 at the initial state (nothing is robustly reached in the limit), with solve times 0.4 / 0.5 / 0.5 / 0.05 s (IMPACT / IntervalMDP.jl / PRISM / Storm).

3.28.6 Notes

- The dense finite-horizon path recomputed a large diff matrix each sweep; the loop-invariant hoist was applied to the infinite-horizon functions, which is why the sparse solve (0.07 s) is much faster than the dense SYCL solve (4.3 s) here.
- PRISM rejects zero lower-bound interval edges, so its run used intervals floored to $[1e-9, hi]$ — an ε -perturbation of the exported model.

3.29 BA — Building Automation (4D)

The BA benchmark is a four-dimensional building-automation (temperature-control) system with affine dynamics, one heating input and additive Gaussian noise. The specification is finite-horizon safety: remain inside the temperature envelope for 6 steps. Source: `examples/ARCH-COMP/2025/BA/BA_4d.cpp`.

3.29.1 System

- **State** (4D): two zone temperatures and two heater states, gridded over $[19, 21] \times [19, 21] \times [30, 36] \times [30, 36]$ with steps $\{0.5, 0.5, 1, 1\}$.
- **Input** (1D): supply temperature over $[17, 20]$ with step 1.
- **Noise**: additive Gaussian with per-dimension standard deviations `sqrt(0.0774)`, `sqrt(0.0774)`, `sqrt(0.3872)`, `sqrt(0.3098)`.
- **Specification**: finite-horizon safety, $H = 6$ — stay within the state grid (leaving the grid is the avoid event).

The dynamics are affine, $\mathbf{x}' = \mathbf{A} \mathbf{x} + \mathbf{B} \mathbf{u} + \mathbf{Q}$:

```
auto dynamics = [](const vec& x, const vec& u) -> vec {
    vec xx(dim_x);
    const mat A = {{0.6682, 0.0, 0.02632, 0.0},
                   {0.0, 0.6830, 0.0, 0.02096},
                   {1.0005, 0.0, -0.000499, 0.0},
                   {0.0, 0.8004, 0.0, 0.1996}};
    const vec B = {0.1320, 0.1402, 0.0, 0.0};
    const vec Q = {3.4378, 2.9272, 13.0207, 10.4166};
    xx = A*x + B*u + Q;
    return xx;
};
```

3.29.2 Code walkthrough

Set up the spaces, dynamics and noise. A safety specification needs no target set — only the probability intervals of *leaving* the safe grid:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setInputSpace(is_lb, is_ub, is_eta);
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);
```

Abstract the avoid vectors (probability of exiting the grid) and the cell-to-cell transition matrices:

```
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
```

Export the IMDP and synthesize the robust finite-horizon safety controller:

```
mdp.exportIMDP("BA.imdp");
mdp.finiteHorizonSafeControllerSorted(true, 6); // true = pessimistic
mdp.saveController();
```

Build and run with `make && ./BA_4d` in `examples/ARCH-COMP/2025/BA`.

3.29.3 Sparse-abstraction route

BA is affine, so it runs through `benchmarks/sparse_arch.cpp` (only=BA) with an exact per-dimension mean range. With the grid aligned to the dense point-grid convention the sparse cell count matches the dense abstraction exactly (1225 cells).

BA is the canonical *coupled-dynamics* case: because **A** is non-diagonal, the sparse box bound (product of per-dimension intervals) is a **sound over-approximation** — the per-dimension worst cases are not jointly attainable — while the dense joint NLOpt optimisation is tighter. A one-step brute-force check on a corner cell gives a true value of 0.2610 versus the sparse bound 0.2664 (+0.0054): both sound, sparse slightly conservative. Grid alignment moved the sparse interior safety value from 0.107 to 0.348, against the dense (NLOpt) 0.471.

3.29.4 Results

Dense SYCL path (AdaptiveCpp/OMP, 56 GB host):

cells	abstraction (s)	synthesis (s)
1225	7.6	0.27

Sparse path (logged run): 1.09M nonzeros, 26.2 MB (dense 0.02 GB), build + solve 0.25 + 0.22 s.

3.29.5 Cross-tool validation

The exported `BA.imdp` was solved identically by `IntervalMDP.jl`, `PRISM` and `Storm`. Under the native finite horizon the initial-state robust safety value is **0.4533 in all three tools** (IMPACT, `IntervalMDP.jl`, `Storm`), and IMPACT's per-cell robust values match `IntervalMDP.jl` to $\Delta \leq 4.3e-7$. On the infinite-horizon safety reduction the robust value is 0, matching `PRISM` and `Storm` (verified at eps 1e-4); `IntervalMDP.jl` reports 1.9e-4 at its defaults. Peer solve times: 4.9 / 0.3 / 0.7 s (`IntervalMDP.jl` / `PRISM` / `Storm`).

3.29.6 Notes

- IMPACT's *optimistic* infinite-horizon safety OVI did not converge within 180 s on this model — a large safe end component (ISSUE-0010, since resolved: topological Gauss–Seidel brings the eps=1e-6 robust solve to 7.8 s). The robust (pessimistic) value, which is the headline guarantee, is unambiguous and agrees across tools.
- Optimistic safety is convention-dependent across tools (how the cooperative sense resolves `[0,hi]` leak edges); only the robust value should be compared directly.
- BA's noise window spans roughly 7 cells against a 4-cell domain, so low finite-horizon safety values are expected on this coarse grid.

3.30 VP — Van der Pol Oscillator (2D)

The VP benchmark is an autonomous (input-free) two-dimensional Van der Pol oscillator, discretized in time with step 0.1, under additive Gaussian noise. The specification is infinite-horizon reachability of a small target box, making this a verification (rather than control-synthesis) problem and the primary validation case for IMPACT's nonlinear abstraction. Source: `examples/ARCH-COMP/2025/VP/VP.cpp`.

3.30.1 System

- **State** (2D): gridded over $[-5, 5] \times [-5, 5]$ with step 0.2 per dimension.
- **Input**: none ($\text{dim}_u = 0$).
- **Noise**: additive Gaussian, standard deviation $\sqrt{0.04}$ per dimension. (A custom uniform PDF is included in the source, commented out, as an example of `NoiseType::CUSTOM`.)
- **Specification**: infinite-horizon reachability of $[-1.2, -0.9] \times [-2.9, -2.0]$ (robust/pessimistic).

The dynamics are the Euler-discretized Van der Pol equations — genuinely nonlinear:

```
auto dynamics = [](const vec& x) -> vec {
    vec xx(dim_x);
    xx[0] = x[0] + 0.1*x[1];
    xx[1] = x[1] + (-x[0] + (1 - x[0]) * (1 - x[0]) * x[1]) * 0.1;
    return xx;
};
```

3.30.2 Code walkthrough

No input space is set — this is verification of an autonomous system:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);
mdp.setTargetSpace(target_condition, true);
```

Abstraction — because the dynamics are nonlinear, the per-cell transition bounds are computed by NLOpt (a local optimiser, LN_SBPLX by default; `setAlgorithm` can change it):

```
mdp.targetTransitionVectorBounds();
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
mdp.exportIMDP("VP.imdp");
```

Infinite-horizon synthesis. The default solver is sound interval iteration; the example notes that `ValueIteration` (peer-style pure VI) uses far fewer sweeps:

```
//mdp.setIterationMethod(IterationMethod::ValueIteration); // optional
mdp.infiniteHorizonReachControllerSorted(true); // true = pessimistic
mdp.saveController();
```

Build and run with `make && ./VP` in `examples/ARCH-COMP/2025/VP`.

3.30.3 Sparse-abstraction route

VP is the validation case for the sparse *nonlinear* path in `benchmarks/sparse_arch.cpp` (only=VP). The mean range over each source cell is estimated by evaluating the dynamics on a K-per-dimension sub-grid:

- **FAST** (default): evaluate at the 2^{dim} cell corners;
- **EXACT** (exact, $K=5$, or `exactK=K`): a K-per-dimension sub-grid giving a tight joint min/max.

mode	cells	nnz	sparse (MB)	build + solve	interior max	dense ref
FAST	2601	0.20M	4.8	0.1 + 0.5 s	0.1985	0.2297
EXACT K=7/11	2601	0.20M	4.8	0.6 + 0.5 s	0.2143	0.2297

The enclosure is K-stable at $K = 7$ (unchanged at $K = 11$) and close to the dense/IntervalMDP.jl reference 0.2297. The residual gap is the finite sub-grid under-sampling the optimum the dense continuum optimiser finds — expected for non-affine dynamics (ISSUE-0021: for nonlinear f both the sampled sparse enclosure and the dense NLOpt bound are heuristic; the validation is K-convergence plus agreement with the dense/peer value).

3.30.4 Results

Dense SYCL path (AdaptiveCpp/OMP, 56 GB host):

cells	abstraction (s)	synthesis (s)
2591	1.9	29.1 (interval iteration) / 0.59 (value iteration; was 214)

The maximum reachability value over the grid is **0.2297** (matched by the peer tools). The value-iteration solve at 0.59 s is at or below the peers' solve times.

3.30.5 Cross-tool validation

The exported `VP.imdp` was solved by IntervalMDP.jl, PRISM and Storm on the identical model. IMPACT's per-cell robust values match IntervalMDP.jl to $\Delta \leq 2.3e-5$. The robust value at the initial state is 0 in all four tools, with solve times 1.8 / 0.55 / 1.4 / 0.04 s (IMPACT / IntervalMDP.jl / PRISM / Storm).

3.30.6 Notes

- VP was the motivating case for adding `IterationMethod::ValueIteration`: interval iteration is a sound two-sided bracket but took 214 s before the loop-invariant hoist (now 29.1 s); pure VI converges in 0.59 s with residual-based stopping.
- ISSUE-0021 tracks a certified interval-arithmetic mean enclosure for nonlinear dynamics, which would replace the sampled (heuristic) enclosure on the sparse path.

3.31 IC — Integrator Chain (2D)

The IC benchmark is a discrete-time double integrator (a two-dimensional integrator chain) with one bounded input and additive Gaussian noise. It comes in two variants sharing the same dynamics: **IC_reach** (finite-horizon reachability of a central box) and **IC_safe** (finite-horizon safety). Sources: `examples/ARCH-COMP/2025/IC/IC_reach_2d.cpp` and `IC_safe_2d.cpp`.

3.31.1 System

- **State** (2D): position and velocity, gridded over $[-10, 10] \times [-10, 10]$ with step 0.5 per dimension.
- **Input** (1D): acceleration over $[-1, 1]$ with step 0.5.
- **Noise**: additive Gaussian, standard deviation $\sqrt{0.01}$ per dimension.
- **Specifications** (both robust/pessimistic, $H = 5$):
 - *IC_reach*: reach the box $[-8, 8] \times [-8, 8]$ within 5 steps;
 - *IC_safe*: remain within the state grid for 5 steps.

The dynamics are the sampled double integrator with sampling time $N_s = 0.1$:

```
auto dynamics = [](const vec& x, const vec& u) -> vec {
    vec xx(dim_x);
    float Ns = 0.1;
    xx[0] = x[0] + Ns*x[1] + (Ns*Ns)/2.0*u[0];
    xx[1] = x[1] + Ns*u[0];
    return xx;
};
```

3.31.2 Code walkthrough

Both variants share the setup:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setInputSpace(is_lb, is_ub, is_eta);
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);
```

IC_reach labels the target region, abstracts target/avoid vectors and the transition matrices, then synthesizes a robust finite-horizon reach controller:

```
mdp.setTargetSpace(target_condition, true);
mdp.targetTransitionVectorBounds();
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
mdp.exportIMDP("IC_reach.imdp");
mdp.finiteHorizonReachControllerSorted(true, 5);
mdp.saveController();
```

IC_safe needs no target set — only the avoid vectors (probability of leaving the grid) and the transition matrices — and calls the safety synthesizer:

```
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
mdp.exportIMDP("IC_safe.imdp");
mdp.finiteHorizonSafeControllerSorted(true, 5);
mdp.saveController();
```

Build and run with `make && ./IC_reach_2d` (or `./IC_safe_2d`) in `examples/ARCH-COMP/2025/IC`.

3.31.3 Sparse-abstraction route

The integrator is affine with dynamics matrix $A = \begin{bmatrix} 1 & 0.1 \\ 0 & 1 \end{bmatrix}$, so the sparse path in `benchmarks/sparse_arch.cpp` (only=IC_reach / only=IC_safe) computes the per-dimension mean range exactly. With the grid aligned to the dense point-grid convention the sparse cell counts match the dense exactly (592 reach / 1681 safe).

bench	cells x act	nnz	sparse	dense	build + solve	sparse interior mean	dense (nlopt)
IC_reach (fin H5)	1600 x 5	0.04M	1.0 MB	0.20 GB	0.01 + 0.00 s	0.057	0.072
IC_safe (fin H5)	1600 x 5	0.13M	3.0 MB	0.20 GB	0.02 + 0.01 s	0.750	0.761

IC_safe matches the dense values closely; the small IC_reach gap follows the coupled-A analysis on the BA page (the sparse per-dimension product bound is sound but slightly conservative when A is not diagonal).

3.31.4 Results

Dense SYCL path (AdaptiveCpp/OMP, 56 GB host):

case	cells	abstraction (s)	synthesis (s)
IC_reach	592	1.8	0.06
IC_safe	1681	2.7	0.47

3.31.5 Cross-tool validation

The exported IMDPs were solved by IntervalMDP.jl, PRISM and Storm on the identical models. IMPACT’s per-cell robust values match IntervalMDP.jl to $\Delta \leq 6.3e-7$ for both variants. On the infinite-horizon reductions all four tools agree: robust reach and safety are 0 at the initial state, and even the tiny optimistic reach value agrees to high precision (IC_reach opt $\approx 1.20e-26$ in all four). Solve times (IMPACT / IntervalMDP.jl / PRISM / Storm): IC_reach 0.1 / 0.4 / 0.2 / 0.05 s; IC_safe 0.15 / 0.5 / 0.06 / 0.02 s.

3.31.6 Notes

- The v1 status of IC was “reduced (scale)”; the v2 runs above cover the 2D configuration under both its native finite-horizon specs.
- PRISM required flooring $[0, hi]$ interval edges to $[1e-9, hi]$ (it rejects a zero lower bound), so its values are an ε -perturbation of the exported model.

3.32 PD — Package Delivery (2D)

The PD benchmark is a two-dimensional package-delivery robot with stable diagonal affine dynamics, two inputs and additive Gaussian noise. The ARCH-COMP specification is co-safe LTL; the shipped examples collapse it into two infinite-horizon reach-avoid problems with different targets — **PD_p1** (deliver to the first drop-off) and **PD_p3** (deliver to the second) — around a common obstacle. Sources: `examples/ARCH-COMP/2025/PD/PD_target_p1.cpp` and `PD_target_p3.cpp`.

3.32.1 System

- **State** (2D): position, gridded over $[-6, 6] \times [-6, 6]$ with step 0.5.
- **Input** (2D): velocity commands over $[-1, 1] \times [-1, 1]$ with step 0.1 (441 actions).
- **Noise**: additive Gaussian, standard deviation $\sqrt{0.02}$ per dimension.
- **Specifications** (infinite-horizon, robust reach-avoid):
 - *PD_p1*: reach $[5, 6] \times [-1, 1]$, avoid $[0, 1] \times [-5, 1]$;
 - *PD_p3*: reach $[-4, -2] \times [-4, -3]$, avoid $[0, 1] \times [-5, 1]$.

The dynamics are diagonal affine (each coordinate contracts independently):

```
auto dynamics = [](const vec& x, const vec& u) -> vec {
    vec xx(dim_x);
    xx[0] = 0.9*x[0] + 1.4*u[0];
    xx[1] = 0.8*x[1] + 1.4*u[1];
    return xx;
};
```

3.32.2 Code walkthrough

The reach-avoid specification is set in one call, which labels target and avoid cells and makes them absorbing:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setInputSpace(is_lb, is_ub, is_eta);
mdp.setTargetAvoidSpace(target_condition, avoid_condition, true);
```

(continues on next page)

(continued from previous page)

```
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);
```

Abstraction and export follow the standard pattern:

```
mdp.targetTransitionVectorBounds();
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
mdp.exportIMDP("PD_p1.imdp");    // "PD_p3.imdp" in the p3 variant
```

Infinite-horizon synthesis. On PD_p3 the default interval iteration does **not** converge (see Notes); switching to ValueIteration (or the OVI dispatch) does:

```
//mdp.setIterationMethod(IterationMethod::ValueIteration);    // needed for PD_p3
mdp.infiniteHorizonReachController(true);                      // true = pessimistic
mdp.saveController();
```

Build and run with make && ./PD_target_p1 (or ./PD_target_p3) in examples/ARCH-COMP/2025/PD.

3.32.3 Sparse-abstraction route

PD's A is diagonal, so the sparse abstraction in benchmarks/sparse_arch.cpp (only=PD_p1 / only=PD_p3) is **exact** — sparse equals dense (grid-aligned cell count 571, matching the dense abstraction).

bench	cells x act	nnz	sparse	dense	build + solve	interior mean
PD_p1 (inf reach-avoid)	576 x 441	5.1M	122 MB	2.34 GB	0.8 + 15 s	1.000 (dense 1.000)
PD_p3 (inf reach-avoid)	576 x 441	5.1M	122 MB	2.34 GB	0.7 + 1.1 s	1.000 (peers 1.0)

3.32.4 Results

Dense SYCL path (AdaptiveCpp/OMP, 56 GB host):

case	cells	abstraction (s)	synthesis (s)
PD_p1	571	37	1.29 (interval iteration) / 1.34 (value iteration; was 6.5)
PD_p3	571	~37	1.36 (value iteration; interval iteration non-convergent)

3.32.5 Cross-tool validation

The exported IMDPs were solved by IntervalMDP.jl, PRISM and Storm on the identical models. The robust reach value is **1.0 in all four tools** for both PD_p1 and PD_p3; for PD_p1 the per-cell robust values match IntervalMDP.jl to $\Delta \leq 7.1e-7$. Solve times (IMPACT / IntervalMDP.jl / PRISM / Storm): PD_p1 3.4 / 1.4 / 1.7 / 2.8 s; PD_p3 3.6 / 1.6 / 2.3 / 2.2 s.

3.32.6 Notes

- **ISSUE-0003 (nature trap):** PD_p3 exhibits a nature-confinable end component on which interval iteration's upper bound never contracts, so the default solver does not converge. Two remedies ship in IMPACT: `IterationMethod::ValueIteration` (converges, residual stopping) and **OptimisticVI** (OVI — default on the sparse path, also available on the dense path via `infiniteHorizonOVIDispatch`), which is both convergent and certified: dense-OVI on PD_p3 returns the bracket $[0.9998, 1.0]$ with gap $1e-5$ in 3.3 s. Empirically, MEC-collapse preprocessing does *not* fix this — support-graph MECs miss nature-confinable end components; OVI or robust-EC deflation is required.

- The on-disk examples are the co-safe LTL specification collapsed into separate reach-avoid targets (p1/p3); replacing this with a true DFA-product co-safe run is the planned Phase-2 form of the benchmark.

3.33 PR_minimal — Patrol Robot (2D)

The PR_minimal benchmark is a reduced two-dimensional patrol robot commanded by speed and heading, with wide additive Gaussian noise, under an infinite-horizon reach-avoid specification. It is the tool's canonical *memory-wall* case: the dense abstraction matrices reach 8.84 GB (an out-of-memory failure under a 4 GB cap, ISSUE-0006), and it is the case that motivated the sparse abstraction path. Source: `examples/ARCH-COMP/2025/PR_minimal/PR_minimal.cpp`.

3.33.1 System

- **State** (2D): position, gridded over $[-10, 10] \times [-10, 10]$ with step 0.5.
- **Input** (2D): speed and heading over $[-1, 1] \times [-1, 1]$ with step 0.1 (441 actions).
- **Noise**: additive Gaussian, standard deviation $\sqrt{1/1.3333}$ (≈ 0.87) per dimension — very wide relative to the 0.5 grid step.
- **Specification**: infinite-horizon reach-avoid — reach $[5, 8] \times [5, 8]$ while avoiding $[-1.5, 1.5] \times [-1.5, 1.5]$ (robust/pessimistic).

The dynamics are affine in the state (identity) with a nonlinear input map:

```
auto dynamics = [] (const vec& x, const vec& u) -> vec {
    vec xx(dim_x);
    xx[0] = x[0] + 2*u[0]*cos(u[1]);
    xx[1] = x[1] + 2*u[0]*sin(u[1]);
    return xx;
};
```

3.33.2 Code walkthrough

Standard reach-avoid setup, abstraction and export:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setInputSpace(is_lb, is_ub, is_eta);
mdp.setTargetAvoidSpace(target_condition, avoid_condition, true);
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);

mdp.targetTransitionVectorBounds();
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
mdp.exportIMDP("PR_minimal.imdp");

mdp.infiniteHorizonReachControllerSorted(true); // true = pessimistic
mdp.saveController();
```

Build and run with `make && ./PR_minimal` in `examples/ARCH-COMP/2025/PR_minimal`. Note that the dense path needs a large-memory host (see Results).

3.33.3 Sparse-abstraction route

Because the state part of the dynamics is the identity (diagonal), the sparse mean enclosure in `benchmarks/sparse_arch.cpp` (`only=PR_minimal`) is **exact** — the sparse abstraction matches the dense with no accuracy loss, at a fraction of the memory. The nonlinearity lives entirely in the input map, which is evaluated per discrete action. The driver skips `PR_minimal` in `fast` mode (its wide kernel makes it the slowest case); run it explicitly with `only=PR_minimal`.

3.33.4 Results

Dense SYCL path (AdaptiveCpp/OMP, 56 GB host): 1583 cells, abstraction ~533 s using two 8.84 GB dense matrices; the SYCL value iteration did not converge on this model. Under a 4 GB memory cap the dense path is killed outright — the v1 baseline builds a dense (state x input) x state matrix of $698103 \times 1583 \approx 8.84$ GB (ISSUE-0006).

Sparse path (single-threaded, pure C++):

cells x act	nnz	sparse	dense	build + solve	interior value
1600 x 441	161M	3.86 GB	18.1 GB	28.5 + 814 s	~1.0

This is the headline of the sparse rerun: **the ISSUE-0006 dense-OOM case now completes** (3.86 GB versus 18 GB; it would not run at all on the dense path). The 814 s solve time reflects the very wide noise kernel — $\sigma \approx 0.87$ against $\eta = 0.5$ yields ~217 nonzeros per row — under single-threaded optimistic value iteration.

3.33.5 Cross-tool validation

`PR_minimal` appears in the cross-tool study as an *abstraction-scalability* case: the peer tools (IntervalMDP.jl, PRISM, Storm) cannot abstract continuous dynamics at all, so building this IMDP is outside their scope, and it is not part of the four-tool solve sweep. The exported `.imdp` uses the same neutral format validated on the other ARCH cases, where all four tools agree on every robust value.

3.33.6 Notes

- **ISSUE-0006:** the dense abstraction stores a (state x input) x state matrix for each of the min/max bounds; `PR_minimal` was the smallest shipped example that made this infeasible on ordinary hardware, confirming the need for sparse storage.
- The grid-aligned sparse abstraction is exact here (diagonal state dynamics), so the sparse route is strictly preferable: identical model, 5x less memory, and it completes the synthesis the dense SYCL solver could not.

3.34 AV_minimal — Autonomous Vehicle (3D)

The `AV_minimal` benchmark is a reduced three-dimensional autonomous vehicle — a kinematic bicycle model with position and heading states, speed and steering inputs, and wide additive Gaussian noise — under an infinite-horizon reach-avoid specification. Like `PR_minimal` it is primarily an abstraction-scalability case: the dense matrices are 7.42 GB each, while the sparse path builds and solves the model. Source: `examples/ARCH-COMP/2025/AV_minimal/AV_minimal.cpp`.

3.34.1 System

- **State** (3D): planar position and heading, gridded over $[-5, 5] \times [-5, 5] \times [-3.4, 3.4]$ with steps $\{0.5, 0.5, 0.4\}$.
- **Input** (2D): speed $[-1, 4]$ (step 1) and steering angle $[-0.4, 0.4]$ (step 0.2) — 30 discrete actions.
- **Noise:** additive Gaussian, standard deviation $\sqrt{1/1.5}$ (≈ 0.82) per dimension — wide relative to the 0.5 grid step.

- **Specification:** infinite-horizon reach-avoid — reach the lane region $[-5.75, 0.25] \times [-0.25, 5.75] \times [-3.45, 3.45]$ while avoiding the adjacent strip $[-5.75, 0.25] \times [-0.75, -0.25] \times [-3.45, 3.45]$ (robust/pessimistic).

The dynamics are the nonlinear kinematic bicycle, sampled at $T_s = 0.1$:

```
const auto dynamics = [](const vec& x, const vec& u) -> vec {
    float Ts = 0.100;
    vec f(3);
    vec xx = x;
    f[0] = u[0]*cos(atan((float)(tan(u[1])/2.0))+xx[2])/
    ↪ cos((float)atan((float)(tan(u[1])/2.0)));
    f[1] = u[0]*sin(atan((float)(tan(u[1])/2.0))+xx[2])/
    ↪ cos((float)atan((float)(tan(u[1])/2.0)));
    f[2] = u[0]*tan(u[1]);
    xx = xx + Ts*f;
    return xx;
};
```

3.34.2 Code walkthrough

Standard reach-avoid pipeline — spaces, specification labelling, dynamics/noise, abstraction, export, synthesis:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setInputSpace(is_lb, is_ub, is_eta);
mdp.setTargetAvoidSpace(target_condition, avoid_condition, true);
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);

mdp.targetTransitionVectorBounds();
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();
mdp.exportIMDP("AV_minimal.imdp");

mdp.infiniteHorizonReachControllerSorted(true); // true = pessimistic
mdp.saveController();
```

Build and run with `make && ./AV_minimal` in `examples/ARCH-COMP/2025/AV_minimal`.

3.34.3 Sparse-abstraction route

AV_minimal is nonlinear in the state (through the heading), so the sparse path in `benchmarks/sparse_arch.cpp` (only=AV_minimal) uses the sampled mean enclosure: FAST mode evaluates the dynamics at the source-cell corners; EXACT mode (exact / exactK=K) uses a K-per-dimension sub-grid for a tighter joint enclosure. For non-affine dynamics this enclosure is heuristic (ISSUE-0021), validated by K-convergence on the VP benchmark.

3.34.4 Results

Dense SYCL path (56 GB host): 5562 cells; the two dense matrices are 7.42 GB each, so the abstraction fits in 14.8 GB on the big machine (it OOMs on ordinary hosts); no dense synthesis result is reported.

Sparse path (FAST mode, single-threaded):

cells	actions	nnz	sparse	dense est.	build + solve	interior reach
7938	30	324M	7784 MB	30.3 GB	420 + 1161 s	[0, 0.3398 , mean 0.031]

The abstraction **builds where the dense path OOMs** (7.78 GB versus an estimated 30 GB dense) and solves robustly to an interior maximum reach of 0.3398.

3.34.5 Cross-tool validation

AV_minimal appears in the cross-tool study as an *abstraction-scalability* case: building the IMDP from continuous dynamics is outside the scope of IntervalMDP.jl, PRISM and Storm, and this model is not part of the four-tool solve sweep. The exported .imdp uses the same neutral format validated on the smaller ARCH cases, where all four tools agree on every robust value.

3.34.6 Notes

- The wide noise ($\sigma \approx 0.82$ against $\eta = 0.5$) gives a kernel of roughly 10 cells per dimension, hence $\sim 10^3$ successors per row; that makes the single-threaded OVI solve heavy (1161 s), similar to PR_minimal.
- The modest robust reach value reflects the noise magnitude relative to the grid: with noise this wide it is genuinely hard to *robustly* steer into the target, so a low guaranteed lower bound is the sound answer, not an artefact.
- ISSUE-0021 (certified interval-arithmetic mean enclosure) applies to this case: the FAST corner sampling is a heuristic enclosure for non-affine dynamics.

3.35 LM — Lane Merging Vehicle (6D)

The LM benchmark is a six-dimensional nonlinear lane-merging vehicle with two wheel inputs, mostly deterministic dynamics (noise in only two of six dimensions), and a long-horizon reachability specification ($H = 200$). It is the headline scalability case of the suite: the dense abstraction is infeasible at roughly 96 GB per matrix, while the sparse path builds and solves the model in about 40 s at 196 MB. Source: `examples/ARCH-COMP/2025/LM/LM.cpp`.

3.35.1 System

- **State** (6D): yaw rate, lateral velocity, planar position and two angle states, gridded over $[-0.5, 0.5] \times [-0.5, 0.5] \times [-10, 4] \times [-5, 4] \times [-3.14, 3.14] \times [-3.14, 3.14]$ with steps $\{0.5, 0.5, 1, 1, 3.14, 3.14\}$.
- **Input** (2D): wheel speeds over $[0.5, 3.5] \times [0.5, 3.5]$ with step 1.5 per dimension.
- **Noise**: Gaussian with standard deviation $\{0, 0, 0, 0, 0.001, 0.001\}$ — four of the six dimensions are deterministic.
- **Specification**: finite-horizon reach-avoid with $H = 200$ — reach the merge point (a box collapsed to the corner $x_2 = -10, x_3 = -5$) while avoiding the occupied lane region $[-0.5, 0.5] \times \dots \times [-4, 4] \times [2.5, 4] \times [-3.14, 3.14] \times [-3.14, 3.14]$.

The dynamics are a nonlinear single-track vehicle model sampled at $T_s = 0.1$ (excerpt; see the source for all six equations and constants):

```
const auto dynamics = [](const vec& x, const vec& u) -> vec {
    float Ts = 0.1; float M = 700; /* D, a, b, d, Afy, Izz, Cgamma ... */
    vec xx = x;
    xx[0] = x[0] + (Ts/Izz)*(Afy*sin(x[5]+(pi/2))*a
        + 2*Cgamma*tan((2*(x[1]-b*x[0]))/(u[0]+u[1])))*b);
    // xx[1] .. xx[5]: lateral force balance, planar kinematics, angle updates
    xx[2] = x[2] + (Ts*(u[0]+u[1])/2)*cos(x[4]);
    xx[3] = x[3] + (Ts*(u[0]+u[1])/2)*sin(x[4]);
    return xx;
};
```

3.35.2 Code walkthrough

The example follows the standard reach-avoid pipeline; the only difference from the smaller cases is the long horizon in the synthesis call:

```
IMDP mdp(dim_x, dim_u, dim_w);
mdp.setStateSpace(ss_lb, ss_ub, ss_eta);
mdp.setInputSpace(is_lb, is_ub, is_eta);
mdp.setTargetAvoidSpace(target_condition, avoid_condition, true);
mdp.setDynamics(dynamics);
mdp.setNoise(NoiseType::NORMAL);
mdp.setStdDev(sigma);

mdp.targetTransitionVectorBounds();
mdp.minAvoidTransitionVector();
mdp.maxAvoidTransitionVector();
mdp.transitionMatrixBounds();           // infeasible on the dense path (~96 GB/matrix)
mdp.exportIMDP("LM.imdp");

mdp.finiteHorizonReachControllerSorted(true, 200); // true = pessimistic
mdp.saveController();
```

In practice the dense build above cannot complete — use the sparse route below.

3.35.3 Sparse-abstraction route

LM is only feasible through `benchmarks/sparse_arch.cpp` (only=LM). The dynamics are nonlinear, so the sparse path samples the mean over each source cell: FAST mode uses the 2^6 cell corners, EXACT mode (exact / exactK=K) a K-per-dimension sub-grid (a heuristic enclosure for non-affine dynamics, ISSUE-0021). The driver grid-aligns to the dense point-grid convention, applies the avoid region as dead absorbing cells, and runs a 200-step robust value iteration.

3.35.4 Results

Dense SYCL path: **infeasible** — 36450 cells, ~96 GB per dense matrix.

Sparse path (FAST mode, single-threaded):

cells	actions	nnz	sparse	dense est.	build + solve	interior max
12150	9	8.19M	196 MB	21.3 GB	24.8 + 14.9 s	0.0000

The 6-D case that is dense-infeasible builds **and** solves through the sparse path in about 40 s at 196 MB total.

3.35.5 Cross-tool validation

LM appears in the cross-tool study as an *abstraction-scalability* case: no peer tool (IntervalMDP.jl, PRISM, Storm) can abstract continuous dynamics, and the dense model never existed to export, so LM is not part of the four-tool solve sweep. The sparse `.imdp` export uses the same neutral format validated on the smaller ARCH cases.

3.35.6 Notes

- **The robust value 0.0000 is sound but vacuous.** Four of six dimensions have $\sigma = 0$ (deterministic), so on this coarse grid the corner means of a cell scatter into different destination cells, producing transition intervals $[0, 1]$; the pessimistic (robust) lower bound of any such interval chain is 0. A finer grid and/or the certified interval-arithmetic enclosure of ISSUE-0021 is needed to obtain a non-trivial robust value on near-deterministic systems.

- LM demonstrates why sparse storage is a prerequisite for higher-dimensional benchmarks: nnz grows linearly in the number of cells (8.19M here), while the dense min/max matrices grow quadratically (the driver's dense estimate is 21.3 GB, and the dense IMPDP class would need ~96 GB per matrix on its grid).

3.36 MDP (class MDP)

class **MDP**

Base class for Markov Decision Processes. Manages state, input, and disturbance spaces, system dynamics, noise distributions, and transition data.

Defined in `src/MDP.h`.

3.36.1 Constructor

MDP(const int x, const int u, const int w)

Create an MDP with the specified dimensions.

Parameters

- **x** – Number of state space dimensions.
- **u** – Number of input space dimensions (0 for no inputs).
- **w** – Number of disturbance space dimensions (0 for no disturbance).

3.36.2 State Space

void **setStateSpace**(vec lb, vec ub, vec eta)

Define the state space as a discretized hyper-rectangle.

Parameters

- **lb** – Lower bounds vector (length `dim_x`).
- **ub** – Upper bounds vector (length `dim_x`).
- **eta** – Discretization step size vector (length `dim_x`).

mat **getStateSpace**()

Returns

Matrix where each row is a discretized state.

void **saveStateSpace**()

Save the state space to `ss.h5` in HDF5 format.

void **loadStateSpace**(string filename)

Load a state space from an HDF5 file.

3.36.3 Input Space

void **setInputSpace**(vec lb, vec ub, vec eta)

Define the input space as a discretized hyper-rectangle.

mat **getInputSpace**()

Returns

Matrix where each row is a discretized input.

void **saveInputSpace**()

Save the input space to `is.h5`.

void **loadInputSpace**(string filename)

Load an input space from an HDF5 file.

3.36.4 Disturbance Space

void **setDisturbSpace**(vec lb, vec ub, vec eta)

Define the disturbance space as a discretized hyper-rectangle.

mat **getDisturbSpace**()

Returns

Matrix where each row is a discretized disturbance.

void **saveDisturbSpace**()

Save the disturbance space to `ws.h5`.

void **loadDisturbSpace**(string filename)

Load a disturbance space from an HDF5 file.

3.36.5 Target Space

void **setTargetSpace**(const function<bool(const vec&> &condition, bool remove)

Separate states matching the condition into a target region.

Parameters

- **condition** – Boolean function evaluated on each state.
- **remove** – If `true`, remove matching states from the state space.

mat **getTargetSpace**()

Returns

Matrix of target states.

void **saveTargetSpace**()

Save the target space to `ts.h5`.

void **loadTargetSpace**(string filename)

Load a target space from an HDF5 file.

3.36.6 Avoid Space

void **setAvoidSpace**(const function<bool(const vec&> &condition, bool remove)

Separate states matching the condition into an avoid region.

Parameters

- **condition** – Boolean function evaluated on each state.
- **remove** – If `true`, remove matching states from the state space.

mat **getAvoidSpace**()

Returns

Matrix of avoid states.

void **saveAvoidSpace**()

Save the avoid space to `as.h5`.

void **loadAvoidSpace**(string filename)

Load an avoid space from an HDF5 file.

3.36.7 Combined Target and Avoid

void **setTargetAvoidSpace**(const function<bool(const vec&> &target_condition, const
function<bool(const vec&> &avoid_condition, bool remove)

Set both target and avoid regions simultaneously.

3.36.8 Dynamics

void **setDynamics**(function<vec(const vec&, const vec&, const vec&)> d)

Set dynamics with state, input, and disturbance parameters.

void **setDynamics**(function<vec(const vec&, const vec&)> d)

Set dynamics with state and input parameters.

void **setDynamics**(function<vec(const vec&)> d)

Set dynamics with state parameter only (autonomous system).

3.36.9 Noise Configuration

void **setNoise**(*NoiseType* n, bool diagonal = true)

Set the noise distribution type.

Parameters

- **n** – `NoiseType::NORMAL` or `NoiseType::CUSTOM`.
- **diagonal** – If `true`, use diagonal covariance (default).

void **setNoise**(*NoiseType* n, bool diagonal, size_t monte_carlo_samples)

Set noise with Monte Carlo integration.

Parameters

monte_carlo_samples – Number of samples for MC integration.

void **setStdDev**(vec sig)

Set standard deviations for diagonal normal distribution.

void **setInvCovDet**(mat inv_cov, double det)

Set the inverse covariance matrix and determinant for full covariance normal.

void **setCustomDistribution**(double (*c)(double*, size_t, void*), size_t monte_carlo_samples)

Set a custom probability distribution function.

mat **getInvCov**()

Returns

The inverse covariance matrix.

double **getDet**()

Returns

The covariance matrix determinant.

vec **getStdDev**()

Returns

The standard deviation vector.

3.36.10 Transition Data

void **saveTransitionMatrix**()

Save the transition matrix to HDF5.

void **loadTransitionMatrix**(string filename)

Load a transition matrix from HDF5.

void **saveTargetTransitionVector**()

Save the target transition vector.

void **loadTargetTransitionVector**(string filename)

Load a target transition vector.

void **saveAvoidTransitionVector**()

Save the avoid transition vector.

void **loadAvoidTransitionVector**(string filename)

Load an avoid transition vector.

void **trackMDP**(bool store)

Enable or disable intermediate MDP data storage.

3.36.11 Stopping Condition

void **setStoppingCondition**(double eps)

Set the convergence threshold for infinite-horizon synthesis.

Parameters

eps – Convergence epsilon (default: 0.00001).

3.36.12 Enums and Structs

enum class **NoiseType**

enumerator **NORMAL**

Gaussian noise (diagonal or full covariance).

enumerator **CUSTOM**

User-defined distribution via `src/custom.cpp`.

struct **customParams**

Parameters passed to custom PDF functions.

vec **mean**

Result of applying dynamics to the current state.

vec **state_start**

Current state being evaluated.

function<vec(const vec&)> **dynamics1**

function<vec(const vec&, const vec&)> **dynamics2**

function<vec(const vec&, const vec&, const vec&)> **dynamics3**

System dynamics functions.

vec **input**

Current input.

vec **disturb**

Current disturbance.

vec **lb**

vec **ub**

Integration bounds.

vec **eta**

Discretization parameter.

3.37 IMPDP (class IMPDP)

class **IMPDP** : public *MDP*

Interval MDP class for abstraction, controller synthesis, and data management. Inherits all functionality from *MDP*.

Defined in `src/IMPDP.h` with implementations in `src/IMPDP.cpp`.

3.37.1 Constructor

Inherits the *MDP* constructor:

```
IMPDP imdp(dim_x, dim_u, dim_w);
```

3.37.2 Optimization Algorithm

void **setAlgorithm**(nlopt::algorithm alg)

Set the nonlinear optimization algorithm used during abstraction.

Parameters

alg – An NLOpt algorithm (default: `nlopt::LN_SBPLX`).

See [NLOpt Algorithms](#) for all options.

Abstraction

3.37.3 Transition Matrix

void **minTransitionMatrix**()

Compute the minimal transition probability matrix for state-to-state transitions.

void **maxTransitionMatrix**()

Compute the maximal transition probability matrix for state-to-state transitions.

void **transitionMatrixBounds**()

Compute both min and max transition matrices efficiently. Exploits sparsity: if max is zero, min is skipped. Use this instead of calling `minTransitionMatrix()` and `maxTransitionMatrix()` separately.

3.37.4 Target Transition Vector

void **minTargetTransitionVector**()

Compute the minimal transition probability vector to the target region.

void **maxTargetTransitionVector**()

Compute the maximal transition probability vector to the target region.

void **targetTransitionVectorBounds**()

Compute both min and max target vectors efficiently with sparsity exploitation.

3.37.5 Avoid Transition Vector

void **minAvoidTransitionVector**()

Compute the minimal transition probability vector to outside the state space (including the avoid region).

void **maxAvoidTransitionVector**()

Compute the maximal transition probability vector to outside the state space.

Note

Avoid vectors are always required, even when no avoid region is defined, as they capture transitions out of the state space.

Controller Synthesis

All synthesis functions take `IMDP_lower`:

- `true` — pessimistic (worst-case noise first, then fix for upper bound)
- `false` — optimistic (best-case noise first, then fix for lower bound)

3.37.6 Standard Synthesis

void **infiniteHorizonReachController**(bool `IMDP_lower`)

Synthesize a reachability (or reach-avoid) controller over an infinite horizon using the interval iteration algorithm.

void **infiniteHorizonSafeController**(bool `IMDP_lower`)

Synthesize a safety controller over an infinite horizon.

void **finiteHorizonReachController**(bool `IMDP_lower`, size_t `timeHorizon`)

Synthesize a reachability (or reach-avoid) controller for a finite number of steps using value iteration.

void **finiteHorizonSafeController**(bool `IMDP_lower`, size_t `timeHorizon`)

Synthesize a safety controller for a finite number of steps.

3.37.7 Sorted Synthesis (GPU-Compatible)

Sorted variants eliminate the GLPK dependency and support GPU execution (implemented in `src/GPU_synthesis.cpp`):

void **infiniteHorizonReachControllerSorted**(bool `IMDP_lower`)

void **infiniteHorizonSafeControllerSorted**(bool `IMDP_lower`)

void **finiteHorizonReachControllerSorted**(bool `IMDP_lower`, size_t `timeHorizon`)

void **finiteHorizonSafeControllerSorted**(bool `IMDP_lower`, size_t `timeHorizon`)

void **finiteHorizonReachControllerSortedStoreMDP**(bool `IMDP_lower`, size_t `timeHorizon`)

Sorted finite-horizon reachability with intermediate MDP data storage.

Save Functions

void **saveMinTransitionMatrix**()

void **saveMaxTransitionMatrix**()

void **saveMinTargetTransitionVector**()

void **saveMaxTargetTransitionVector**()

void **saveMinAvoidTransitionVector**()

void **saveMaxAvoidTransitionVector**()

void **saveController**()

Save the synthesized controller to `controller.h5`.

Load Functions

void **loadMinTransitionMatrix**(string filename)

void **loadMaxTransitionMatrix**(string filename)

void **loadMinTargetTransitionVector**(string filename)

void **loadMaxTargetTransitionVector**(string filename)

void **loadMinAvoidTransitionVector**(string filename)

void **loadMaxAvoidTransitionVector**(string filename)

void **loadController**(string filename)

Load a previously saved controller from HDF5.

3.38 GPU Synthesis

GPU-accelerated synthesis functions are implemented in `src/GPU_synthesis.cpp` using SYCL via AdaptiveCpp. These are the **sorted** variants of the standard synthesis algorithms.

3.38.1 Overview

The sorted approach:

1. Sorts states by transition probability, enabling efficient parallel reduction
2. Removes the dependency on the GLPK linear programming library
3. Enables execution on GPUs (NVIDIA, Intel, etc.) via SYCL

All sorted synthesis functions are methods of the *IMDP* class. See *IMDP* (*class IMPD*) for the full function signatures.

3.38.2 Usage

To use GPU synthesis:

1. **Compute abstraction on CPU** and save to HDF5 files
2. **Update the Makefile**: replace `IMDP.cpp` with `GPU_synthesis.cpp` and change `--acpp-targets` to your GPU target
3. **Load abstraction** from files and call sorted synthesis functions

```
IMDP imdp(dim_x, dim_u, dim_w);

// Load pre-computed data
imdp.loadStateSpace("ss.h5");
imdp.loadInputSpace("is.h5");
imdp.loadMinTransitionMatrix("minTM.h5");
imdp.loadMaxTransitionMatrix("maxTM.h5");
// ... load other vectors ...

// GPU-accelerated synthesis
imdp.infiniteHorizonReachControllerSorted(true);
imdp.saveController();
```

3.38.3 Makefile Configuration

```
CC = acpp
CFLAGS = --acpp-targets="cuda:sm_70" -O3 \
        -lnlopt -lm \
        -I/usr/include/hdf5/serial \
        -L/usr/lib/x86_64-linux-gnu/hdf5/serial \
        -lhdf5 -lgsl -lgslcblas \
        -DH5_USE_110_API -larmadillo

SRCS = example.cpp ../../src/GPU_synthesis.cpp ../../src/MDP.cpp
```

Note: `-lgsl` is not needed for sorted synthesis.

See [GPU Setup](#) for full GPU setup instructions.

3.39 IO Utilities (IMPACT_IO)

Utility functions for saving and loading Armadillo matrices and vectors to/from HDF5 format. Defined in `src/IO_utils.h`.

```
template<typename T>
```

```
void saveData(const T &data, const string &default_filename, const string &data_name)
```

Save an Armadillo matrix or vector to an HDF5 file.

Parameters

- **data** – The matrix or vector to save.
- **default_filename** – Output filename (e.g., "ss.h5").
- **data_name** – Human-readable name for error messages.

If `data` is empty, prints a warning and does not create a file.

```
template<typename T>
```

```
void loadData(T &data, const string &filename, const string &data_name)
```

Load an Armadillo matrix or vector from an HDF5 file.

Parameters

- **data** – The matrix or vector to load into.
- **filename** – Path to the HDF5 file.
- **data_name** – Human-readable name for error messages.

```
std::vector<int> getSortedIndices(const vec &values, bool ascending = true)
```

Return indices that would sort the input values. Used internally by GPU sorted synthesis functions.

Parameters

- **values** – Armadillo vector of values to sort.
- **ascending** – Sort order (default: ascending).

Returns

Vector of indices in sorted order.

3.40 Examples

IMPACT includes a comprehensive set of examples demonstrating different specifications, system dimensions, and features. Each example contains a configuration file (`.cpp`), a Makefile, and MATLAB plotting scripts.

Tip

Smaller examples (2D robot, 4D BAS) run on personal computers. **Larger examples** (5D+) may require more powerful machines.

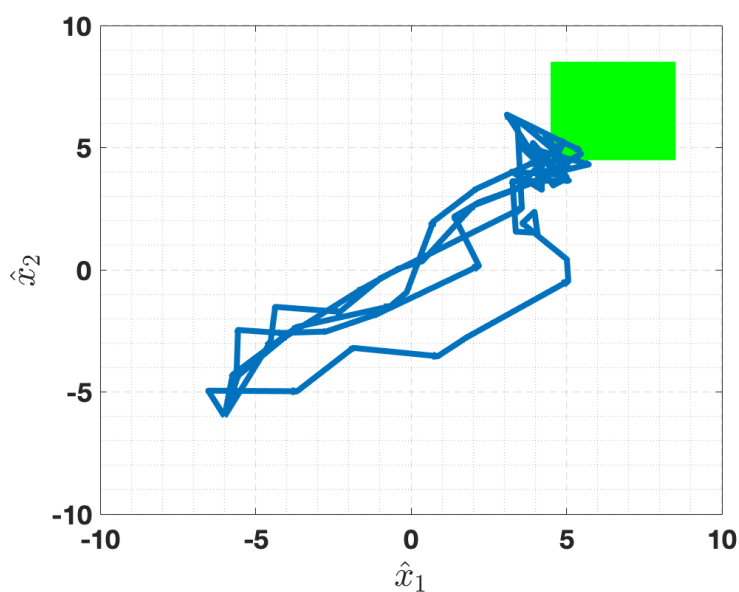
Example	Dim	Specifica- tion	Horizon	Directory
2D Robot (Undisturbed)	2	Reachability	Infinite	ex_2Drobot-R-U
2D Robot (Disturbed)	2	Reachability	Infinite	ex_2Drobot-R-D
2D Robot Reach-Avoid (Undis- turbed)	2	Reach-Avoid	Infinite	ex_2Drobot-RA-U
2D Robot Reach-Avoid (Dis- turbed)	2	Reach-Avoid	Infinite	ex_2Drobot-RA-D
3D Autonomous Vehicle	3	Reach-Avoid	Infinite	ex_3Dvehicle-RA
3D Room Temperature	3	Safety	Finite (10)	ex_3Droom-S
4D Building Automation	4	Safety	Finite (10)	ex_4DBAS-S
5D Room Temperature	5	Safety	Finite	ex_5Droom-S
7D Building Automation	7	Safety (Veri- fication)	Infinite	ex_7DBAS-S
14D Stochastic System	14	Safety	Finite	ex_14Dstochy-S
GPU Synthesis	–	Various	Various	ex_GPU
Custom PDF	–	Various	Various	ex_customPDF
Multivariate Normal	–	Various	Various	ex_multivariateNormalPDF
Load & Synthesize	–	Reachability	Various	ex_load_reach
Absorbing States	–	Safety	Various	ex_load_safe

3.40.1 Reachability

Synthesize a controller that drives the system to a target region.

2D Robot — Undisturbed (ex_2Drobot-R-U)

A 2D mobile robot navigates to a target region using synthesized control inputs.



System:

- **Dimensions:** 2D state (position), 2D input (speed, angle)

- **State space:** $[-10, 10]^2$, step size 1
- **Input space:** $[-1, 1]^2$, step size 0.2
- **Dynamics:** $x_{k+1} = x_k + 2u_0 \cos(u_1)$, $y_{k+1} = y_k + 2u_0 \sin(u_1)$
- **Noise:** Diagonal Gaussian, $\sigma = \sqrt{1/1.333}$
- **Target:** Square region $[5, 8]^2$
- **Specification:** Infinite-horizon reachability (pessimistic)

Run:

```
cd examples/ex_2Drobot-R-U
make && ./robot2D
```

This is the simplest example and serves as the basis for the *Quick Start*.

2D Robot — Disturbed (ex_2Drobot-R-D)

Same system with an additive disturbance input. This is the most thoroughly commented example and is recommended for understanding the configuration file format.

Additional features:

- 2D disturbance space
- Three-parameter dynamics: `function<vec(const vec&, const vec&, const vec&)>`
- Demonstrates both infinite and finite horizon (finite is commented out)

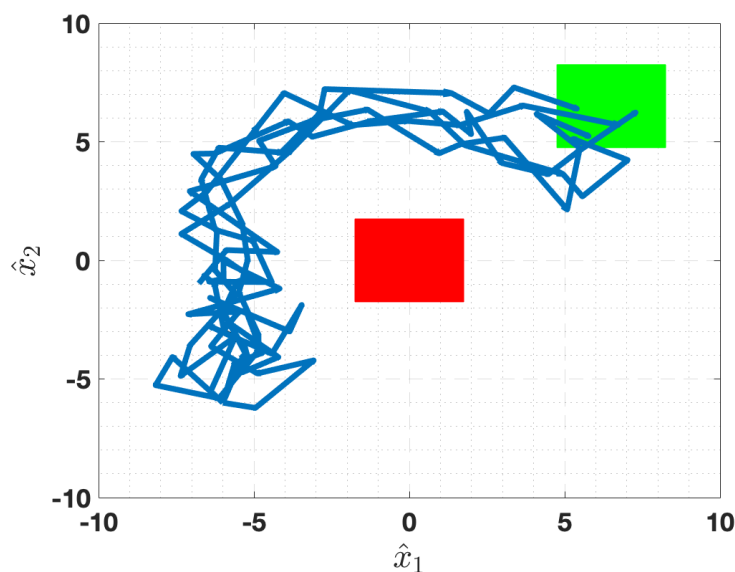
Run:

```
cd examples/ex_2Drobot-R-D
make && ./robot2D
```

3.40.2 Reach-Avoid

Synthesize a controller that reaches a target region while avoiding unsafe regions. IMPACT automatically detects reach-avoid when both target and avoid regions are defined.

2D Robot Reach-Avoid — Undisturbed (ex_2Drobot-RA-U)



System:

- **Dimensions:** 2D state, 2D input
- **Target:** Green region
- **Avoid:** Red region
- **Specification:** Infinite-horizon reach-while-avoid

The configuration uses `setTargetAvoidSpace()` to define both regions simultaneously.

Run:

```
cd examples/ex_2Drobot-RA-U
make && ./robot2D
```

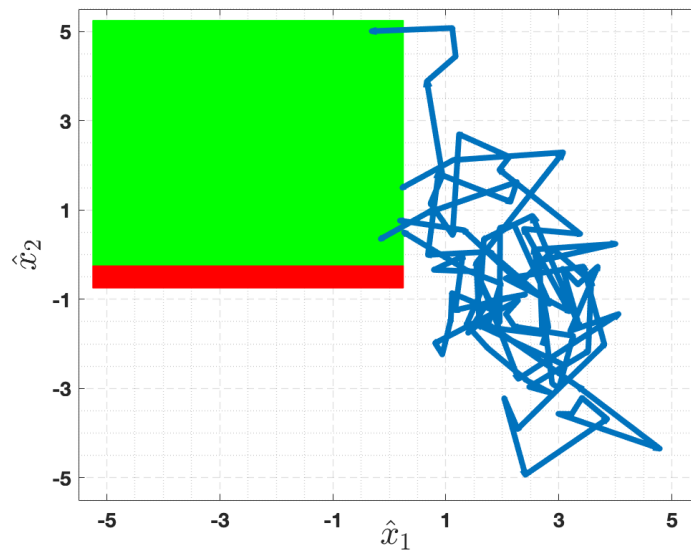
2D Robot Reach-Avoid — Disturbed (ex_2Drobot-RA-D)

Same reach-avoid problem with additive disturbance.

Run:

```
cd examples/ex_2Drobot-RA-D
make && ./robot2D
```

3D Autonomous Vehicle (ex_3Dvehicle-RA)



A 3D autonomous vehicle controller navigates to a target region while avoiding obstacles. This is a larger example that demonstrates scaling to higher dimensions.

System:

- **Dimensions:** 3D state, 2D input
- **Specification:** Infinite-horizon reach-while-avoid

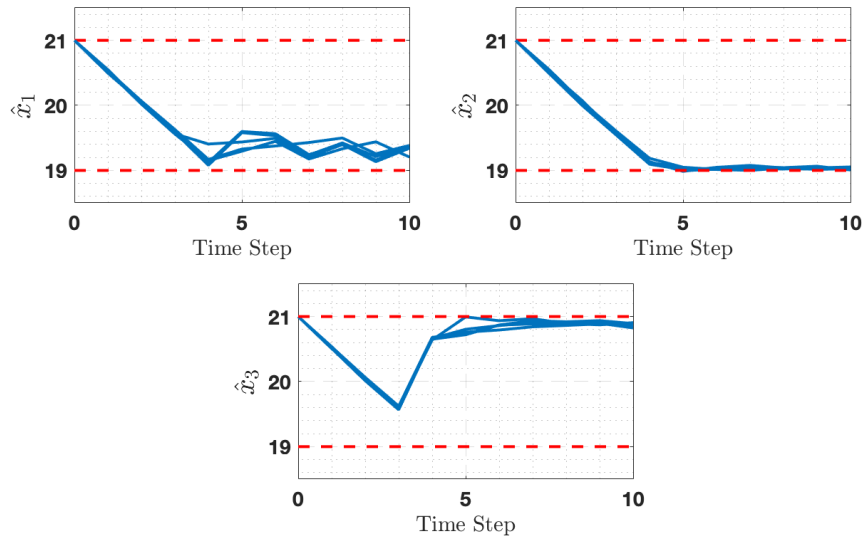
Run:

```
cd examples/ex_3Dvehicle-RA
make && ./vehicle3D
```

3.40.3 Safety

Synthesize a controller (or verify) that the system remains within a safe region. In safety specifications, the entire state space is the safe region by default.

3D Room Temperature (ex_3Droom-S)

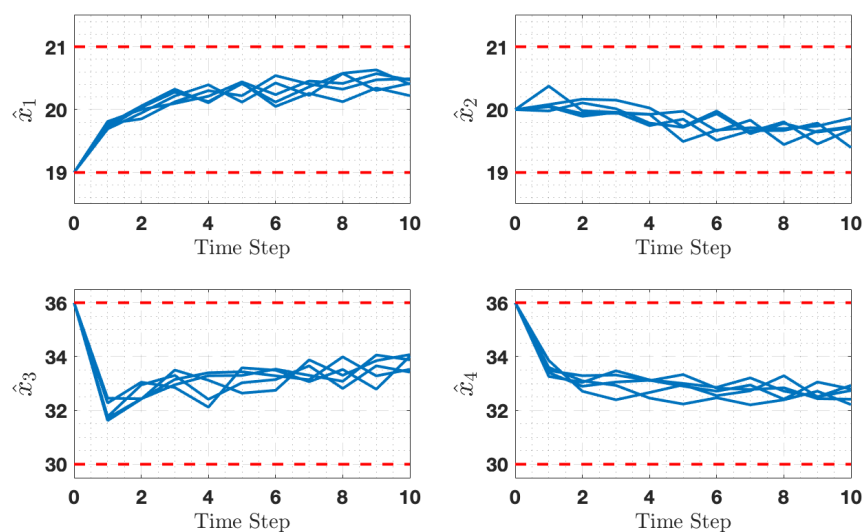


A 3D room temperature model is controlled to stay within safe temperature bounds (shown by red dashes) over a finite time horizon of 10 steps.

Run:

```
cd examples/ex_3Droom-S
make && ./room3D
```

4D Building Automation (ex_4DBAS-S)



A 4D building automation system controller maintains temperatures in 4 rooms within safe bounds over 10 time steps.

This is a small example suitable for personal computers.

Run:

```
cd examples/ex_4DBAS-S
make && ./bas4D
```

5D Room Temperature (ex_5Droom-S)

Extension of the 3D room temperature model to 5 dimensions. Requires more computational resources.

Run:

```
cd examples/ex_5Droom-S
make && ./room5D
```

7D Building Automation — Verification (ex_7DBAS-S)

A 7D building automation system with **no control inputs** (`dim_u = 0`). IMPaCT automatically performs **verification** (IMC) rather than synthesis, returning satisfaction probabilities for each state.

Run:

```
cd examples/ex_7DBAS-S
make && ./bas7D
```

14D Stochastic System (ex_14Dstochy-S)

The highest-dimensional example, demonstrating IMPaCT's scalability to 14 dimensions. This requires significant computational resources.

Run:

```
cd examples/ex_14Dstochy-S
make && ./stochy14D
```

3.40.4 Advanced

These examples demonstrate specialized features of IMPaCT.

GPU Synthesis (ex_GPU)

Demonstrates GPU-accelerated sorted synthesis. The abstraction is pre-computed on CPU and loaded from HDF5 files, then synthesis runs on the GPU.

See [GPU Setup](#) for setup instructions.

Run:

```
cd examples/ex_GPU
make && ./gpu_example
```

Custom Distributions (ex_customPDF)

Uses a user-defined probability distribution instead of the built-in Gaussian. The custom PDF is defined in `src/custom.cpp` and integrated via Monte Carlo.

Key configuration:

```
imdp.setNoise(NoiseType::CUSTOM, false, 1000);
```

See [Configuration](#) for details on custom distributions.

Run:

```
cd examples/ex_customPDF
make && ./customPDF
```

Multivariate Normal (ex_multivariateNormalPDF)

Demonstrates a Gaussian distribution with a **full covariance matrix** (non-diagonal, with off-diagonal correlations). Uses Monte Carlo integration.

Key configuration:

```
imdp.setNoise(NoiseType::NORMAL, false, 1000);
imdp.setInvCovDet(inv_cov, det);
```

Run:

```
cd examples/ex_multivariateNormalPDF
make && ./multivariateNormal
```

Loading Pre-Computed Data (ex_load_reach)

Shows how to load transition matrices computed externally (e.g., from data-driven methods or other tools) and use IMPaCT for synthesis only.

This makes IMPaCT useful beyond model-based approaches — any method that produces transition probability bounds can feed into IMPaCT’s synthesis algorithms.

Run:

```
cd examples/ex_load_reach
make && ./load_reach
```

Absorbing States (ex_load_safe)

Demonstrates handling absorbing states when loading pre-computed matrices for safety specifications.

Run:

```
cd examples/ex_load_safe
make && ./load_safe
```

See *Loading and Saving* for details on the save/load API and debugging tips.

3.40.5 By Specification

Reachability Synthesize controllers to reach a target region, with and without disturbance.

Reachability **Reach-Avoid** Reach a target while avoiding unsafe regions.

Reach-Avoid **Safety** Keep the system within a safe region over finite or infinite horizons.

Safety **Advanced** GPU synthesis, custom distributions, loading pre-computed data.

Advanced

3.41 Citing

If you use IMPaCT in your research, please cite:

3.41.1 BibTeX

QEST+FORMATS 2024 (primary):

```
@inproceedings{wooding2024qestformats,
  title      = {IMPACT: Interval MDP Parallel Construction for Controller
                Synthesis of Large-Scale Stochastic Systems},
  author     = {Ben Wooding and Abolfazl Lavaei},
  booktitle  = {International Conference on Quantitative Evaluation of
                Systems and Formal Modeling and Analysis of Timed Systems},
  pages      = {249--267},
  year       = {2024},
  organization = {Springer}
}
```

HSCC 2024 Poster:

```
@inproceedings{wooding2024hsc,
  title      = {IMPACT: A parallelized software tool for IMDP construction
                and controller synthesis with convergence guarantees},
  author     = {Wooding, Ben and Lavaei, Abolfazl},
  booktitle  = {Proceedings of the 27th ACM International Conference on
                Hybrid Systems: Computation and Control},
  pages      = {1--2},
  year       = {2024}
}
```

3.41.2 Related Papers

- [arXiv preprint](#)
- [HSCC 2024 Poster](#)
- [QEST+FORMATS 2024](#)

3.41.3 Authors

- Ben Wooding (ORCID: 0000-0003-1713-717X)
- Abolfazl Lavaei

3.41.4 License

IMPACT is licensed under the [Creative Commons Attribution 4.0 International License](#).

C

customParams (C++ struct), 70
 customParams::disturb (C++ member), 70
 customParams::dynamics1 (C++ member), 70
 customParams::dynamics2 (C++ member), 70
 customParams::dynamics3 (C++ member), 70
 customParams::eta (C++ member), 70
 customParams::input (C++ member), 70
 customParams::lb (C++ member), 70
 customParams::mean (C++ member), 70
 customParams::state_start (C++ member), 70
 customParams::ub (C++ member), 70

I

IMDP (C++ class), 71
 IMDP::finiteHorizonReachController (C++ function), 72
 IMDP::finiteHorizonReachControllerSorted (C++ function), 72
 IMDP::finiteHorizonReachControllerSortedStore (C++ function), 72
 IMDP::finiteHorizonSafeController (C++ function), 72
 IMDP::finiteHorizonSafeControllerSorted (C++ function), 72
 IMDP::infiniteHorizonReachController (C++ function), 72
 IMDP::infiniteHorizonReachControllerSorted (C++ function), 72
 IMDP::infiniteHorizonSafeController (C++ function), 72
 IMDP::infiniteHorizonSafeControllerSorted (C++ function), 72
 IMDP::loadController (C++ function), 73
 IMDP::loadMaxAvoidTransitionVector (C++ function), 73
 IMDP::loadMaxTargetTransitionVector (C++ function), 73
 IMDP::loadMaxTransitionMatrix (C++ function), 73
 IMDP::loadMinAvoidTransitionVector (C++ function), 73
 IMDP::loadMinTargetTransitionVector (C++ function), 73
 IMDP::loadMinTransitionMatrix (C++ function), 73

IMDP::maxAvoidTransitionVector (C++ function), 71
 IMDP::maxTargetTransitionVector (C++ function), 71
 IMDP::maxTransitionMatrix (C++ function), 71
 IMDP::minAvoidTransitionVector (C++ function), 71
 IMDP::minTargetTransitionVector (C++ function), 71
 IMDP::minTransitionMatrix (C++ function), 71
 IMDP::saveController (C++ function), 72
 IMDP::saveMaxAvoidTransitionVector (C++ function), 72
 IMDP::saveMaxTargetTransitionVector (C++ function), 72
 IMDP::saveMaxTransitionMatrix (C++ function), 72
 IMDP::saveMinAvoidTransitionVector (C++ function), 72
 IMDP::saveMinTargetTransitionVector (C++ function), 72
 IMDP::saveMinTransitionMatrix (C++ function), 72
 IMDP::setAlgorithm (C++ function), 71
 IMDP::targetTransitionVectorBounds (C++ function), 71
 IMDP::transitionMatrixBounds (C++ function), 71
 IMPaCT_IO::getSortedIndices (C++ function), 74
 IMPaCT_IO::loadData (C++ function), 74
 IMPaCT_IO::saveData (C++ function), 74

M

MDP (C++ class), 67
 MDP::getAvoidSpace (C++ function), 68
 MDP::getDet (C++ function), 69
 MDP::getDisturbSpace (C++ function), 68
 MDP::getInputSpace (C++ function), 67
 MDP::getInvCov (C++ function), 69
 MDP::getStateSpace (C++ function), 67
 MDP::getStdDev (C++ function), 69
 MDP::getTargetSpace (C++ function), 68
 MDP::loadAvoidSpace (C++ function), 68
 MDP::loadAvoidTransitionVector (C++ function), 70
 MDP::loadDisturbSpace (C++ function), 68

MDP::loadInputSpace (C++ *function*), 67
 MDP::loadStateSpace (C++ *function*), 67
 MDP::loadTargetSpace (C++ *function*), 68
 MDP::loadTargetTransitionVector (C++ *function*), 69
 MDP::loadTransitionMatrix (C++ *function*), 69
 MDP::MDP (C++ *function*), 67
 MDP::saveAvoidSpace (C++ *function*), 68
 MDP::saveAvoidTransitionVector (C++ *function*), 70
 MDP::saveDisturbSpace (C++ *function*), 68
 MDP::saveInputSpace (C++ *function*), 67
 MDP::saveStateSpace (C++ *function*), 67
 MDP::saveTargetSpace (C++ *function*), 68
 MDP::saveTargetTransitionVector (C++ *function*), 69
 MDP::saveTransitionMatrix (C++ *function*), 69
 MDP::setAvoidSpace (C++ *function*), 68
 MDP::setCustomDistribution (C++ *function*), 69
 MDP::setDisturbSpace (C++ *function*), 68
 MDP::setDynamics (C++ *function*), 69
 MDP::setInputSpace (C++ *function*), 67
 MDP::setInvCovDet (C++ *function*), 69
 MDP::setNoise (C++ *function*), 69
 MDP::setStateSpace (C++ *function*), 67
 MDP::setStdDev (C++ *function*), 69
 MDP::setStoppingCondition (C++ *function*), 70
 MDP::setTargetAvoidSpace (C++ *function*), 68
 MDP::setTargetSpace (C++ *function*), 68
 MDP::trackMDP (C++ *function*), 70

N

NoiseType (C++ *enum*), 70
 NoiseType::CUSTOM (C++ *enumerator*), 70
 NoiseType::NORMAL (C++ *enumerator*), 70